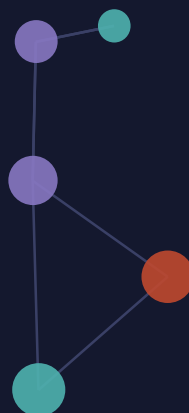


Knowledge Graphs: From First Principles to Production





How to use this tutorial

This deck is designed to be studied in order: every later stage assumes the vocabulary built earlier.

1 Who it is for

Data scientists, machine learning engineers, data engineers, architects and analysts who want to design, build and operate knowledge graphs with rigour and confidence.

2 Prerequisites

Comfort with data modelling and basic SQL. Some Python helps in later stages. No prior Semantic Web knowledge is assumed: every concept is built from first principles.

3 How it is organised

Four parts: the background and need, the formal foundations, a twelve stage development lifecycle studied in depth, and a synthesis with tools, pitfalls and a maturity model.

4 Reading the diagrams

Ovals are entities, rounded rectangles are classes, amber boxes are literal values, arrows are directed relations, and dashed coral arrows are inferred, predicted or problematic facts.



The journey through this tutorial



Part I: Background

The data integration crisis, a history of knowledge representation, definitions, types of knowledge graphs and the case for building one. Slides 5 to 22.

Part II: Foundations

Graph theory, RDF and property graphs, RDFS and OWL semantics, identity, reification, qualifiers and Linked Data. Slides 23 to 40.

Part III: Lifecycle

Twelve development stages from requirements to governance, each with a stage card and eight deep dive slides. Slides 41 to 152.

Part IV: Synthesis

An end to end walkthrough, the tool landscape, common pitfalls, a maturity model, glossary and references. Slides 153 to 160.

Running example: throughout Part III we build a supplier risk knowledge graph for a manufacturer, so every stage produces a concrete artefact.



By the end you will be able to



Explain: what a knowledge graph is, how it differs from databases and ontologies, and why organisations need one.



Construct: the graph through mappings, NLP and LLM extraction, entity resolution and knowledge fusion.



Model: knowledge with RDF triples, property graphs, RDFS and OWL, choosing the right formalism.



Operate: storage, reasoning, SHACL validation and quality measurement at production scale.



Scope: a project with competency questions, use case canvases and measurable success criteria.



Complete: missing knowledge with embeddings and graph neural networks, evaluated with MRR and Hits@k.



Design: an ontology with established methodologies, design patterns and reusable vocabularies.



Serve and govern: the graph through SPARQL, APIs and GraphRAG, with versioning, provenance and FAIR practice.

PART I

Background and the Need for Knowledge Graphs

Why connected, meaningful data has become a strategic necessity.

- The data integration crisis
- Sixty years of knowledge representation
- Defining the knowledge graph
- Types, scale and characteristics
- Why organisations need knowledge graphs
- Knowledge graphs in the age of LLMs





The problem: data is abundant, meaning is scarce

175 ZB

IDC projection of the global datasphere by 2025, from its 2018 Data Age 2025 study.

80 to 90%

Widely cited estimate of the share of enterprise data that is unstructured: text, email, documents and media.

Many silos

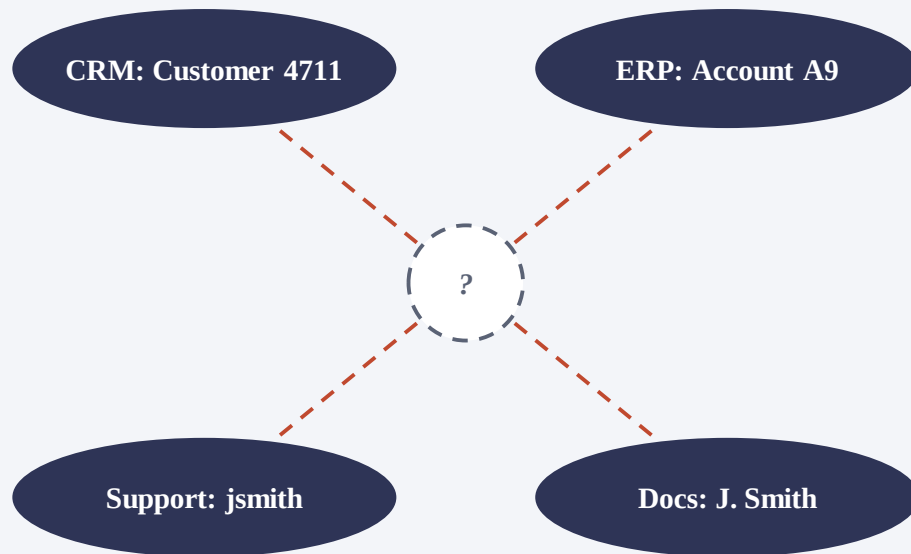
Large organisations run hundreds of applications, each with its own schema, identifiers and vocabulary.

- Volume is rarely the bottleneck; meaning is. Facts about the same customer, product or molecule are scattered across systems that share neither identifiers nor definitions.
- Every question that crosses a system boundary triggers a bespoke integration project: extract, reconcile, join, and then repeat for the next question.
- Knowledge graphs address this by giving every thing a global identifier, every relation an explicit meaning, and every fact a common, connectable shape.

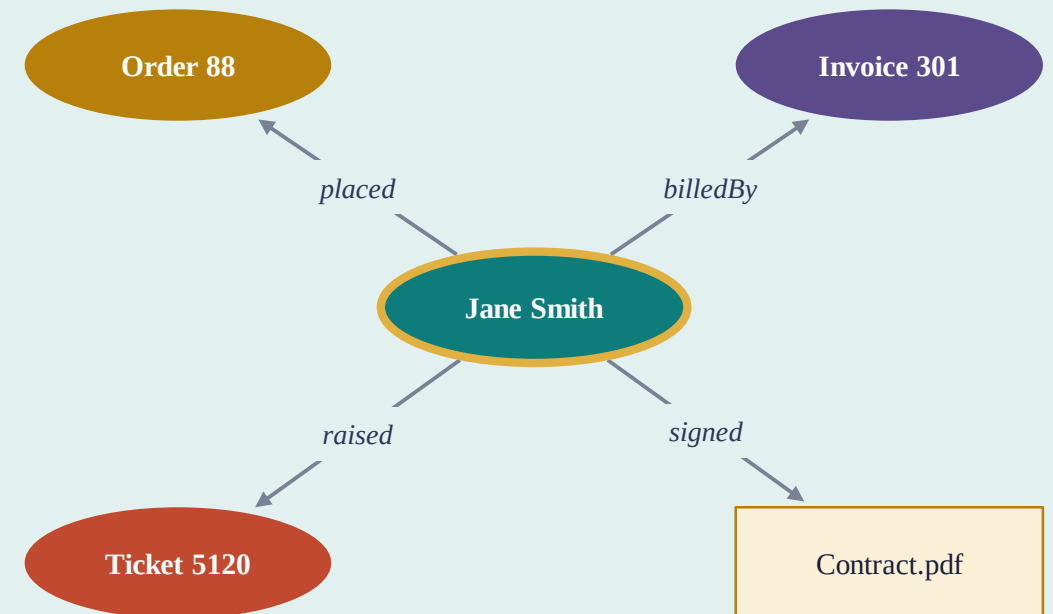


From siloed records to connected knowledge

Siloed: four systems, four identities



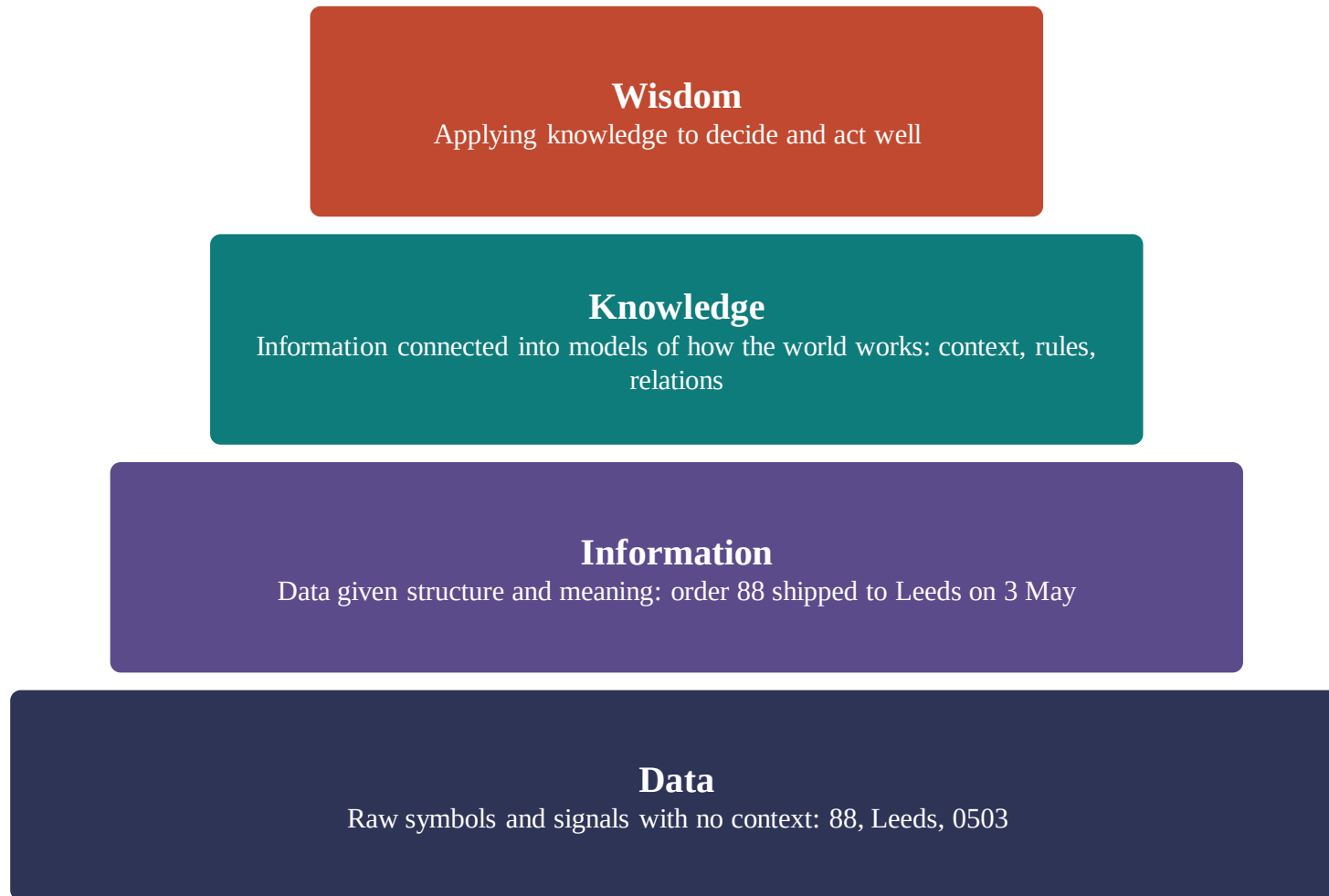
Connected: one entity, many perspectives



Key idea: the graph does not replace the source systems; it links their records to shared entities so questions can cross boundaries.



Where knowledge sits: the DIKW hierarchy

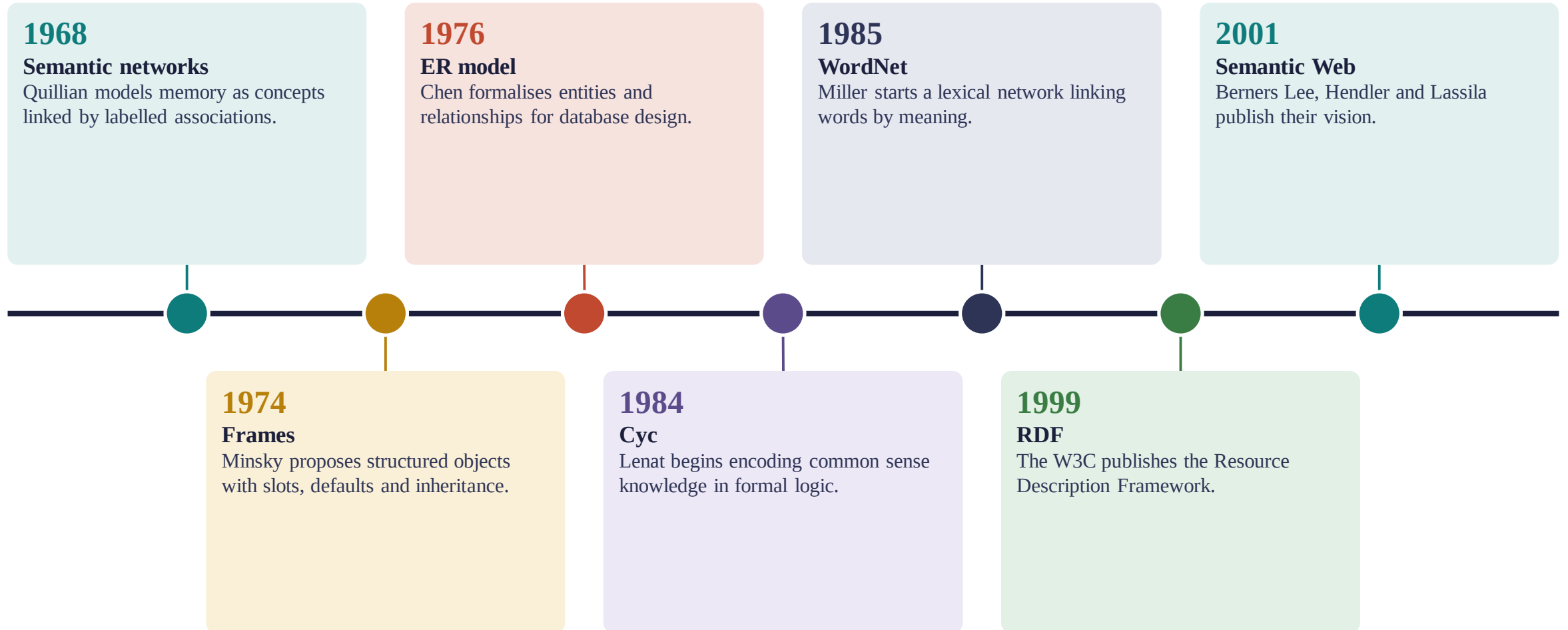


KEY POINTS

- **Origin:** the hierarchy is usually attributed to Ackoff (1989), though its roots are older.
- **Transition:** knowledge graphs operate at the step from information to knowledge by connecting facts and attaching formal meaning.
- **Machine usable:** the defining feature is that the context is explicit, so software can use it, not only people.
- **Caution:** the pyramid is a teaching metaphor, not a formal theory; its value is in the distinction between isolated facts and connected, interpreted knowledge.

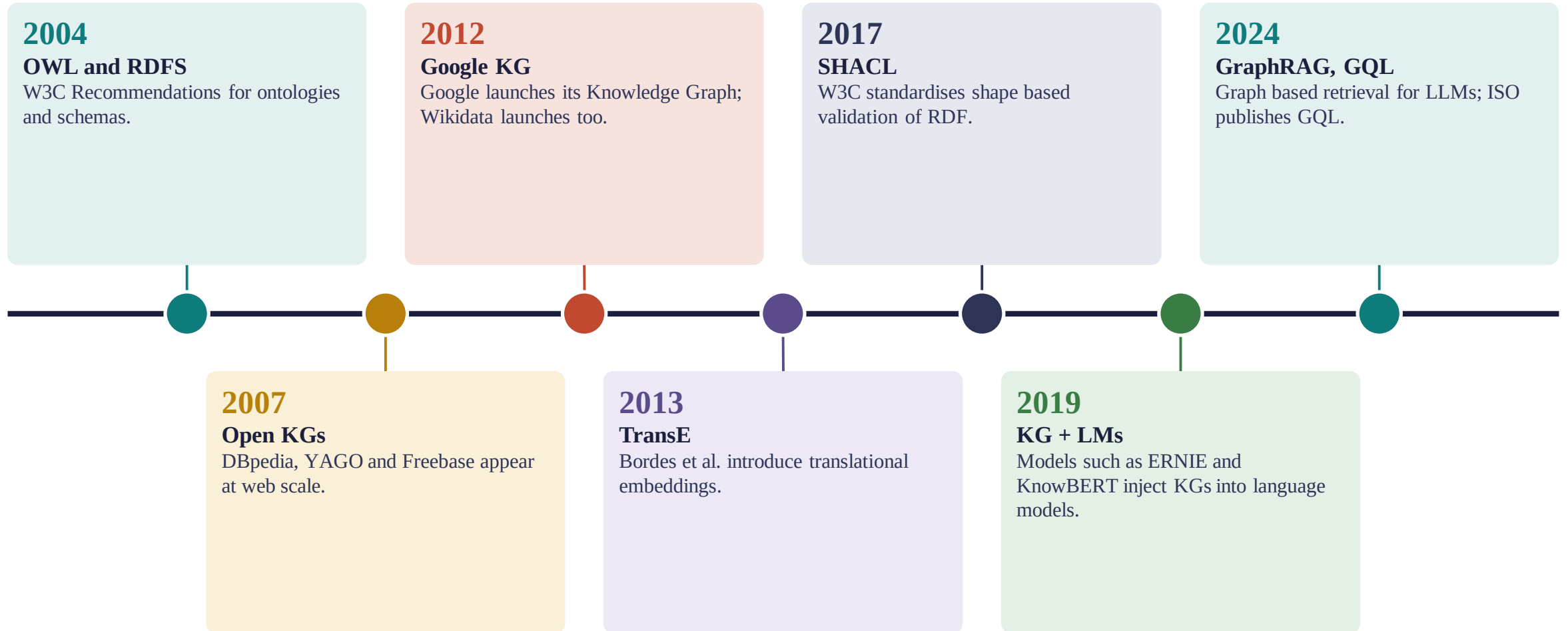


Roots of knowledge representation, 1968 to 2001



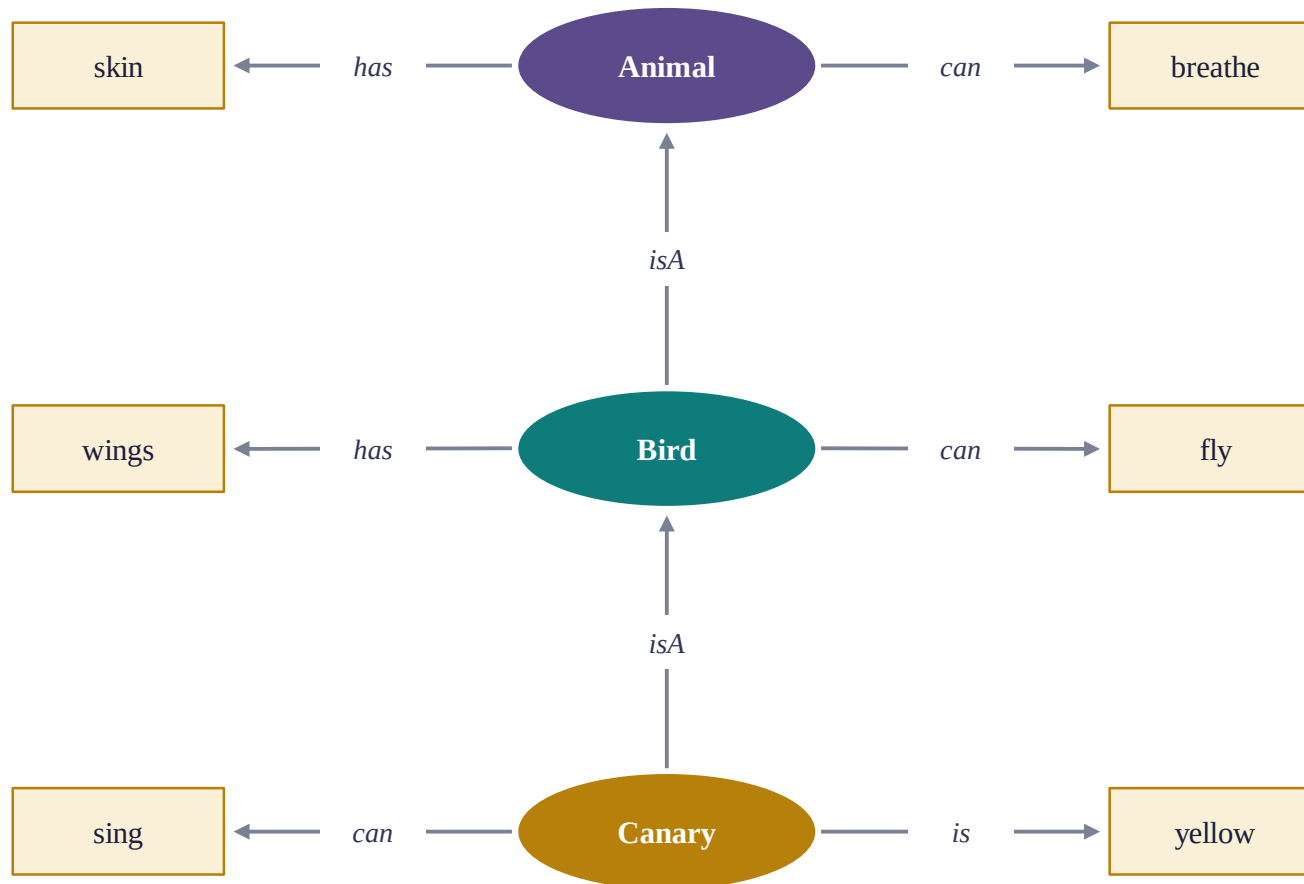


The modern era, 2004 to 2024





Semantic networks: the first knowledge graphs



ANNOTATIONS

- **Inheritance:** a canary can fly because it is a bird; the property is stored once and inherited down the isA chain.
- **Cognitive economy:** Collins and Quillian (1969) found people verify facts stored further up the hierarchy more slowly, supporting the model.
- **Limitation:** links had no formal semantics. Woods (1975), in What is in a link, showed that isA meant different things to different authors.
- **Legacy:** modern KGs keep the network form but add formal, machine checkable meaning to every node and edge.



The Semantic Web layer cake

Trust and proof

Provenance, signatures and justification of answers

Rules and logic

RIF, SWRL and unifying logic for inference beyond ontologies

Ontologies: OWL

Rich class and property axioms grounded in description logic

Query: SPARQL

Standard pattern matching query language over RDF graphs

Schema: RDFS

Classes, subclasses, domains and ranges

Data model: RDF

Everything is a set of subject, predicate, object triples

Identifiers

IRIs and Unicode: global names for every thing and relation

KEY POINTS

- **Design principle:** each layer builds on the one below and adds expressive power.
- **Standards:** everything shown up to SPARQL, plus SHACL for validation, is a W3C Recommendation used in production today.
- **Reality check:** the upper layers of trust and proof remain research topics; production KGs rely mostly on the lower five.
- **Relevance:** even property graph systems borrow these ideas: global identifiers, schemas and standard query languages.



Google, 2012: things, not strings

“

Things, not strings.

Amit Singhal, Google, announcing the Knowledge Graph, May 2012

- Search moved from matching keywords to recognising the entities that queries refer to: the city, the painter, the film.
- At launch Google reported more than 500 million objects and over 3.5 billion facts about them and their relations.
- Knowledge panels, direct answers and related entity suggestions all became possible because the entity was the unit of retrieval.
- The phrase knowledge graph became the industry name for any large, graph structured knowledge base, even though the ideas were decades older.



What exactly is a knowledge graph?

There is no single agreed definition. Four influential views, paraphrased, and the working definition used here:

1 Hogan et al. (2021)

A graph of data intended to accumulate and convey knowledge of the real world, where nodes are entities of interest and edges are relations between them.

2 Paulheim (2017)

Describes real world entities and their interrelations as a graph, defines classes and relations in a schema, allows arbitrary entities to relate, and covers various domains.

3 Ehrlinger and Wöß (2016)

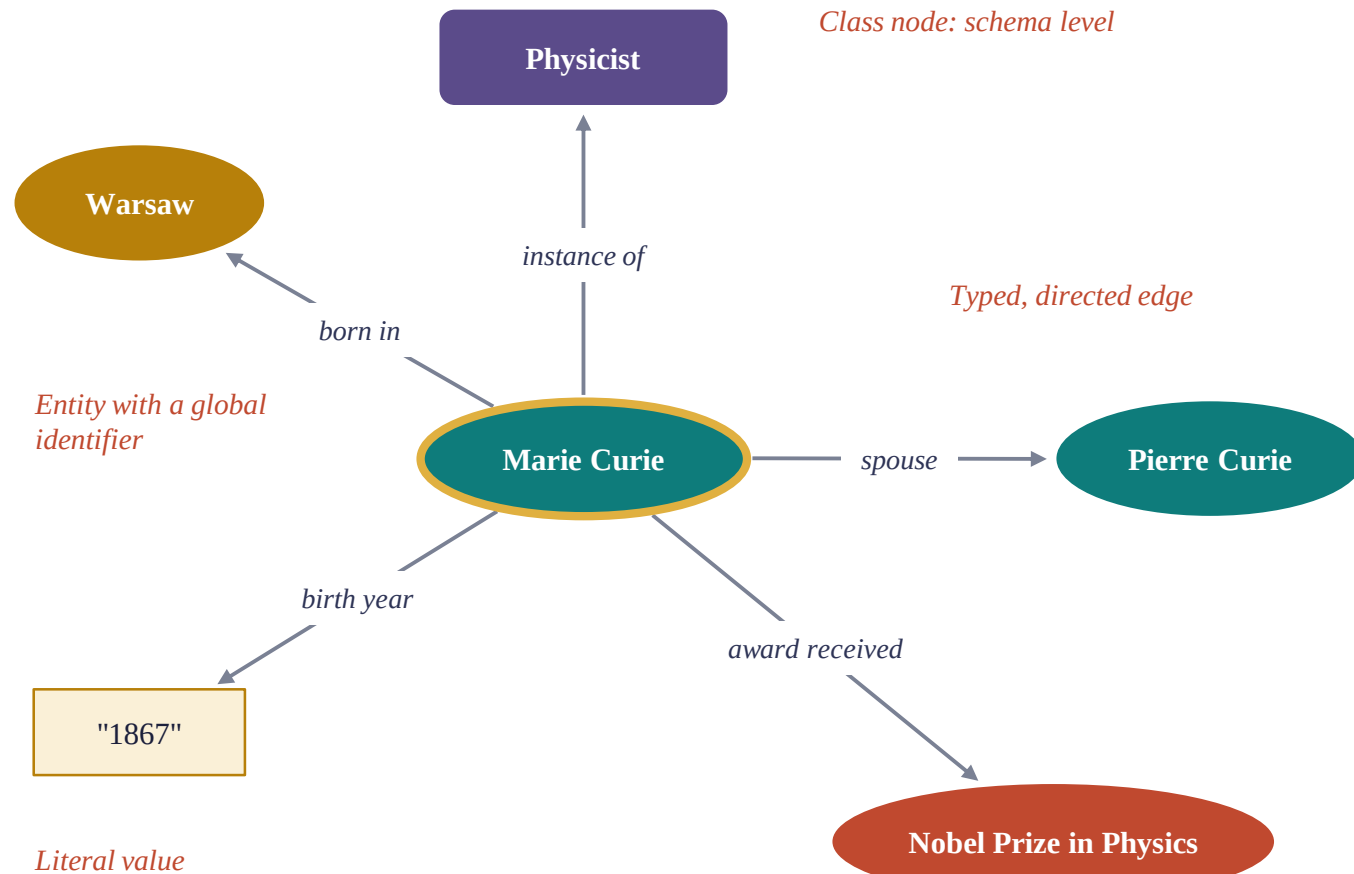
Acquires and integrates information into an ontology and applies a reasoner to derive new knowledge from it.

★ Working definition

A graph of entities and typed relations, governed by an explicit schema or ontology, integrated from many sources, and maintained so machines and people can query, reason over and trust it.



Anatomy of a knowledge graph



ANNOTATIONS

- **Entities:** real world things, each with one stable identifier (an IRI or ID).
- **Classes:** types that group entities and carry the schema, such as Physicist.
- **Relations:** named, directed edges whose meaning is defined in the schema.
- **Literals:** plain values such as numbers, dates and strings, attached to entities.
- **Schema:** the rules that say which relations are allowed between which classes.
- **Everything is data:** the schema itself can be stored and queried as part of the graph.



Knowledge graphs compared with related technologies

Technology	What it is	Schema	Question it answers best
Relational database	Tables of rows linked by foreign keys, optimised for transactions	Fixed, defined up front	What is the balance of account 42?
Data warehouse	Integrated, historical tables modelled for analytics	Star or snowflake schema	What were sales by region last quarter?
Ontology	A formal specification of classes, relations and axioms in a domain	Is the schema	What kinds of things exist and how may they relate?
Knowledge base	Any store of facts and rules used by a reasoning system	Varies widely	What follows from what we know?
Knowledge graph	Instance data plus ontology in graph form, integrated from many sources	Explicit but evolvable	How is this entity connected to everything else, and why?

Relationship: a knowledge graph is roughly an ontology plus the instance data it describes, stored and served as a graph.



Types of knowledge graphs

1 Encyclopedic, open

Wikidata, DBpedia and YAGO: broad, general knowledge about millions of entities, openly licensed and community maintained.

2 Enterprise

Customers, products, suppliers, employees and documents inside one organisation; for example product graphs at large retailers.

3 Domain specific

Deep coverage of one field: UMLS and Hetionet in biomedicine, FIBO in finance, GeoNames for places.

4 Commonsense

ConceptNet, ATOMIC and Cyc capture everyday knowledge: that cups hold liquids, or that people eat when hungry.

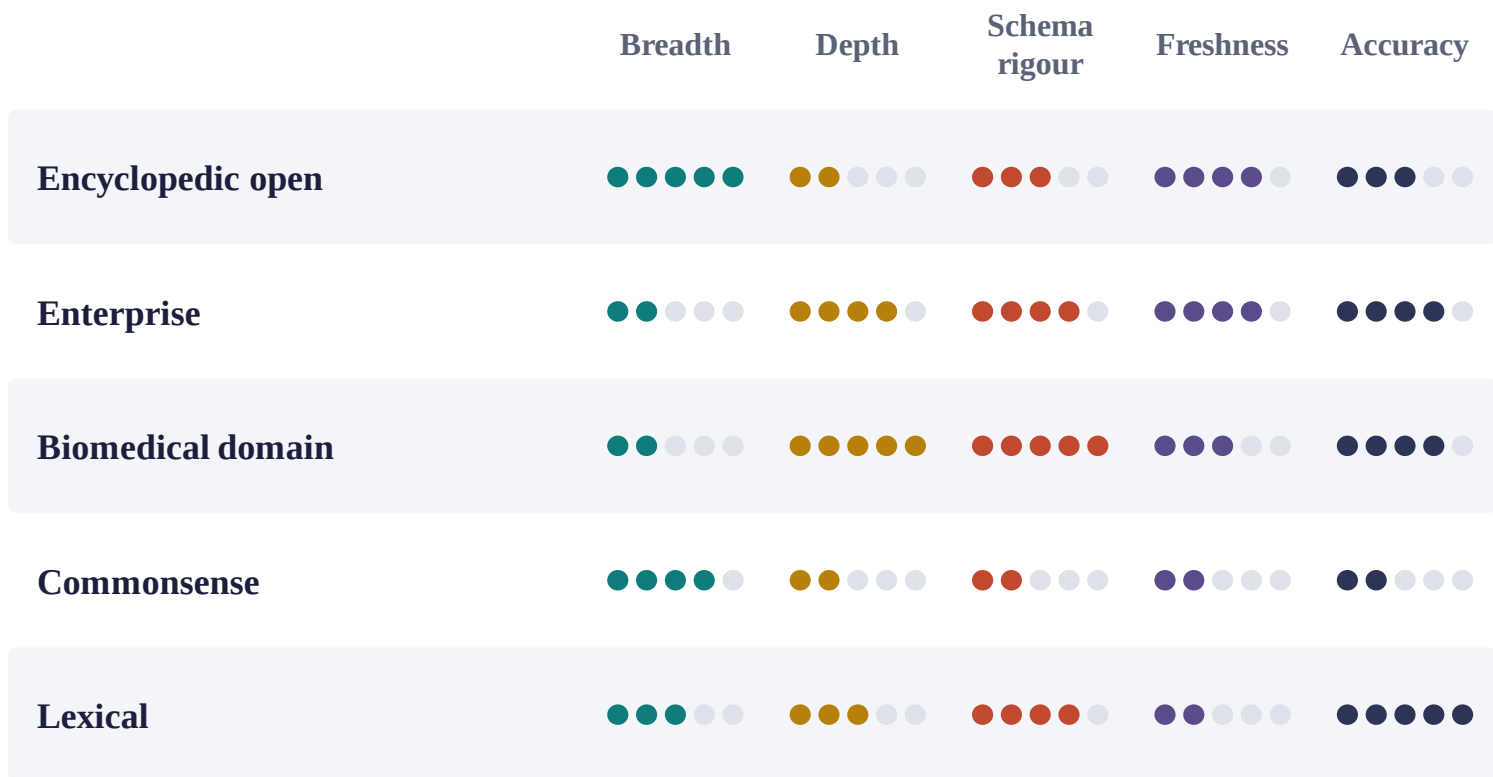
5 Lexical and linguistic

WordNet, BabelNet and FrameNet link words, senses and frames; widely used in NLP pipelines.

6 Event and temporal

Graphs of events with participants, times and places, such as GDELT; facts carry validity intervals.

Characteristics of knowledge graph types



INTERPRETATION

- **Trade off:** breadth and depth compete for the same curation budget; almost no graph maximises both.
- **Rigour:** domain graphs such as biomedical ontologies invest heavily in formal definitions because errors are costly.
- **Implication:** enterprise graphs usually combine a deep internal core with links into broad open graphs for context.

Illustrative qualitative assessment of typical instances; individual graphs vary widely.



Eight reasons organisations build knowledge graphs

1 Integration

Link data across silos through shared entities instead of point to point pipelines.

2 Context

Answer questions that depend on relationships several hops away.

3 Shared semantics

One agreed meaning for customer, product or risk across teams.

4 Reasoning

Derive facts nobody entered, such as indirect ownership or exposure.

5 Explainability

Every answer traces back through named relations and sources.

6 Flexibility

Add new entity types and relations without schema migrations.

7 Discovery

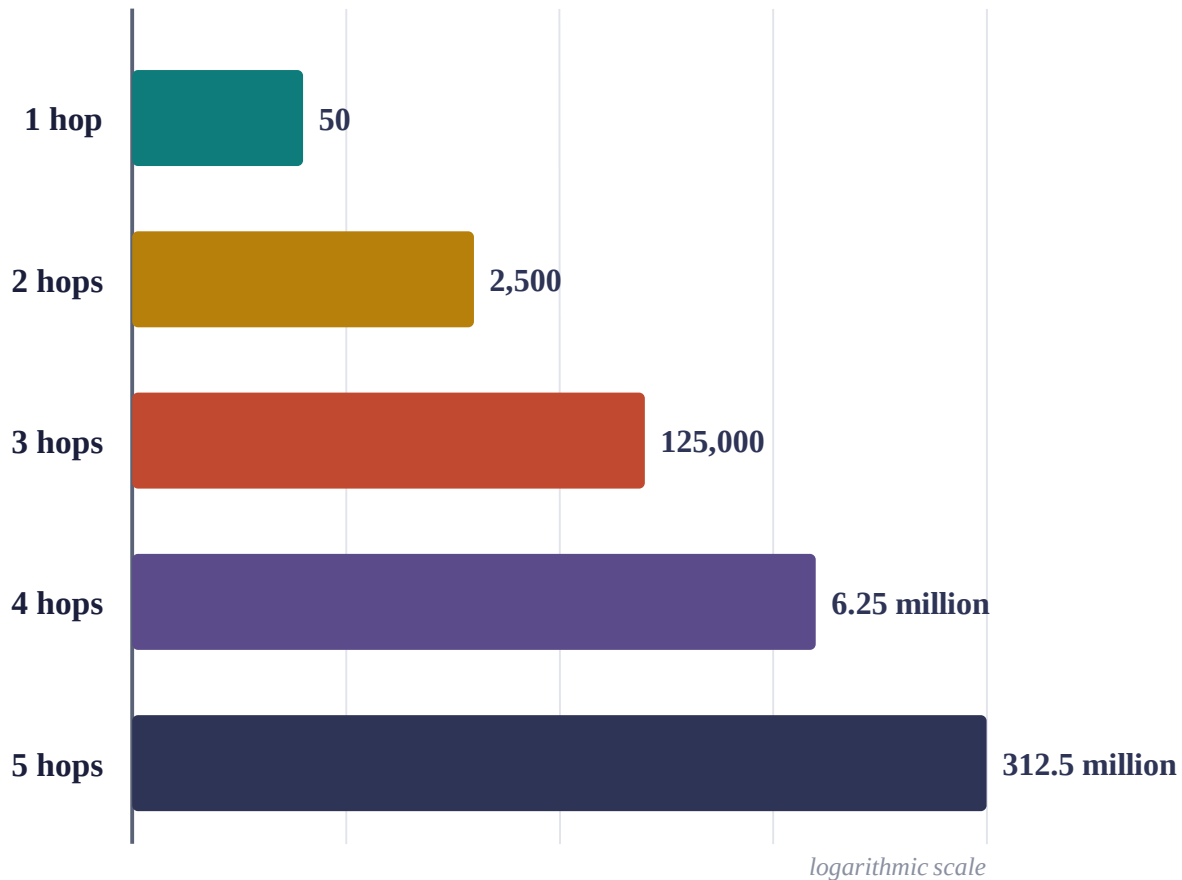
Surface unexpected connections: hidden links, clusters and paths.

8 AI grounding

Supply LLMs and ML models with accurate, current, structured facts.



Why multihop questions strain tables



Paths explored from one node when every node has 50 neighbours. In SQL each hop is one more self join.

READING THE CHART

- **Combinatorics:** with average degree d , the frontier after k hops grows like d to the power k , so intermediate results explode.
- **Relational cost:** each hop is a join that looks up keys in a global index, and the number of joins must be fixed in the query text.
- **Graph approach:** graph engines follow stored adjacency from node to node, and query languages express variable length paths directly.
- **Honest caveat:** neither model escapes the combinatorics; good graph queries still bound depth and filter early.



Knowledge graphs and large language models

Large language models

- **Strength:** fluent language, broad world knowledge, flexible reasoning over text.
- **Weakness:** can hallucinate plausible but false facts.
- **Weakness:** knowledge frozen at a training cutoff and hard to update.
- **Weakness:** no reliable citations or audit trail for a claim.
- **Weakness:** private enterprise facts are absent unless supplied.

vs

Knowledge graphs

- **Strength:** explicit, verifiable, attributable facts with provenance.
- **Strength:** updated in minutes, not retraining cycles.
- **Strength:** precise multihop and aggregate queries.
- **Weakness:** costly to build and always incomplete.
- **Weakness:** require formal queries that most users cannot write.

Complementary: KGs ground LLMs with current, attributable facts; LLMs give KGs natural language access and speed up construction.



Where knowledge graphs deliver value

1 Search and question answering

Entity aware search, knowledge panels and direct answers, from web search to internal document portals.

2 Recommendation

Paths between users, items and attributes explain why an item is suggested and ease cold start problems.

3 Fraud and financial crime

Shared devices, addresses and ownership chains expose rings and beneficial owners hidden behind shell companies.

4 Drug discovery

Graphs of genes, proteins, diseases and compounds support target identification and drug repurposing.

5 Supply chain

Multitier supplier, part and facility graphs reveal exposure to disruption, sanctions and single points of failure.

6 Enterprise data fabric

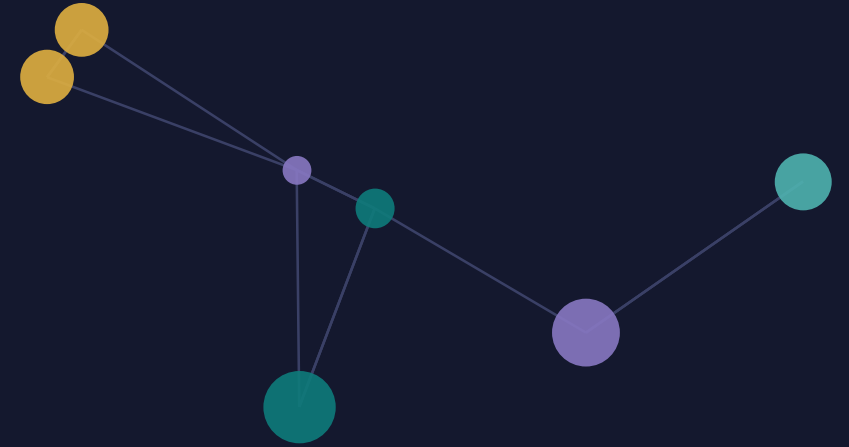
A semantic layer over many databases gives analysts one consistent, governed view of core business entities.

PART II

Formal Foundations

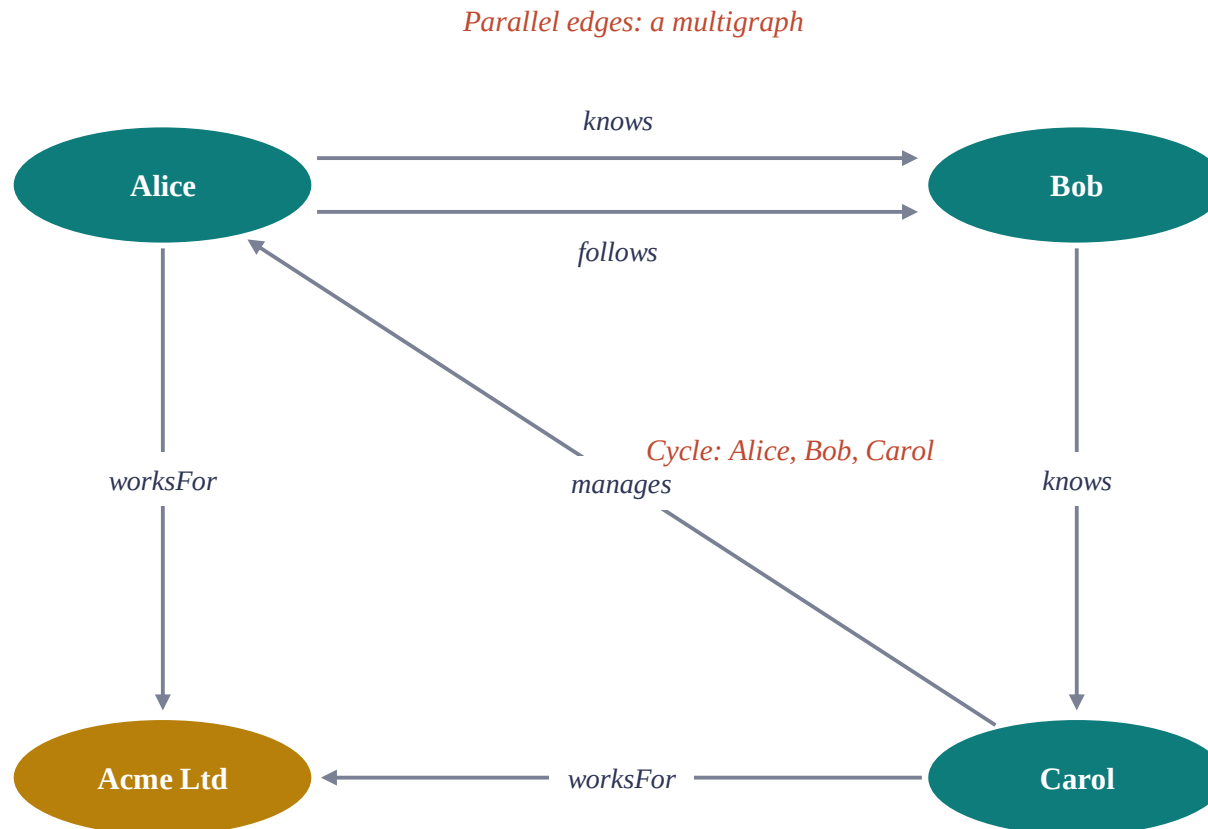
The data models, languages and logic that give a knowledge graph its meaning.

- Graph theory essentials
- RDF triples and property graphs
- RDFS and OWL semantics
- Open and closed world reasoning
- Identity, reification and qualifiers
- Linked Data principles





Graph theory essentials for knowledge graphs



ANNOTATIONS

- **Graph:** $G = (V, E)$, a set of vertices V and edges E between them.
- **Directed:** every edge has a direction; worksFor points from person to company.
- **Labelled:** every edge carries a type, which is what makes the graph semantic.
- **Multigraph:** two nodes may be joined by several edges with different labels.
- **Degree, path, cycle:** counts of edges, sequences of edges, and paths returning to their start.
- **In short:** a knowledge graph is a directed, edge labelled multigraph.



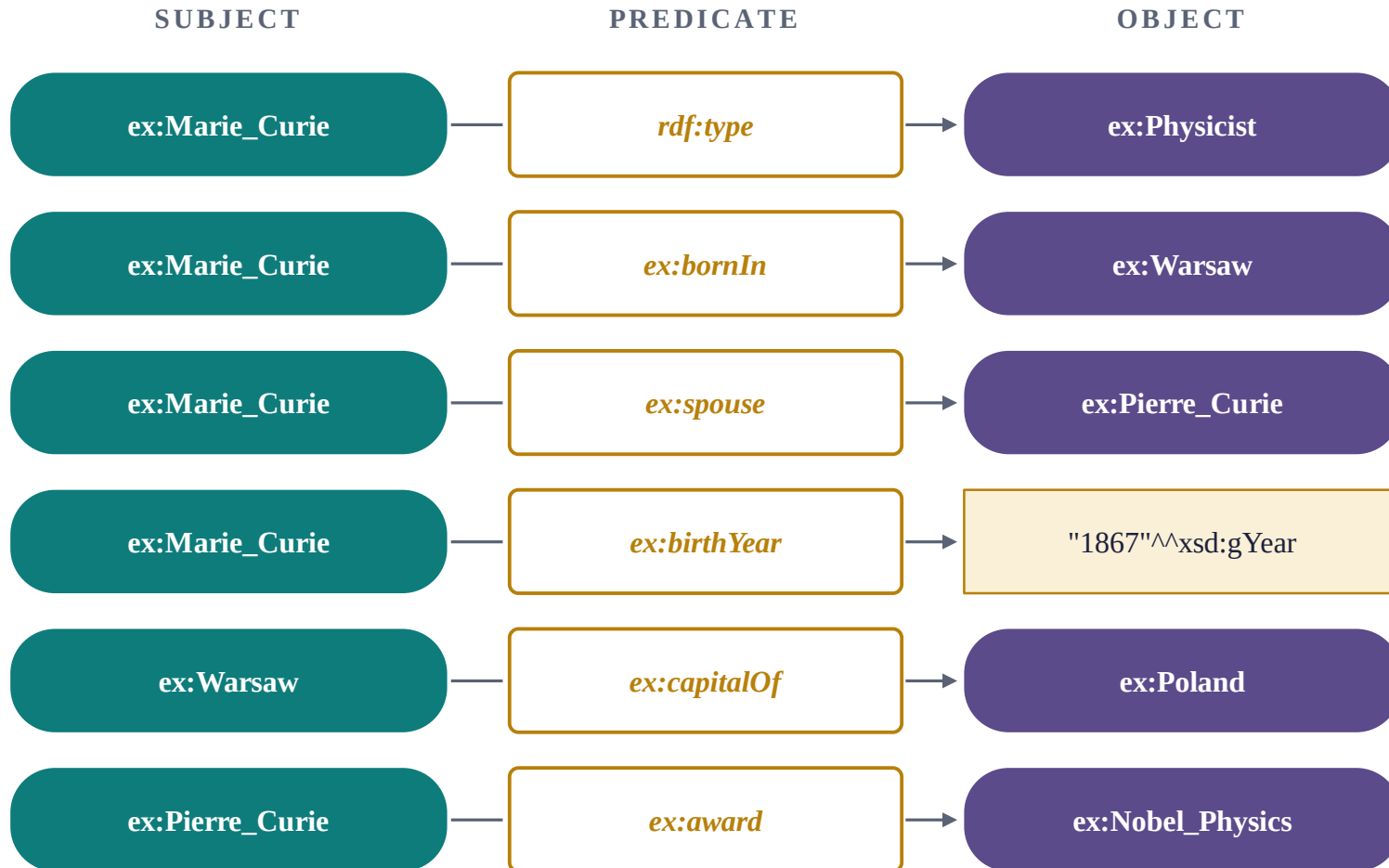
The two dominant graph data models

Aspect	RDF graph	Labelled property graph (LPG)
Unit of data	The triple: subject, predicate, object	Nodes and relationships, each with key value properties
Identifiers	Global IRIs, designed to be shared across the web	Internal IDs, local to one database
Facts about edges	Reification, named graphs or RDF 1.2 triple terms	Native: properties sit directly on relationships
Schema and logic	RDFS and OWL with formal semantics; SHACL for validation	Optional constraints; semantics defined by the application
Standards	W3C: RDF, SPARQL, OWL, SHACL	ISO GQL (2024), SQL/PGQ, openCypher
Sweet spot	Integration across organisations, reasoning, open data	Traversal heavy applications and graph analytics

Convergence: many platforms now support both models, and the differences matter less than a clear, documented schema.



The triple: the atom of an RDF graph



ANNOTATIONS

- **Subject:** an IRI or blank node: the thing being described.
- **Predicate:** always an IRI: the relation or attribute.
- **Object:** an IRI, blank node or literal value.
- **A graph:** is simply a set of triples; order does not matter and duplicates collapse.
- **Merging:** two graphs combine by set union, which is why integration is natural in RDF.



The building blocks of RDF

1 IRIs

Internationalised Resource Identifiers name things globally, for example `http://example.org/Marie_Curie`. Two graphs that use the same IRI are talking about the same thing, which makes merging trivial.

2 Literals

Values with a datatype or language tag: `"1867"^^xsd:gYear`, `"3.14"^^xsd:decimal`, `"Warszawa"@pl`. Literals may only appear in the object position.

3 Blank nodes

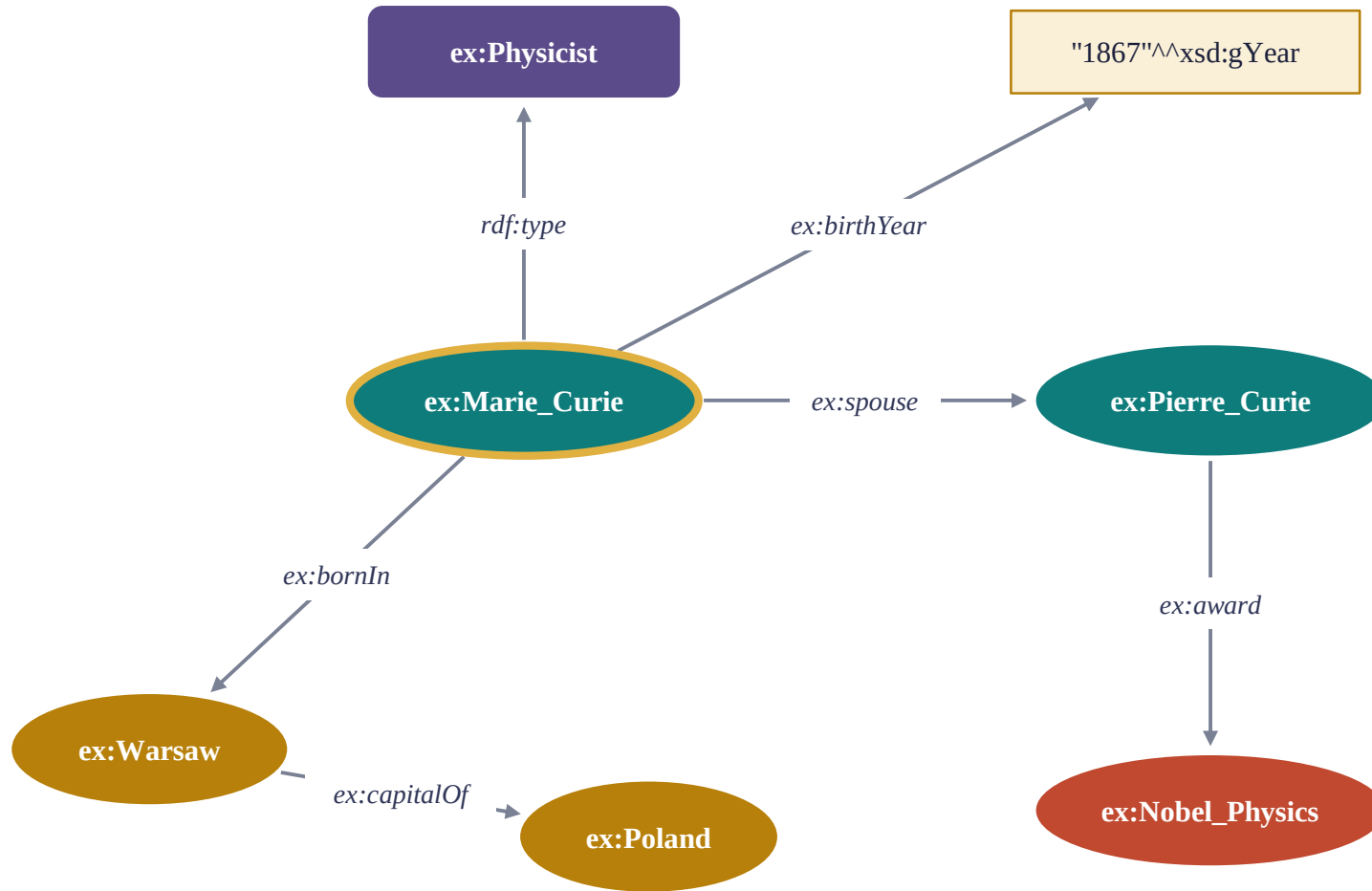
Anonymous nodes with no global name, written `_:b1` or `[ ]`. Useful for structures such as an address, but hard to reference from outside and troublesome when merging graphs.

4 Prefixes

Compact names: with `ex:` bound to `http://example.org/`, `ex:Warsaw` expands to the full IRI. Standard prefixes include `rdf`, `rdfs`, `owl`, `xsd`, `skos` and `schema`.



The same triples drawn as a graph



ANNOTATIONS

- **Nodes:** every distinct subject or object becomes a node; the literal becomes a leaf.
- **Edges:** every predicate becomes a directed, labelled edge.
- **Schema in data:** the class `ex:Physicist` is itself a node, so schema and data share one graph.
- **Traversal:** Marie Curie to Warsaw to Poland answers in which country was she born, a two hop question.
- **Open world:** absence of an award edge for Marie Curie does not mean she had none; it is merely unstated.



Writing RDF: the Turtle syntax

Turtle

```
@prefix ex: <http://example.org/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

ex:Marie_Curie a ex:Physicist , ex:Chemist ;
  ex:name      "Marie Curie"@en , "Maria Skłodowska"@pl ;
  ex:birthYear "1867"^^xsd:gYear ;
  ex:bornIn    ex:Warsaw ;
  ex:spouse    ex:Pierre_Curie ;
  ex:award     [ a ex:NobelPrize ;
                  ex:field ex:Physics ;
                  ex:year  "1903"^^xsd:gYear ] .

ex:Warsaw ex:capitalOf ex:Poland .
```

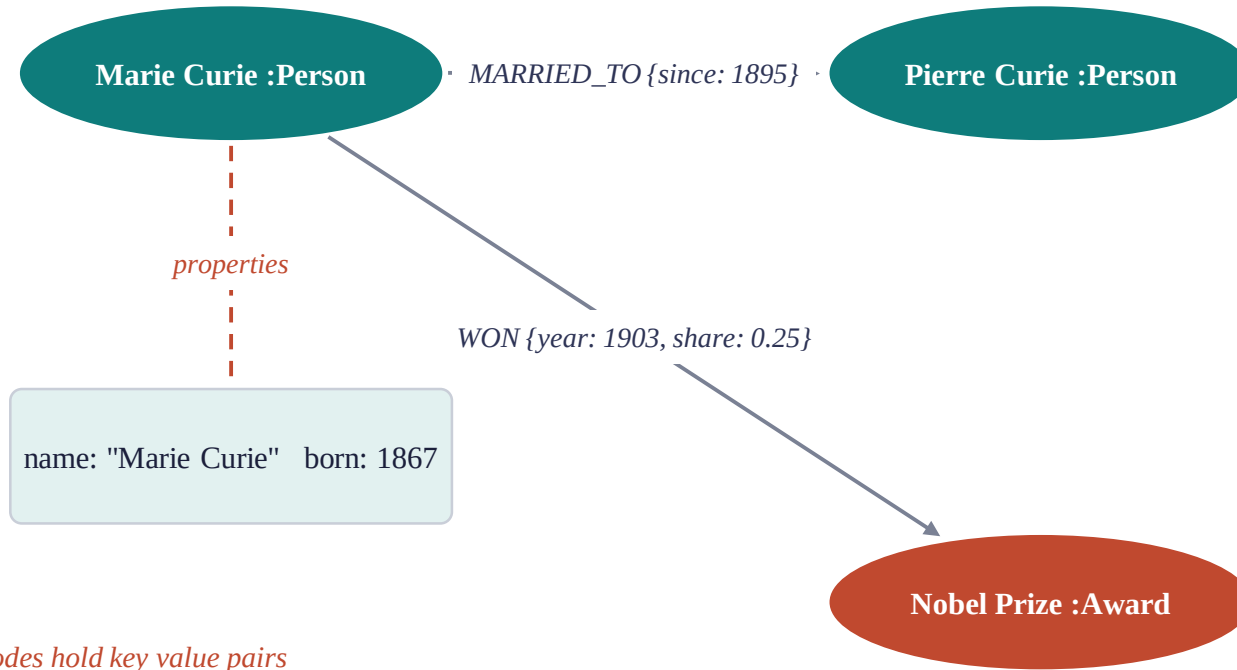
HOW TO READ IT

- **a:** shorthand for `rdf:type`, the class membership predicate.
- **Semicolon:** repeats the subject for the next predicate.
- **Comma:** repeats subject and predicate for another object.
- **Brackets:** create a blank node, here the prize event with its own attributes.
- **@en and ^^xsd:gYear:** language tags and datatypes on literals.
- **Other syntaxes:** RDF/XML, N Triples, JSON LD and TriG (for named graphs) express the same graph.



The labelled property graph model

Edge properties are native



ANNOTATIONS

- **Labels:** nodes carry one or more labels such as :Person, similar to classes.
- **Properties:** both nodes and relationships hold key value pairs.
- **Relationships:** have exactly one type, a direction, and an internal identity.
- **Querying:** Cypher writes patterns as ASCII art: nodes in parentheses joined by arrows; ISO GQL standardises this style.
- **Trade off:** very ergonomic for applications, but meaning lives in code and conventions rather than in formal semantics.



Choosing between RDF and property graphs

Choose RDF when

- Data must be shared or merged across teams, organisations or the open web.
- You need standard, formal semantics and reasoning with OWL.
- You will reuse public vocabularies and link to Wikidata or domain ontologies.
- Validation with SHACL and provenance with named graphs are requirements.

vs

Choose a property graph when

- The workload is dominated by deep traversals and graph algorithms.
- Relationships carry many attributes, such as weights, timestamps and scores.
- The graph serves one application team that owns its semantics.
- Developer familiarity and ecosystem tooling are the deciding factors.

Middle ground: RDF 1.2 triple terms narrow the gap on edge attributes, and several engines now serve both SPARQL and Cypher over one store.



The vocabulary stack: from data to logic

SHACL

Closed world validation: shapes that data must satisfy

OWL 2

Expressive logic: equivalence, disjointness, cardinality, property characteristics

SKOS

Thesauri and taxonomies: broader, narrower and related concepts

RDFS

Lightweight schema: classes, subclasses, domain and range

RDF

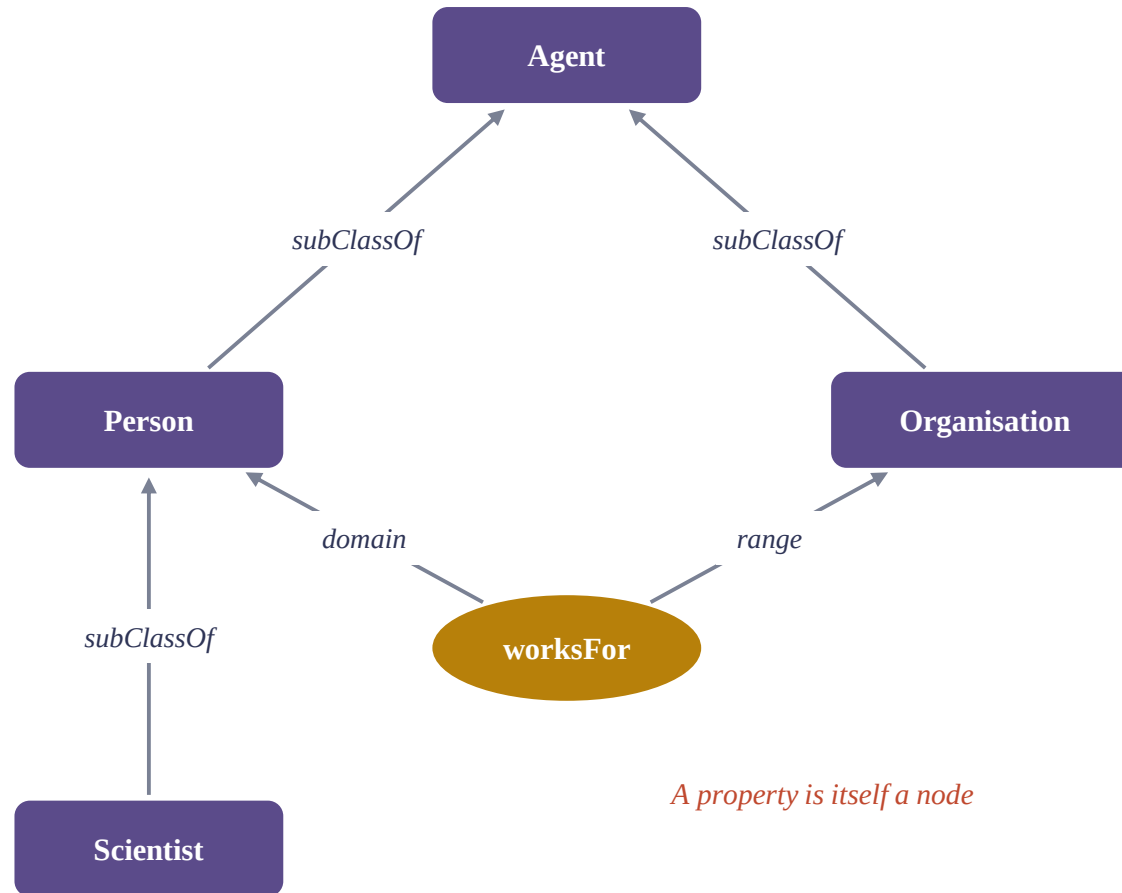
The data model: triples, IRIs, literals and blank nodes

KEY POINTS

- **Increasing power:** each step up adds expressive power and computational cost.
- **Mix and match:** most production graphs use RDFS plus a selection of OWL features, with SHACL for data quality.
- **SKOS is different:** it models concept schemes, not formal classes, so it suits taxonomies and controlled vocabularies.
- **Inference versus validation:** OWL derives new facts under an open world; SHACL checks existing facts under a closed world.



RDF Schema: classes, hierarchies, domains and ranges



ANNOTATIONS

- **rdfs:Class** and **rdfs:subClassOf**: build a class hierarchy; membership propagates upward.
- **rdfs:domain** and **rdfs:range**: state the classes of subject and object for a property.
- **rdfs:subPropertyOf**: builds property hierarchies, for example `ceo subPropertyOf worksFor`.
- **rdfs:label** and **rdfs:comment**: human readable names and definitions for every term.
- **Warning**: domain and range are inference rules, not constraints. If `ex:Acme worksFor ex:Bob`, RDFS concludes Bob is an Organisation rather than raising an error.

OWL 2: key constructs and what they mean

Construct	Meaning	Example
owl:equivalentClass	Two classes have exactly the same members	Vendor equivalent to Supplier
owl:disjointWith	No individual can belong to both classes	Person disjoint with Organisation
owl:inverseOf	One property is the reverse of another	hasPart inverse of partOf
owl:TransitiveProperty	If $a R b$ and $b R c$ then $a R c$	locatedIn: Leeds, England, UK
owl:FunctionalProperty	Each subject has at most one value	birthDate
owl:someValuesFrom	Members have at least one R to class C	Parent: hasChild some Person
Cardinality restrictions	Exact, minimum or maximum counts	Bicycle: hasWheel exactly 2
owl:sameAs	Two IRIs denote the same individual	ex:Curie sameAs wd:Q7186

Formal basis: OWL 2 DL corresponds to the description logic SROIQ, which guarantees decidable reasoning at a high computational price.



Open world versus closed world assumptions

Open world (OWL, RDFS)

- What is not stated is unknown, not false.
- Query: does Marie Curie have a sibling? With no sibling triple, the answer is unknown.
- Suits the web and integration, where no source is ever complete.
- Missing data never causes a contradiction; it may cause an inference instead.

vs

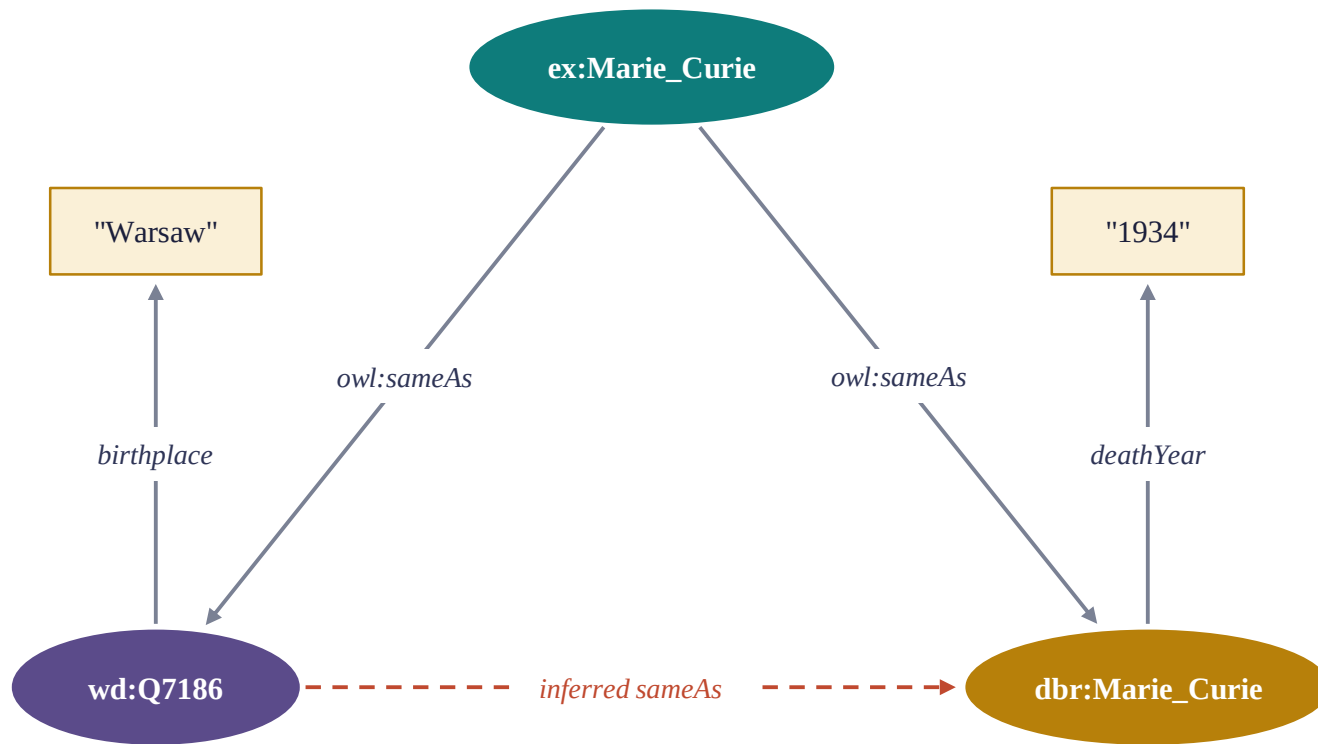
Closed world (SQL, SHACL)

- What is not stated is assumed false.
- Query: does Marie Curie have a sibling? With no row, the answer is no.
- Suits validation and operational systems with authoritative data.
- Missing required data is an error to be reported and fixed.

Practice: reason under the open world with OWL, then validate under the closed world with SHACL. Using both deliberately is a hallmark of mature graphs.



Identity across graphs: owl:sameAs and its alternatives

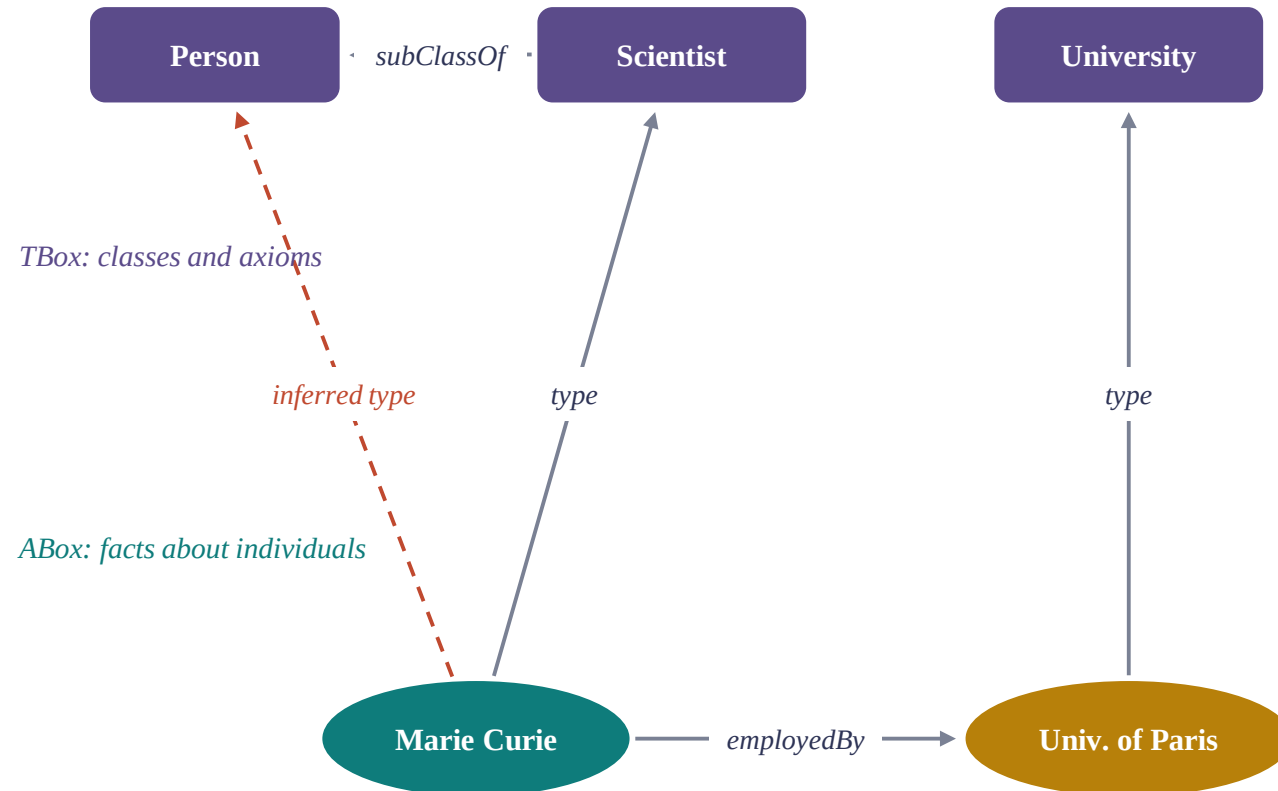


ANNOTATIONS

- **No unique name assumption:** in OWL two different IRIs may denote the same thing unless declared different.
- **owl:sameAs is strong:** everything true of one IRI becomes true of the other, and `sameAs` is symmetric and transitive.
- **Misuse spreads errors:** one wrong `sameAs` merges two identities everywhere; studies of the Linked Data cloud found many such links were inaccurate.
- **Weaker options:** `skos:exactMatch`, `skos:closeMatch` or a custom `sameEntityAs` keep links without full logical merging.



TBox and ABox: terminology versus assertions



ANNOTATIONS

- **TBox:** the terminology: classes, properties and axioms such as `Scientist subClassOf Person`.
- **ABox:** assertions about individuals: Marie Curie is a Scientist.
- **RBox:** in SROIQ, axioms about roles (properties), such as transitivity and chains.
- **Core reasoning tasks:** consistency, subsumption, classification and instance checking.
- **Engineering view:** the TBox is the ontology; the ABox is the instance data. A knowledge graph stores both.



Attaching context to a fact

Real facts need context: Hanabi owns 60% of Busan Components, since 2021, according to the registry. Four ways to say it:

1 Standard reification

Create a node of type `rdf:Statement` with `rdf:subject`, `rdf:predicate` and `rdf:object`, then attach metadata. Standard but verbose: four triples before any context.

2 Named graphs

Put triples into a named graph, a quad store context, and describe the graph: its source, date and licence. Ideal for provenance of whole batches.

3 RDF 1.2 triple terms

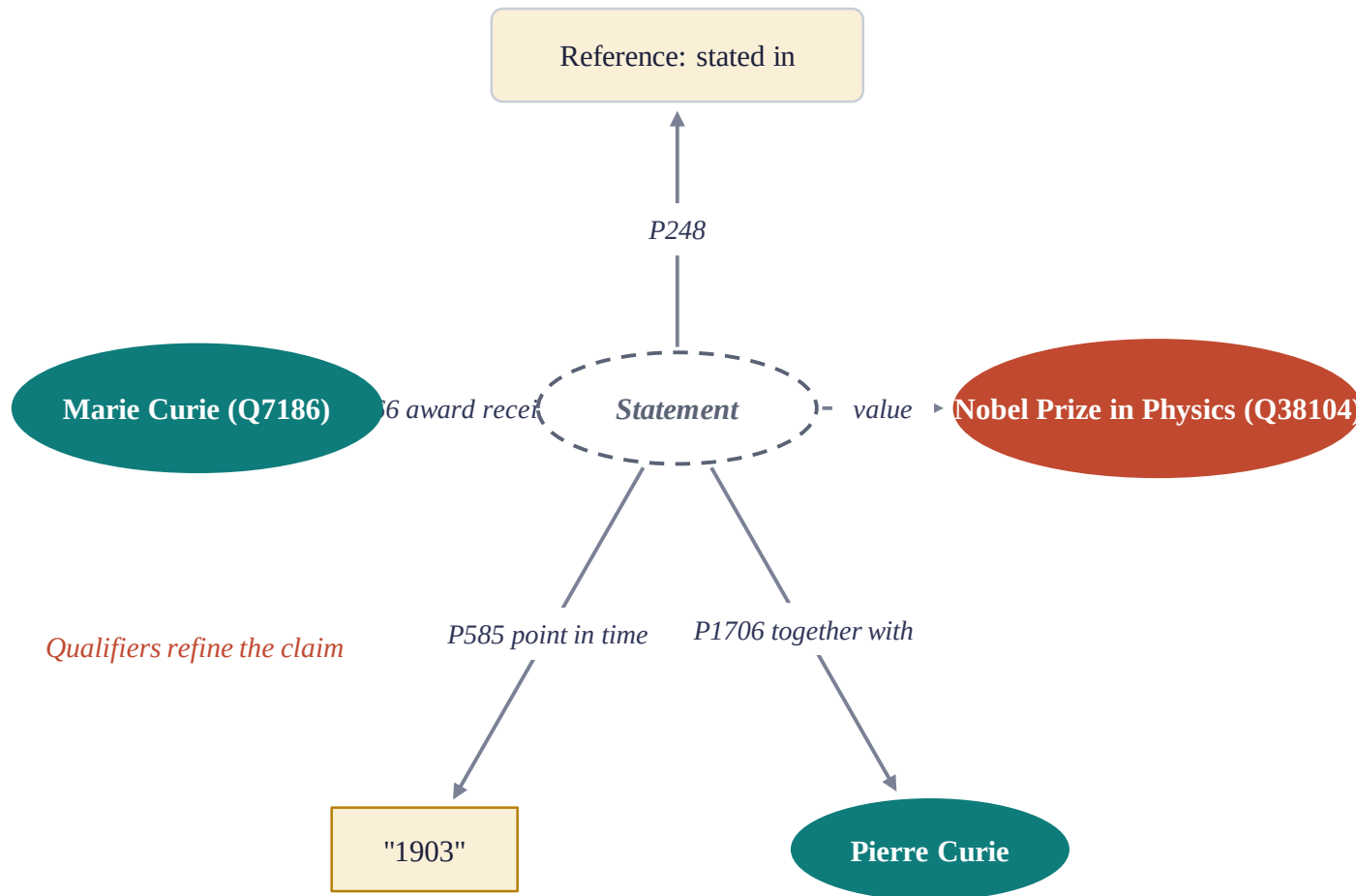
Successor to RDF star: a triple can itself be the subject of statements. Concise, and increasingly supported by engines and SPARQL 1.2.

4 Qualifiers and n ary nodes

Model the relationship as its own node (an Ownership) with share, start date and source. Wikidata uses qualifiers; property graphs use edge properties.



The Wikidata data model: statements, qualifiers, references



ANNOTATIONS

- **Items and properties:** every entity is an item with a Q identifier; every relation is a property with a P identifier.
- **Statements:** a claim links an item to a value through a property and is a first class object.
- **Qualifiers:** add context such as time, co recipients or units, without breaking the main claim.
- **References and ranks:** each statement cites sources and has a rank: preferred, normal or deprecated.
- **Lesson:** real world knowledge is contextual, so plan for qualifiers in your own model from the start.



Linked Data: publishing knowledge on the web



Use URIs as names

Name every thing, not just documents, with a URI.

Use HTTP URIs

So that people and machines can look those names up.

Return useful data

When a URI is looked up, provide information using standards such as RDF and SPARQL.

Link to other URIs

Include links to other things so more can be discovered.

Five star open data: 1 open licence, 2 structured, 3 non proprietary format, 4 URIs for things, 5 linked to other data (Berners Lee, 2006 to 2010).

PART III

The Knowledge Graph Development Lifecycle

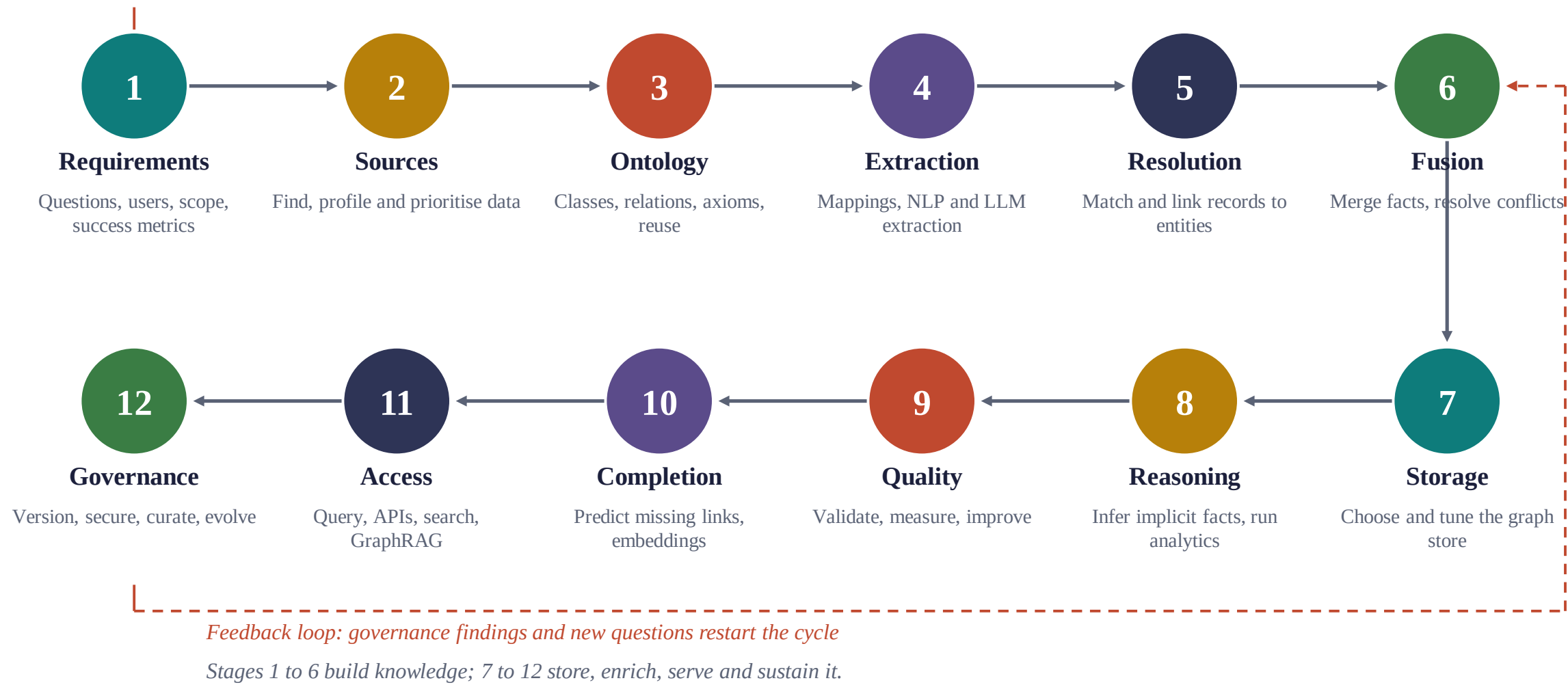
Twelve stages, from the first question to years of governed operation.

- The twelve stages at a glance
- Top down, bottom up and middle out
- Roles on a knowledge graph team
- Stages 1 to 6: from requirements to fusion
- Stages 7 to 12: from storage to governance





The twelve stages at a glance





Top down, bottom up and middle out construction

Top down: schema first

- Design the ontology with experts, then populate it from sources.
- **Strength:** clean semantics, strong consistency, easier reasoning.
- **Risk:** slow start; the model may not fit messy real data.
- **Typical in:** regulated domains such as life sciences and finance.

vs

Bottom up: data first

- Extract entities and relations from data, then induce a schema.
- **Strength:** fast coverage, discovers what the data actually says.
- **Risk:** inconsistent semantics, duplicated relation types, hard to reason over.
- **Typical in:** web scale extraction and exploratory projects.

Recommended: middle out. Model the core of the domain top down, grow the periphery from data, and reconcile the two in each iteration.



Roles on a knowledge graph team

1 Domain experts

Supply the concepts, definitions and competency questions; validate the model and sampled facts.

2 Knowledge engineer

Designs the ontology, patterns and IRI policy; translates expert language into formal models.

3 Data engineer

Builds ingestion, mapping and transformation pipelines; owns scheduling and performance.

4 NLP and ML engineer

Builds extraction, entity resolution, embeddings and completion models with rigorous evaluation.

5 Product owner

Owns use cases, prioritisation and adoption; measures value against the success criteria.

6 Data steward

Owns quality, curation workflows, access policies and the long term health of the graph.

01

STAGE 1 OF 12

Requirements and Scoping

Establish why the graph exists, who will use it, which questions it must answer and how success will be measured, before a single triple is written.

INPUTS

- Business goals and strategy
- User interviews and pain points
- Existing reports and workarounds
- Regulatory and security context

KEY ACTIVITIES

- Stakeholder mapping
- Use case canvases
- Eliciting competency questions
- Scoping and prioritisation
- Defining success metrics

OUTPUTS

- Use case specification
- Prioritised competency questions
- Scope statement
- KPIs and acceptance tests

Guiding question: *Which decisions will be made better because this graph exists?*





Why requirements must come first

- Knowledge graphs are expensive to populate and maintain; unfocused graphs grow wide and shallow and answer nothing well.
- The ontology, the sources, the extraction methods and even the storage engine all follow from the questions the graph must answer.
- Requirements work creates a shared vocabulary between domain experts and engineers early, which is already half of ontology design.
- Explicit success criteria protect the project from scope creep and let you demonstrate value in small increments.
- Graph initiatives that stall very often began with a technology choice rather than a use case; start from the decision, not the database.

Rule of thumb

Start from a handful of high value questions and deliver them end to end, then widen.

A graph that answers ten real questions beats one that stores a billion facts nobody queries.



Stakeholder mapping

Identify everyone who funds, feeds, uses or constrains the graph, and what each needs from it.

1 Executive sponsor

Owns the business outcome and budget. Needs a clear value story and visible milestones.

2 Domain experts

Hold the knowledge. Need their language respected and their time used efficiently.

3 End users and analysts

Ask the questions. Need fast, trustworthy answers in tools they already use.

4 Data owners

Control source systems. Need assurance on load, security and correct use of their data.

5 Engineering and platform

Run the infrastructure. Need clear non functional requirements and operability.

6 Governance, legal, security

Set constraints on privacy, licensing and access. Need auditability and provenance.



The use case canvas: running example

Running example: a supplier risk knowledge graph for a manufacturer of electric vehicles and components.

1 Problem

Procurement cannot see which products depend on suppliers exposed to disruption, sanctions or financial distress.

2 Users

Category managers, supply chain risk analysts and compliance officers.

3 Decisions supported

Dual sourcing, contract renewals, safety stock levels and escalation to compliance.

4 Data required

ERP supplier and bill of materials tables, contracts, corporate registries, sanctions lists and news.

5 Value

Faster disruption response, avoided compliance breaches and lower expediting costs.

6 Success metric

Time to identify affected products falls from days to minutes, with 95% precision on supplier links.



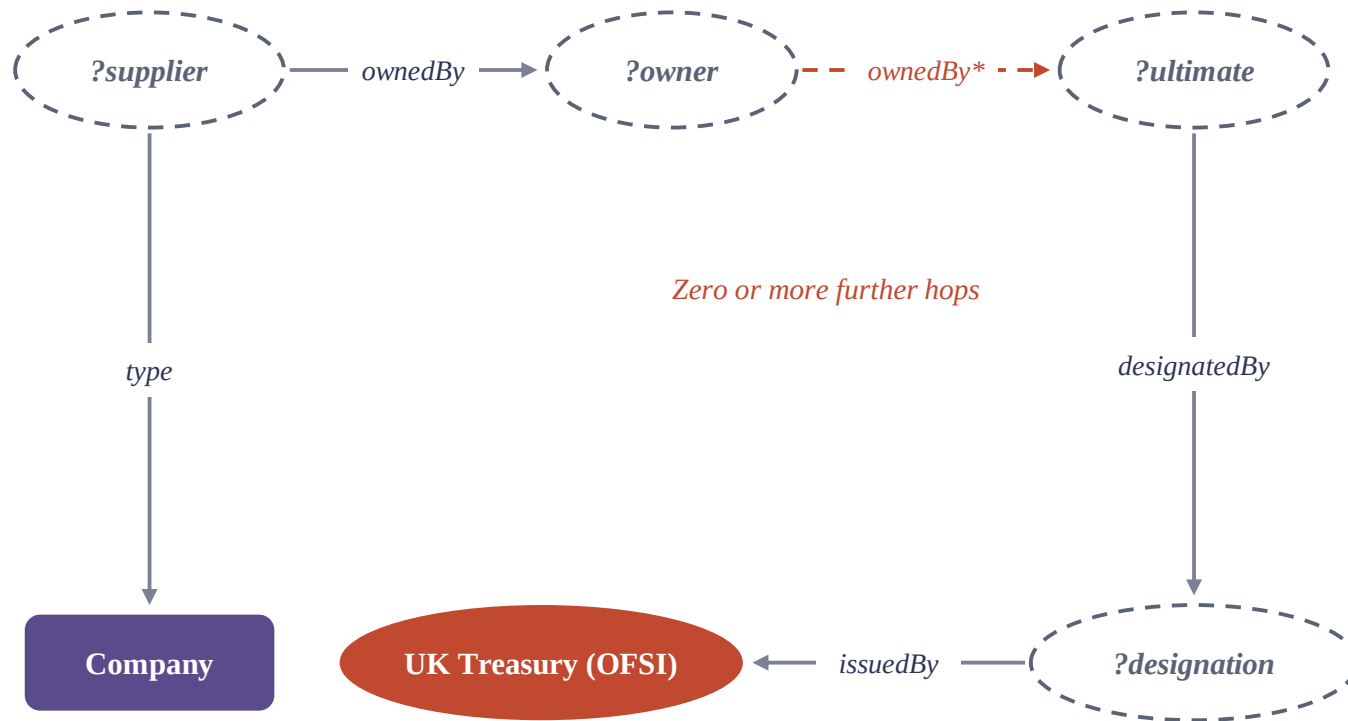
Competency questions: requirements the graph must answer

ID	Competency question	Pattern	Entities and relations
CQ1	Which finished products depend, directly or through subcomponents, on a given supplier?	Recursive traversal	Product, Part, Company; hasComponent, suppliesPart
CQ2	Which suppliers are ultimately owned by a sanctioned entity?	Multihop ownership	Company, Sanction; ownedBy, designatedBy
CQ3	Which facilities lie within 50 km of an active natural hazard event?	Spatial and temporal	Facility, Location, Event
CQ4	How many single source parts does each product line have?	Aggregation	Part, Company, ProductLine
CQ5	What evidence supports the claim that Supplier X is in financial distress?	Provenance	Claim, Document, Source
CQ6	Which suppliers changed ownership in the last twelve months?	Temporal change	Ownership, validity interval

Origin: competency questions were introduced by Grüninger and Fox (1995) as the requirements and the test suite of an ontology.



From a competency question to a graph pattern

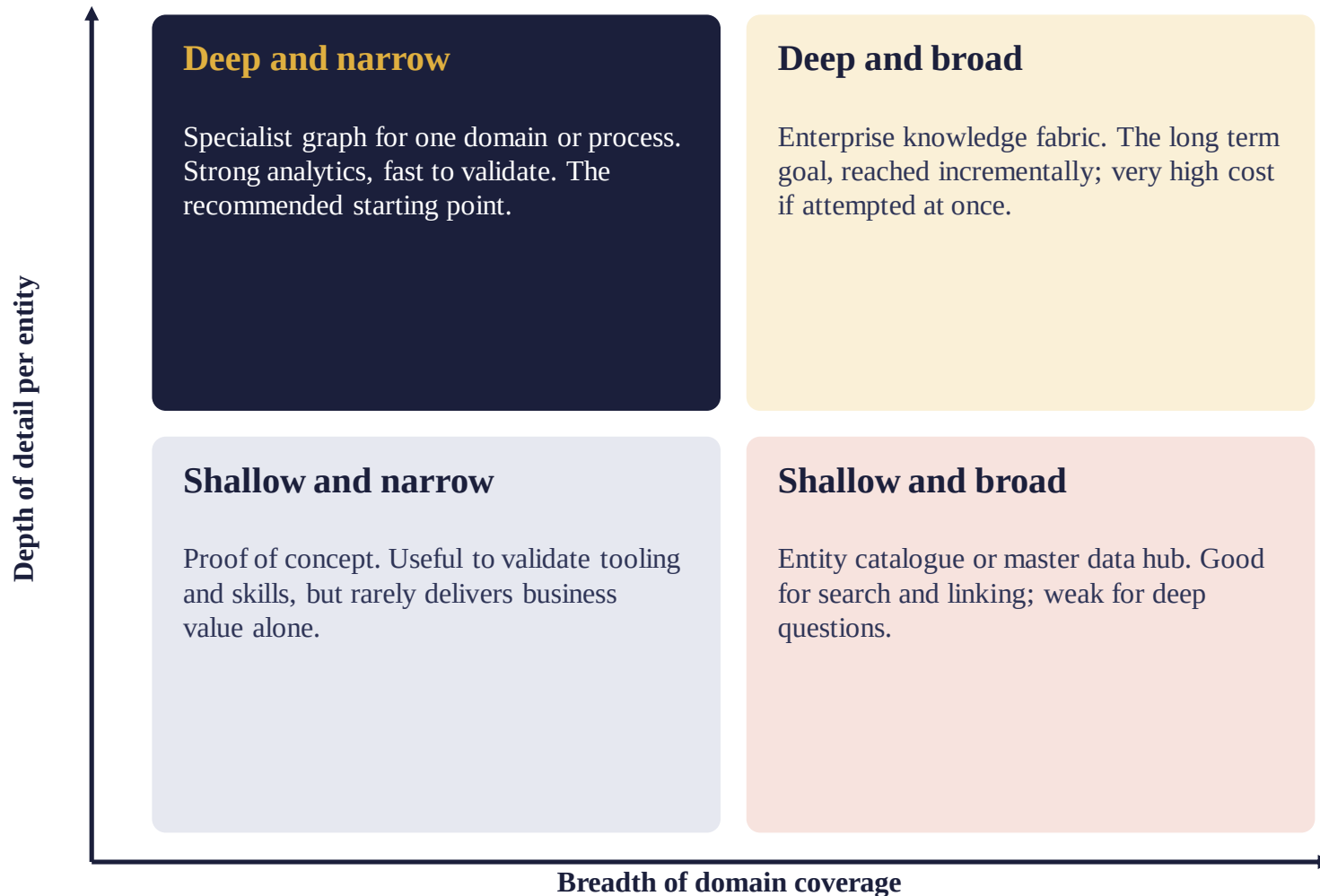


ANNOTATIONS

- **Step 1:** underline the nouns: supplier, entity, sanction. They become candidate classes.
- **Step 2:** circle the verbs: owned by, sanctioned. They become relations.
- **Step 3:** note constraints: ultimately means a transitive path of any length.
- **Step 4:** write the pattern with variables; it becomes a SPARQL query and an acceptance test.
- **Result:** every CQ yields classes, properties and a test, so the ontology is requirements driven.



Scoping: breadth of coverage against depth of detail



HOW TO USE IT

- **Start deep and narrow:** pick the competency questions with the highest value and fewest sources.
- **Expand along one axis at a time:** add new domains or add depth, not both in one release.
- **Write a scope statement:** list what is explicitly out of scope, for example historical data before 2020.
- **Revisit each quarter:** scope is a living decision informed by adoption.



Defining measurable success

90%

of prioritised competency questions
answerable from the graph

95%

precision of sampled supplier facts,
judged by experts

2 s

p95 query latency for the top ten
operational questions

50

weekly active analysts within six
months of launch

- **Measure a baseline first:** how long do analysts take today, and how often are answers wrong or incomplete?
- **Mix metric types:** coverage and quality metrics for the graph, latency for the platform, adoption and outcome metrics for the business.
- **Turn metrics into tests:** each competency question gets an automated SPARQL test with an expected answer set.
- **Tie to outcomes:** link graph metrics to business KPIs such as disruption response time or audit findings.

Example targets for the running example; set your own from the baseline and the value case.



Stage 1 checklist

- ✓ **Use case canvas:** agreed and signed off by the sponsor.
- ✓ **Stakeholder map:** with named contacts and time commitments.
- ✓ **Competency questions:** prioritised, with example answers from experts.
- ✓ **Scope statement:** explicit inclusions, exclusions and assumptions.
- ✓ **Success metrics:** baselines measured and targets set.
- ✓ **Acceptance tests:** drafted as queries for the top questions.
- ✓ **Non functional needs:** latency, freshness, security, availability.
- ✓ **Risk register:** data access, skills, privacy and licensing risks.

Exit criterion: you can explain in one sentence which decision improves, for whom, and how you will know.

02

STAGE 2 OF 12

Knowledge Sources and Data Profiling

Discover every source that could populate the graph, understand its structure, quality, ownership and licence, and decide which to ingest first.

INPUTS

- Competency questions and scope
- System inventories and catalogues
- Glossaries and taxonomies
- Candidate external datasets

KEY ACTIVITIES

- Source discovery
- Structural and statistical profiling
- Identifier and key analysis
- Licence and privacy review
- Prioritisation

OUTPUTS

- Source inventory
- Profiling reports
- Identifier crosswalk candidates
- Ingestion priority list

Guiding question: *Where does the knowledge we need already live, and how trustworthy is it?*





A taxonomy of knowledge sources

1 Structured

Relational databases, data warehouses, CSV and spreadsheets. Explicit schema, highest precision, easiest to map.

2 Semi structured

JSON, XML, APIs, logs and web tables. Nested and irregular, but with keys and field names.

3 Unstructured

Documents, PDFs, email, news and web pages. The largest volume, and the hardest to extract reliably.

4 Existing knowledge assets

Taxonomies, thesauri, ontologies, glossaries and master data. Often the seed of the schema.

5 External open data

Wikidata, GeoNames, company registries and sanctions lists. Context and identifiers for linking.

6 Human expertise

Interviews, curation tools and review queues. Slow, but essential for tacit knowledge and validation.



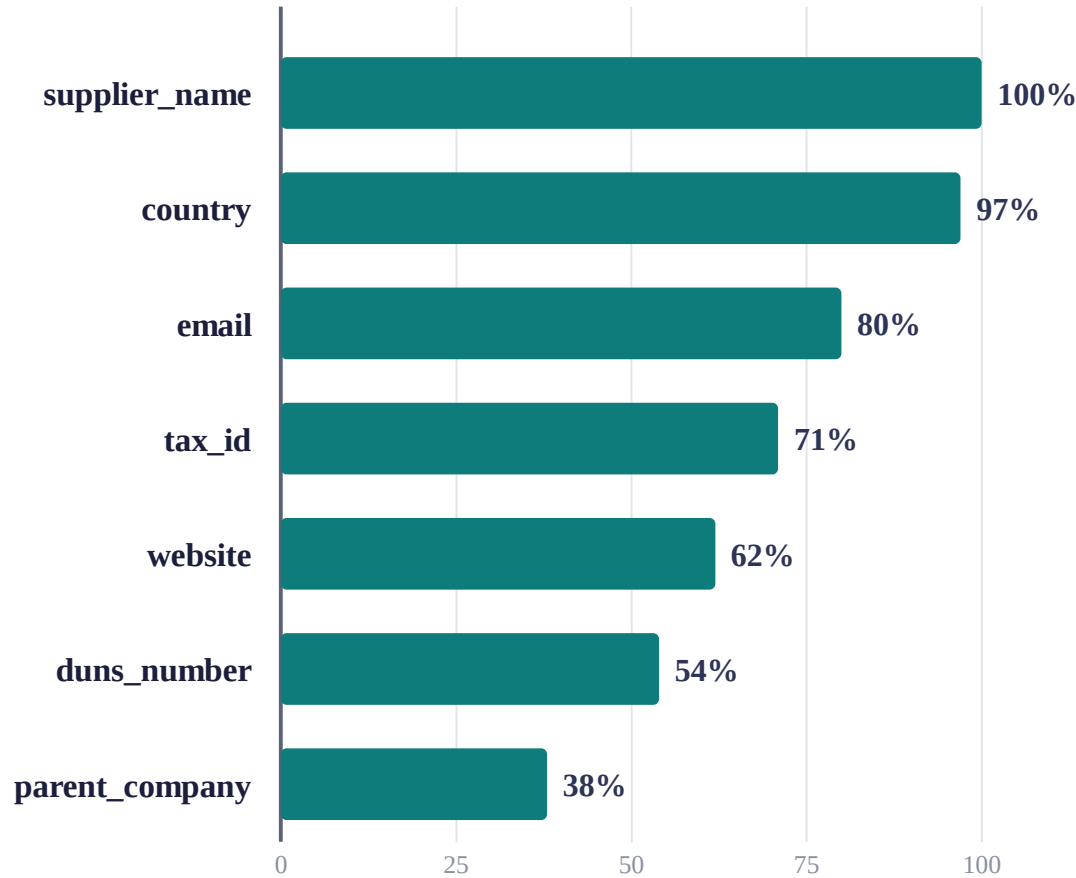
The source inventory: running example

Source	Format	Owner	Refresh	Key identifiers	Access and licence
ERP supplier master	SQL tables	Procurement IT	Daily	vendor_id, tax_id	Internal
Bill of materials	SQL tables	Engineering	Daily	part_no, product_id	Internal
Contracts repository	PDF, DOCX	Legal	On change	contract_no, names	Internal, restricted
Corporate registries	API, bulk files	External	Weekly	company_number	Per registry terms
Sanctions lists	XML, CSV	OFSI, OFAC	Daily	list ID, names	Public
News feed	JSON articles	Vendor	Streaming	None: names in text	Commercial licence
GeoNames	Bulk files	External	Monthly	geonameid	CC BY 4.0

Tip: record the owner and refresh cadence now; they drive pipeline design, data contracts and freshness metrics later.



Profiling example: attribute completeness



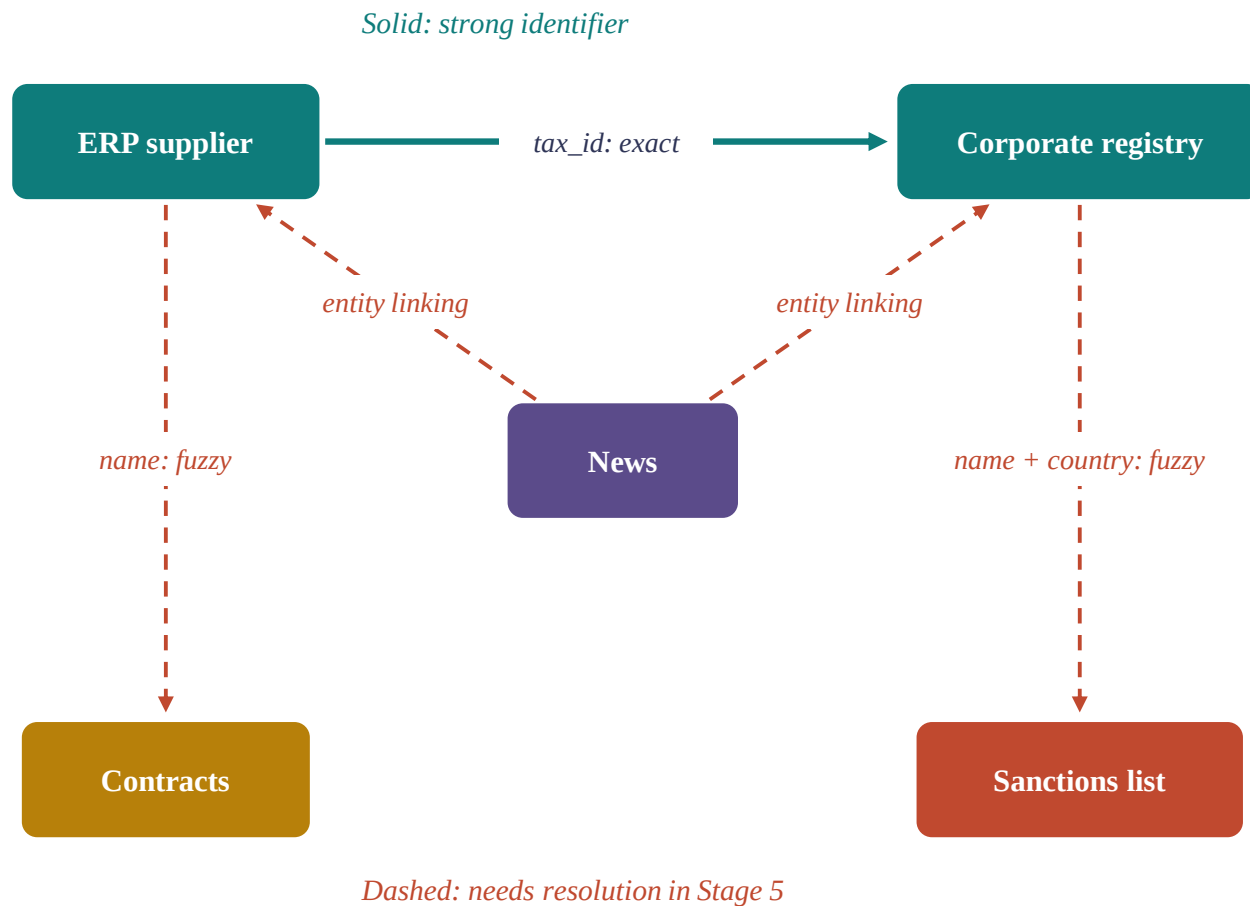
Illustrative profile of an ERP supplier table with 12,400 rows: percentage of rows with a non empty value.

READING THE CHART

- **Completeness:** parent_company at 38% means ownership (CQ2) cannot come from ERP alone; registries are needed.
- **Identifiers:** tax_id at 71% is the best join key but leaves 29% for fuzzy matching.
- **Other dimensions:** uniqueness, validity against patterns, value distributions, outliers and freshness.
- **Tools:** SQL profiling queries, ydata profiling, Great Expectations or dbt tests.



Finding identifiers and join paths between sources



ANNOTATIONS

- **Strong identifiers:** tax numbers, LEIs, company numbers: exact, stable, shared.
- **Weak identifiers:** names, addresses, phone numbers: need normalisation and probabilistic matching.
- **No identifiers:** text sources require entity linking against the graph.
- **Crosswalks:** record known mappings between identifier schemes as data, with provenance.
- **Output:** a join path plan that feeds entity resolution design.



Licensing, privacy and legal review

1 Licences

Wikidata is CC0; DBpedia and schema.org use CC BY SA; GeoNames uses CC BY. Share alike and attribution terms propagate to derived graphs and must be tracked per source.

2 Personal data

Under UK GDPR, identify the lawful basis, minimise attributes, run a DPIA for high risk processing, and plan how erasure requests propagate through linked and inferred facts.

3 Commercial terms

Vendor data such as credit ratings or news may forbid redistribution or derived works. Keep such facts in separate named graphs with access controls.

4 Provenance duty

Record the source, retrieval date and licence of every fact from the first load. Retrofitting provenance later is extremely costly.



Reusing open knowledge graphs and vocabularies

Resource	What it offers	Licence	Typical use
Wikidata	Over 100 million items with multilingual labels and external identifiers	CC0	Entity linking hub, labels, context
DBpedia	Structured facts extracted from Wikipedia infoboxes	CC BY SA	Background facts, categories
schema.org	Widely used web vocabulary for organisations, products, events	CC BY SA	Core vocabulary reuse
GeoNames	Millions of place names with coordinates and hierarchy	CC BY 4.0	Location normalisation
Gene Ontology	Controlled terms for gene functions and processes	CC BY 4.0	Biomedical annotation
FIBO	Financial Industry Business Ontology from the EDM Council	MIT	Financial entities and contracts
UMLS	Integrated biomedical vocabularies and concept identifiers	Licence agreement	Clinical terminology mapping

Check before use: licences and scale change over time; confirm the current terms and release on each provider site.



Prioritising sources: value against effort



HOW TO USE IT

- **Score each source:** value is the number and weight of CQs it helps answer.
- **Effort includes:** access negotiation, format complexity, quality fixes, licence review.
- **Sequence:** deliver quick wins in the first release to prove value, then fund strategic bets.
- **Revisit:** effort estimates drop as reusable pipelines mature.



Stage 2 checklist



Source inventory: owner, format, volume, refresh and access for each.



Privacy assessment: personal data identified and DPIA decision made.



Profiling reports: completeness, uniqueness, validity, distributions.



External reuse plan: which open KGs and vocabularies to link.



Identifier analysis: strong and weak keys, crosswalk candidates.



Priority list: sources ordered by value against effort.



Licence register: terms and obligations per source.



Data contracts: agreed schemas and SLAs with source owners.

Exit criterion: for every prioritised competency question you know which sources will answer it and how they join.

03

STAGE 3 OF 12

Ontology and Schema Design

Define the classes, relations, constraints and identifiers that give the graph its meaning, reusing established vocabularies wherever possible.

INPUTS

- Competency questions
- Source profiles and glossaries
- Existing standards and ontologies
- Expert interviews

KEY ACTIVITIES

- Term extraction and conceptualisation
- Class and property design
- Applying design patterns
- Reuse and alignment
- Expert review and testing

OUTPUTS

- Ontology in OWL, RDFS or LPG schema
- IRI and naming policy
- Documentation
- Backlog of SHACL shapes

Guiding question: *What kinds of things exist in our domain, and how may they relate?*





What an ontology contributes

- **Shared meaning:** every class and relation has a definition agreed by experts and engineers.
- **Integration contract:** each source maps into one conceptual model instead of into every other source.
- **Inference:** axioms such as subclass, inverse and transitive relations let machines derive implicit facts.
- **Validation:** constraints expose errors before they reach applications.
- **Evolvability:** new sources and use cases extend the model rather than break it.
- **Explainability:** answers trace through named, defined relations.

Classic definition

An ontology is a formal, explicit specification of a shared conceptualisation.

Studer, Benjamins and Fensel (1998), refining Gruber (1993).

Formal: machine readable. Explicit: nothing left implicit. Shared: agreed by a community.



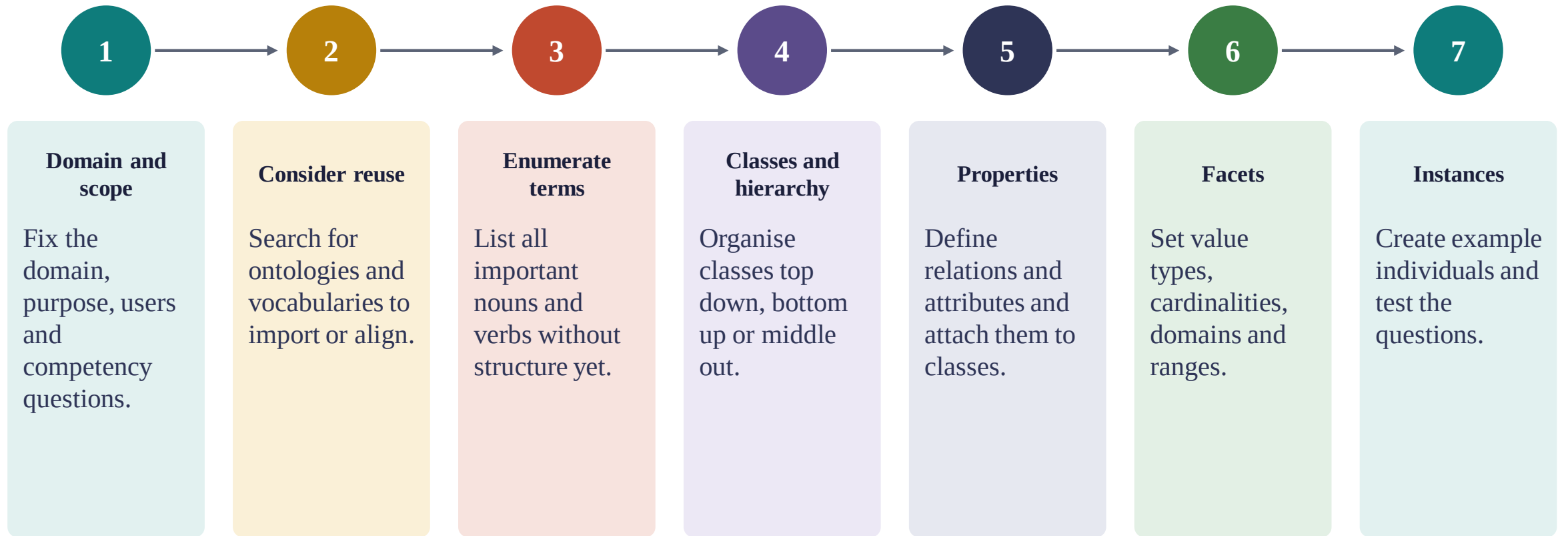
Established ontology engineering methodologies

Methodology	Core idea	Best suited to
Ontology Development 101 (Noy and McGuinness, 2001)	Seven iterative steps from scope to instances; pragmatic and widely taught	Small teams, first ontologies
METHONTOLOGY (Fernández López et al., 1997)	Lifecycle modelled on software engineering: specification, conceptualisation, formalisation, implementation, maintenance	Large, documented projects
NeOn (Suárez Figueroa et al., 2012)	Nine scenarios for building networks of ontologies, emphasising reuse and re engineering	Reusing existing resources
eXtreme Design (Presutti et al., 2009)	Pattern based and test driven: each competency question is solved with a content pattern and a test	Agile teams, modular ontologies
LOT (Poveda Villalón et al., 2022)	Linked Open Terms: lightweight, industry oriented, with requirements, implementation, publication and maintenance	Industrial Linked Data

Common thread: all methodologies are iterative, requirements driven, reuse first and tested against competency questions.



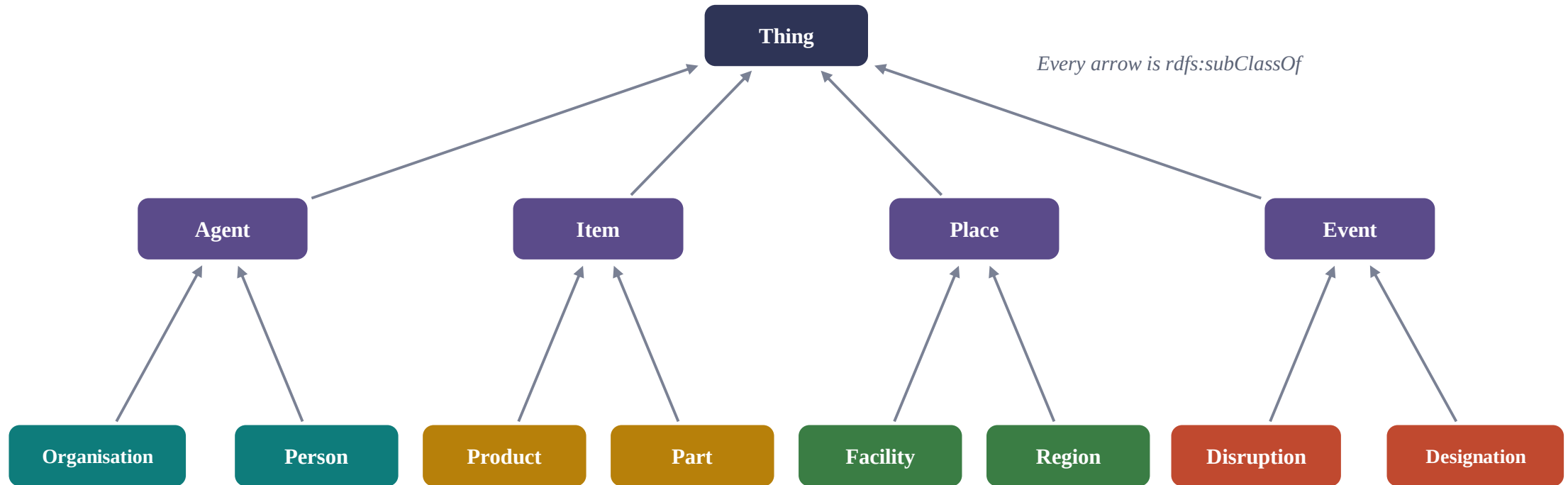
Ontology Development 101: the seven steps



Iterate: the steps loop. Instances and failing competency questions routinely send you back to steps 3 to 6.



Class hierarchy for the supplier risk graph



- **Roles are not classes:** Supplier is modelled as the relation `suppliesPart`, because the same company can be a supplier, a customer and a competitor at once.
- **Siblings should be disjoint:** declaring `Organisation` disjoint with `Person` catches type errors automatically.
- **Depth guideline:** add a subclass only if it has distinct properties, constraints or query value; otherwise use a category attribute or SKOS concept.



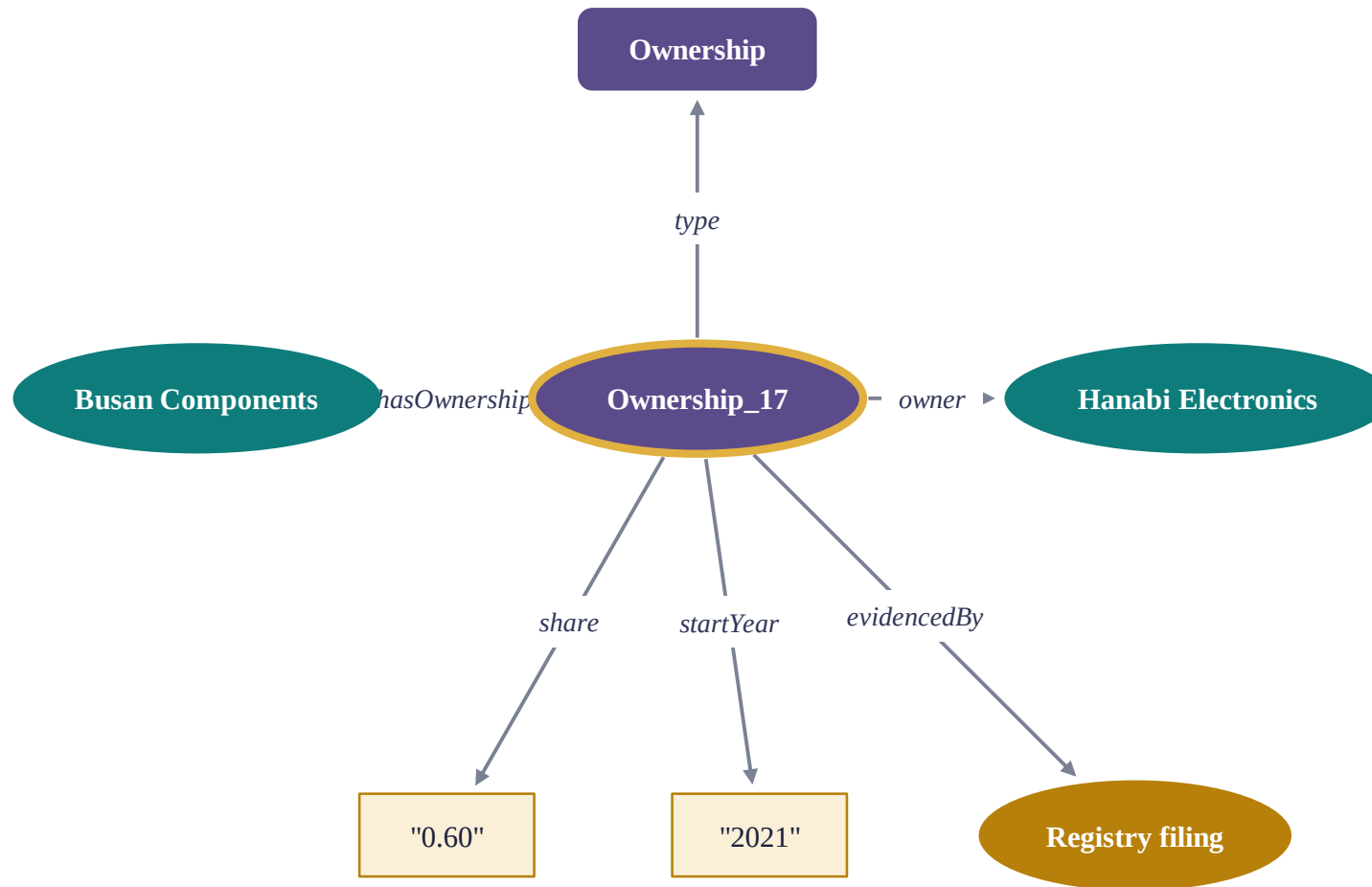
Designing properties: kind, domain, range, characteristics

Property	Kind	Domain → Range	Characteristics and notes
suppliesPart	Object	Organisation → Part	Many to many; qualified by contract and period
hasComponent	Object	Item → Item	Transitive; inverse componentOf
ownedBy	Object	Organisation → Agent	Modelled as an n ary Ownership for share and dates
locatedIn	Object	Facility → Region	Transitive through the region hierarchy
legalName	Datatype	Organisation → xsd:string	Functional: exactly one current legal name
incorporatedOn	Datatype	Organisation → xsd:date	Functional; must precede ownership start dates
riskScore	Datatype	Organisation → xsd:decimal	Derived in Stage 8; stored with provenance

Naming: use camelCase verbs for relations (suppliesPart), nouns for attributes (legalName), and write a definition for every property.



The n ary relation pattern: when a relation has attributes



ANNOTATIONS

- **Problem:** ownedBy as a single edge cannot hold share, dates and evidence.
- **Pattern:** turn the relationship into a node, an Ownership, with its own properties (W3C note on N ary Relations, 2006).
- **Benefit:** history is natural; a new Ownership node records each change.
- **Shortcut:** keep a direct ownedBy edge for fast traversal, derived from current Ownership nodes.
- **Other patterns:** part and whole, participation in events, time indexed situations, and value partitions.



The core ontology in OWL, written in Turtle

OWL / Turtle

```
@prefix sc: <http://example.org/supply#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
# standard rdf and rdfs prefixes omitted for brevity

sc:Organisation a owl:Class ;
  rdfs:label "Organisation"@en ;
  owl:disjointWith sc:Person , sc:Item .
sc:Company a owl:Class ;
  rdfs:subClassOf sc:Organisation .

sc:suppliesPart a owl:ObjectProperty ;
  rdfs:domain sc:Organisation ;
  rdfs:range sc:Part .
sc:hasComponent a owl:ObjectProperty , owl:TransitiveProperty ;
  owl:inverseOf sc:componentOf ;
  rdfs:domain sc:Item ; rdfs:range sc:Item .
sc:legalName a owl:DatatypeProperty , owl:FunctionalProperty ;
  rdfs:range xsd:string .
```

HOW TO READ IT

- **Classes:** declared with `owl:Class` and placed in the hierarchy.
- **Disjointness:** lets a reasoner flag any node typed as both `Organisation` and `Item`.
- **Object properties:** link entities; datatype properties link to literals.
- **Characteristics:** transitivity, inverses and functionality encode domain rules.
- **Tooling:** author in Protégé or WebProtégé, automate builds with ROBOT, and scan for modelling pitfalls with OOPS!



Reuse before you invent

Reuse raises interoperability and lowers cost. Import terms selectively (the MIREOT principle) rather than whole ontologies.

1 **schema.org**

Organisation, Product, Place, Event, Person: the most widely deployed web vocabulary.

2 **FOAF and vCard**

People, contact details and social relations.

3 **Dublin Core**

Document metadata: title, creator, date, subject, rights.

4 **PROV O**

W3C provenance ontology: entities, activities and agents.

5 **SKOS**

Taxonomies and code lists: broader, narrower, related concepts.

6 **Domain standards**

FIBO, GS1, SNOMED CT, Gene Ontology: deep, expert curated models.

04

STAGE 4 OF 12

Knowledge Extraction

Turn source data into candidate triples that conform to the ontology, using declarative mappings for structured data and NLP or LLMs for text.

INPUTS

- Prioritised sources
- Ontology and IRI policy
- Labelled examples and gold data
- Access credentials

KEY ACTIVITIES

- Declarative mapping of tables
- Wrappers for semi structured data
- NER, relation and event extraction
- Schema guided LLM extraction
- Confidence scoring

OUTPUTS

- Candidate triples with provenance
- Mapping files under version control
- Extraction evaluation report

Guiding question: *How do we turn rows, records and sentences into facts?*





The extraction landscape by source type

Unstructured text

Named entity recognition, relation and event extraction, LLM extraction. Probabilistic: needs evaluation and confidence scores.

Semi structured

JSON and XML mappings, web wrappers, table interpretation. Mostly deterministic, but irregular.

Structured

Declarative mappings with R2RML, RML or SQL views. Deterministic and highly precise.

Existing graphs

Direct import, alignment or federated querying of Wikidata, domain ontologies and partner graphs.

KEY POINTS

- **Gradient:** moving up the stack, precision falls and effort rises.
- **Start at the bottom:** structured sources give a reliable backbone of entities to which text facts can link.
- **Uniform output:** every method emits the same shape: triple, source, method, confidence and timestamp.
- **Keep raw inputs:** store source snapshots so extraction can be rerun when models improve.



Mapping a table into the graph

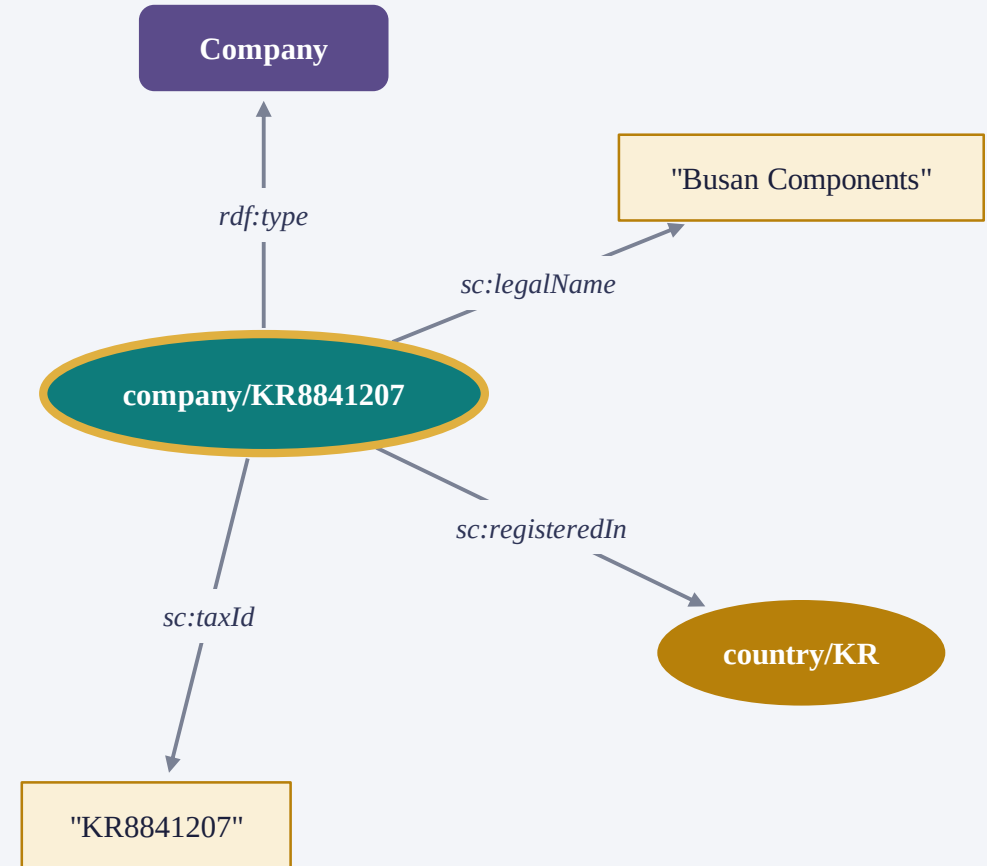
ERP_SUPPLIER table

TAX_ID	SUPPLIER_NAME	COUNTRY
KR8841207	Busan Components	KR
JP5521980	Hanabi Electronics	JP
GB7730114	Kestrel Metals	GB



Rule: each row becomes one node; the key forms its IRI; each column becomes one triple.

Values that denote things (country codes) become links to nodes, not strings.





Declarative mapping with R2RML

R2RML

```
@prefix rr: <http://www.w3.org/ns/r2rml#> .
@prefix sc: <http://example.org/supply#> .

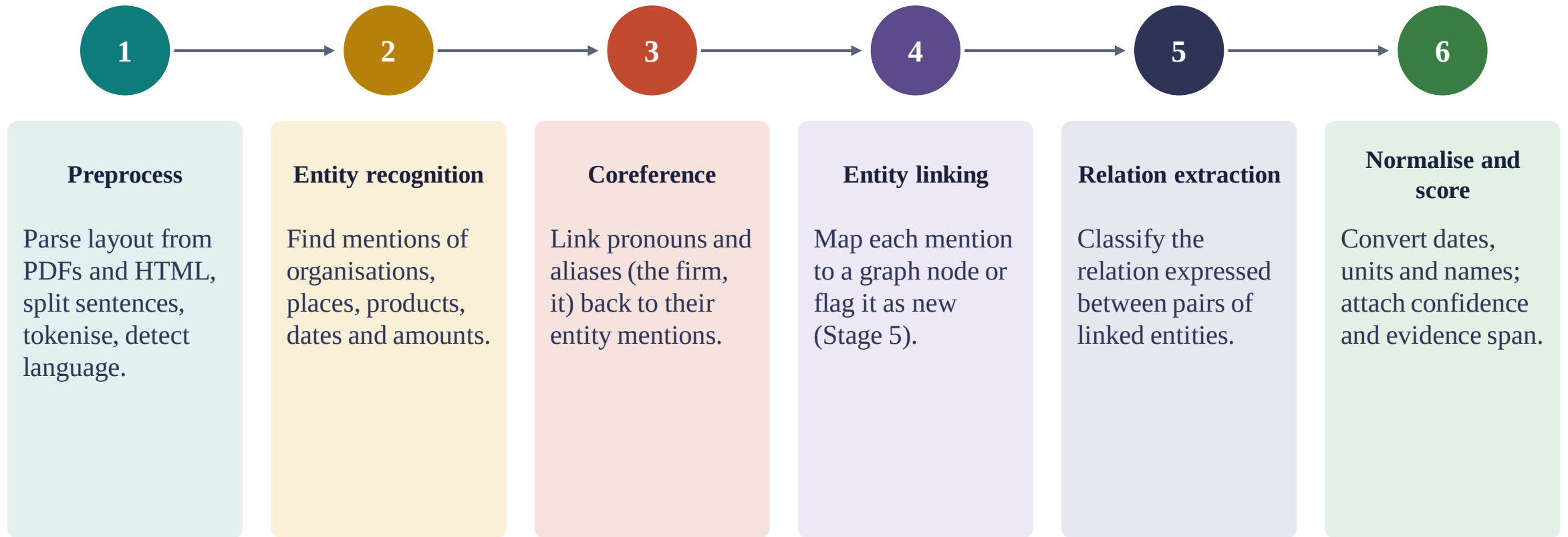
<#SupplierMap>
  rr:logicalTable [ rr:tableName "ERP_SUPPLIER" ] ;
  rr:subjectMap [
    rr:template "http://example.org/company/{TAX_ID}" ;
    rr:class sc:Company
  ] ;
  rr:predicateObjectMap [
    rr:predicate sc:legalName ;
    rr:objectMap [ rr:column "SUPPLIER_NAME" ]
  ] ;
  rr:predicateObjectMap [
    rr:predicate sc:registeredIn ;
    rr:objectMap [
      rr:template "http://example.org/country/{COUNTRY}" ]
    ] .
```

HOW TO READ IT

- **Standard:** R2RML is a W3C Recommendation (2012) for mapping relational data to RDF.
- **Subject map:** the template implements your IRI policy; the class assigns `rdf:type`.
- **Predicate object maps:** one per column or derived value.
- **RML:** extends the idea to CSV, JSON and XML; YARRRML offers a friendlier YAML syntax.
- **Virtual graphs:** engines such as Ontop rewrite SPARQL into SQL, so data need not be copied.
- **Engines:** Morph KGC, RMLMapper and SDM RDFizer materialise mappings at scale.



The NLP pipeline for extracting knowledge from text



Error propagation: mistakes compound along the pipeline, which is why joint models and LLMs that perform several steps at once are attractive.



Worked example: from one sentence to four facts

On **4 March**, **Hanabi Electronics** of **Tokyo** acquired a **60%** stake in **Busan Components**, which supplies **lithium cells** to **Norden Motors**.

DATE ORG LOC PCT ORG PART ORG

EXTRACTED FACTS

Hanabi Electronics	<i>acquiredStakeIn</i>	Busan Components	qualifiers: share 0.60, date 4 March
Hanabi Electronics	<i>locatedIn</i>	Tokyo	
Busan Components	<i>suppliesPart</i>	Lithium cell	
Busan Components	<i>suppliesTo</i>	Norden Motors	

NOTICE

- Mentions become nodes after linking.
- Percentages and dates become qualifiers.
- *suppliesTo* is implicit: which supplies lithium cells to.



Relation extraction approaches compared

Approach	How it works	Strengths	Weaknesses
Hand written patterns	Lexical or syntactic rules, such as Hearst patterns (1992): X such as Y	Precise, transparent, no training data	Low recall, brittle, costly to maintain
Supervised classifiers	Train on sentences labelled with relations	Accurate within domain	Labelled data is expensive
Distant supervision	Label text automatically with facts already in a KG (Mintz et al., 2009)	Large training sets for free	Noisy labels; needs denoising
Open IE	Extract any (argument, phrase, argument) tuple without a schema (Banko et al., 2007)	Schema free, high coverage	Relations must be canonicalised later
Fine tuned transformers	Sequence models such as REBEL generate triples end to end	Strong accuracy, joint NER and RE	Needs training data and GPUs
LLM prompting	Zero or few shot extraction guided by the ontology	Fast to start, flexible	Cost, variance, hallucination risk



Schema guided extraction with a large language model

Prompt and JSON

SYSTEM: Extract facts using ONLY these relations:

acquiredStakeIn(Company, Company, share, date)

suppliesPart(Company, Part)

suppliesTo(Company, Company)

Return JSON. Quote the evidence span for every fact.

If no allowed relation is stated, return an empty list.

OUTPUT:

```
{ "facts": [  
  { "relation": "acquiredStakeIn",  
    "head": "Hanabi Electronics",  
    "tail": "Busan Components",  
    "share": 0.60, "date": "4 March",  
    "evidence": "acquired a 60% stake in Busan Components" },  
  { "relation": "suppliesTo",  
    "head": "Busan Components",  
    "tail": "Norden Motors",  
    "evidence": "supplies lithium cells to Norden Motors" }  
] }
```

HOW TO READ IT

- **Constrain the schema:** list allowed relations and types from the ontology; reject anything else.
- **Structured output:** use JSON schema or constrained decoding so outputs always parse.
- **Demand evidence:** require a quoted span, then verify it really occurs in the source text.
- **Validate:** check types, domains and ranges before facts enter the graph.
- **Measure:** evaluate on a gold set exactly as for any other extractor; track cost per document.



Evaluating extraction: precision, recall and F1

80.0%

Precision: 80 correct facts out of 100 extracted. $TP / (TP + FP)$

66.7%

Recall: 80 found out of 120 facts in the gold standard. $TP / (TP + FN)$

72.7%

F1: harmonic mean, $2PR / (P + R) = 2 \times 0.8 \times 0.667 / 1.467$

- **Define matching first:** exact entity and relation match, or partial credit for boundaries? State it in every report.
- **Evaluate per relation type:** a good average can hide a relation with 30% precision.
- **Micro versus macro averaging:** micro weights frequent relations; macro treats all relation types equally.
- **Precision usually matters more for KG population:** a wrong fact propagates through reasoning and queries; tune thresholds on the precision recall curve.

Worked example: a gold standard of 120 facts; the system extracts 100, of which 80 are correct.

05

STAGE 5 OF 12

Entity Resolution and Linking

Decide which records and mentions refer to the same real world entity, merge them into one canonical node, and link them to reference identifiers.

INPUTS

- Candidate entities from extraction
- Identifier crosswalks
- Reference KGs such as Wikidata
- Labelled match pairs

KEY ACTIVITIES

- Normalisation
- Blocking
- Pairwise matching
- Clustering
- Canonicalisation and entity linking

OUTPUTS

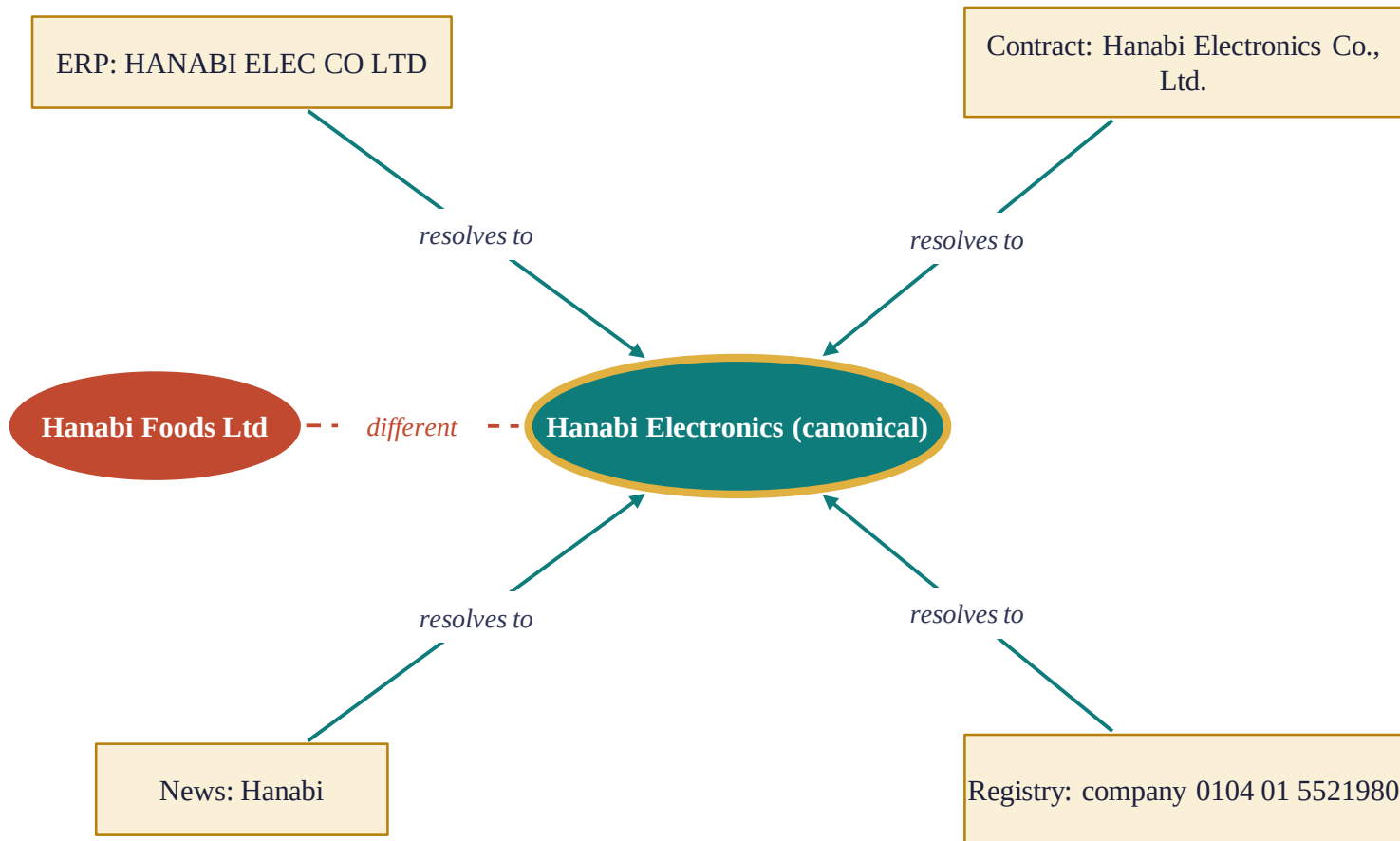
- Canonical entity identifiers
- Match links with confidence
- Merge and split audit trail

Guiding question: *Are these two records the same thing?*





The identity problem

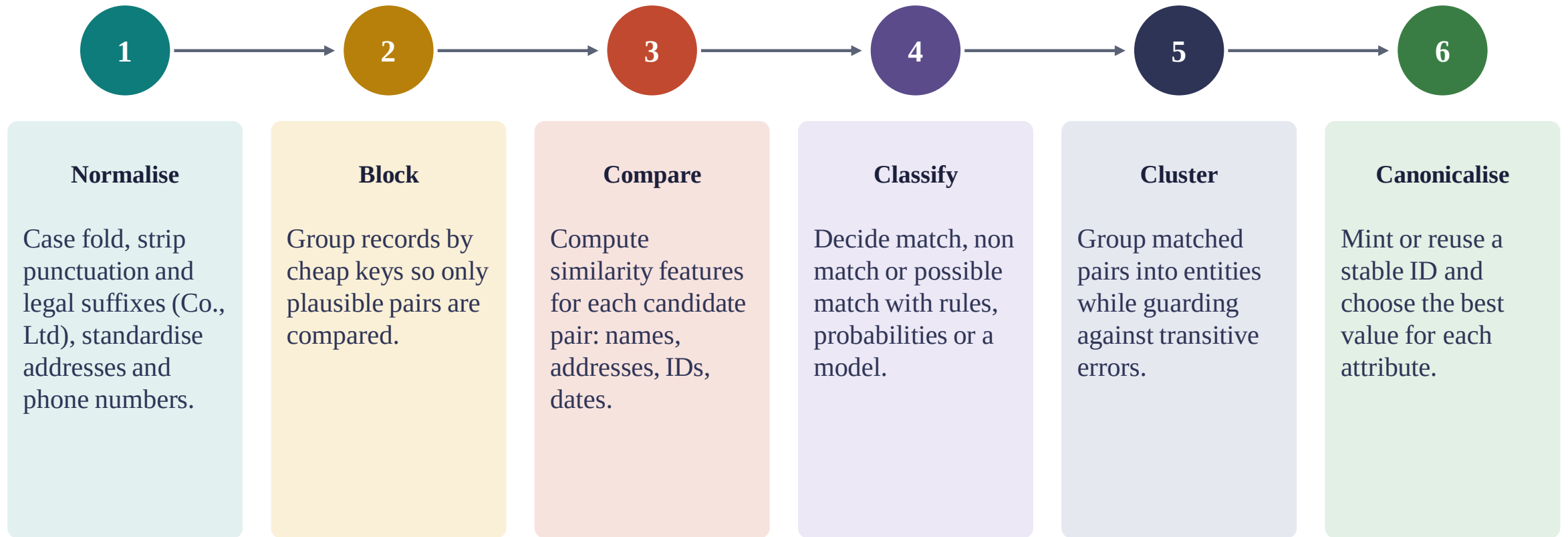


ANNOTATIONS

- **Variation:** abbreviations, legal suffixes, casing, transliteration, typos and missing fields.
- **Two error types:** a false merge fuses different entities; a false split leaves duplicates.
- **Asymmetric cost:** false merges are usually worse, because every downstream fact becomes wrong.
- **Scale:** the naive approach compares every pair of records, which is quadratic.
- **Terminology:** also called record linkage, deduplication or entity matching.



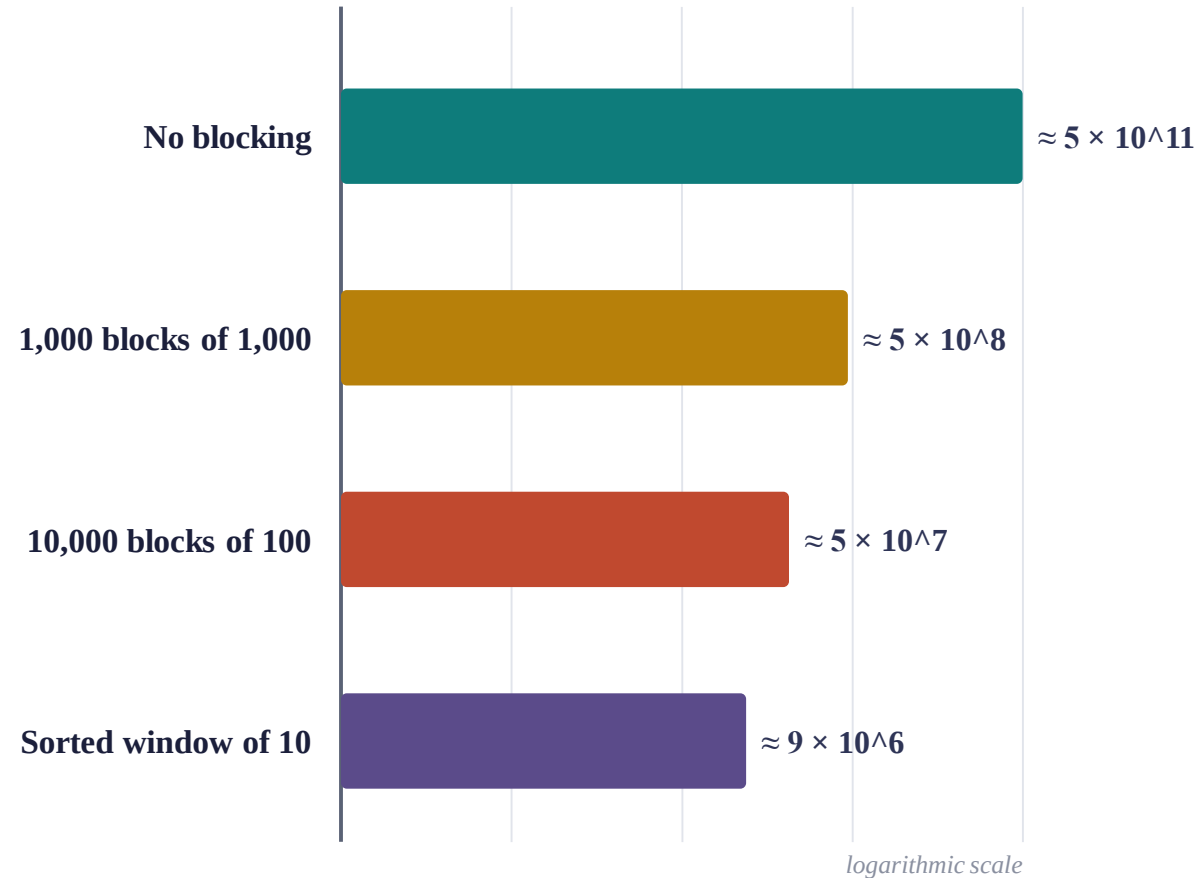
The entity resolution pipeline



Human in the loop: possible matches go to a review queue, and reviewer decisions become training data for the classifier.



Why blocking is essential: pairs to compare



Candidate pairs for one million records. Without blocking: n times (n minus 1), halved.

READING THE CHART

- **Blocking keys:** first letters of the name plus country, postcode, phonetic codes, or token overlap.
- **Sorted neighbourhood:** sort by a key and compare each record only with its nearest neighbours.
- **Learned blocking:** MinHash with locality sensitive hashing, or nearest neighbour search on embeddings.
- **Measure it:** reduction ratio (pairs avoided) and pair completeness (true matches kept). A block that drops true matches caps recall forever.



Similarity measures for comparing records

Measure	Works on	Example	Good for
Exact after normalising	IDs, codes	KR8841207 equals KR8841207	Strong identifiers
Levenshtein distance	Short strings	Jon versus John: one insertion, distance 1	Typos
Jaro Winkler	Person and company names	Rewards shared prefixes: Hanabi Elec versus Hanabi Electronics	Names
Jaccard on tokens	Multiword strings	$ A \cap B / A \cup B $ over word or character sets	Reordered words
TF IDF cosine	Longer text	Downweights common tokens such as Ltd or Group	Addresses, descriptions
Embedding cosine	Any text	Sentence embeddings capture Nippon versus Japan	Semantic variants
Numeric, date, geo	Values	Absolute difference, days apart, kilometres apart	Founding year, location

Feature vector: each candidate pair becomes a vector of similarities, the input to the classification step.



Four families of matching methods

1

Deterministic rules

If tax IDs are equal, or names match exactly and countries agree, then match. Transparent and fast, but rigid; rules multiply as exceptions appear.

2

Probabilistic (Fellegi and Sunter, 1969)

For each field estimate m , the probability of agreement among true matches, and u , among non matches. Sum $\log(m/u)$ weights and apply two thresholds. Can be fitted without labels using EM.

3

Supervised machine learning

Train gradient boosting or logistic regression on labelled pairs of similarity features. Active learning selects the most informative pairs for labelling.

4

Deep learning and LLMs

Pretrained transformers such as Ditto (2020) read both records as text; LLMs can judge difficult pairs with explanations. Accurate but costly: apply only to ambiguous pairs.



Worked example: the effect of normalisation

0.43

Token Jaccard before normalising: 3 shared tokens out of 7 distinct tokens

1.00

Token Jaccard after expanding St to Street and mapping Jon to John

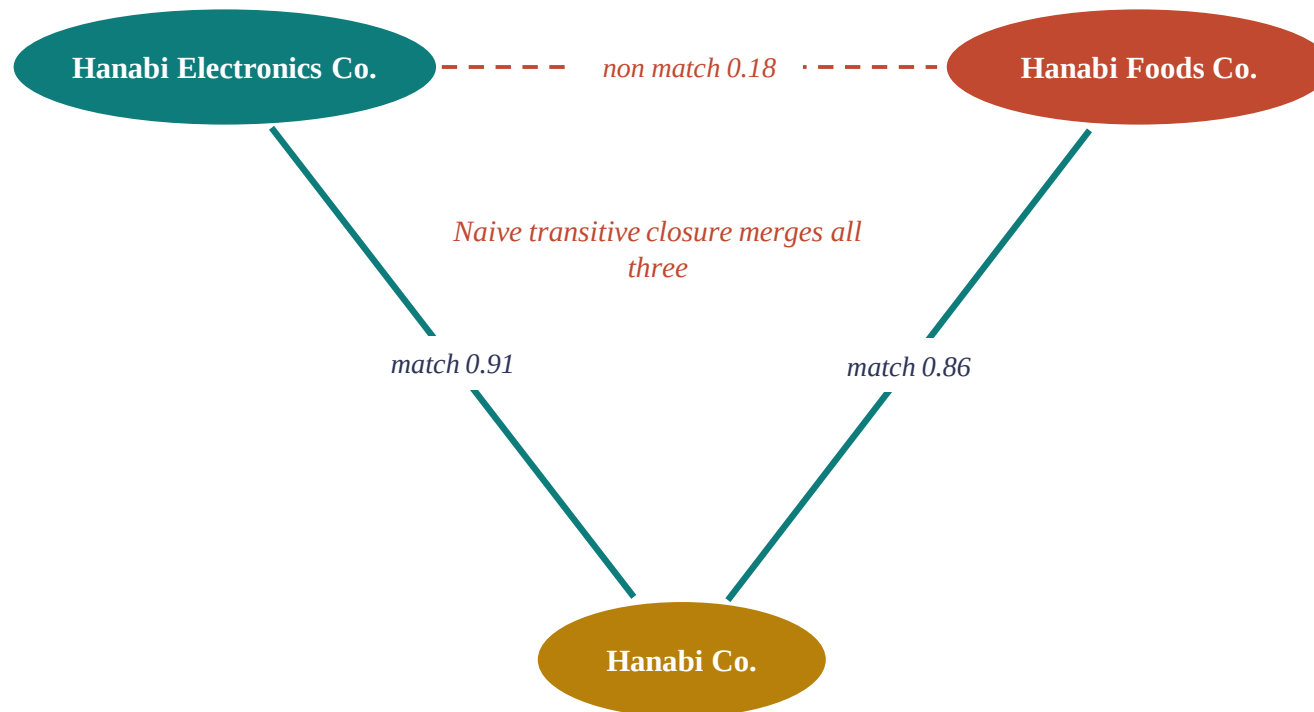
0.90

Levenshtein similarity of Jon Smith and John Smith: 1 edit over 10 characters

- **Record A:** Jon Smith, 12 High St. Tokens: jon, smith, 12, high, st.
- **Record B:** John Smith, 12 High Street. Tokens: john, smith, 12, high, street.
- **Shared:** smith, 12, high (3). Union: 7 distinct tokens. Jaccard = $3 / 7 = 0.43$.
- **Lesson:** cheap normalisation (abbreviation tables, nickname lists, suffix removal) often lifts accuracy more than a better classifier. Keep the original values for audit.



Clustering matches: the transitivity trap

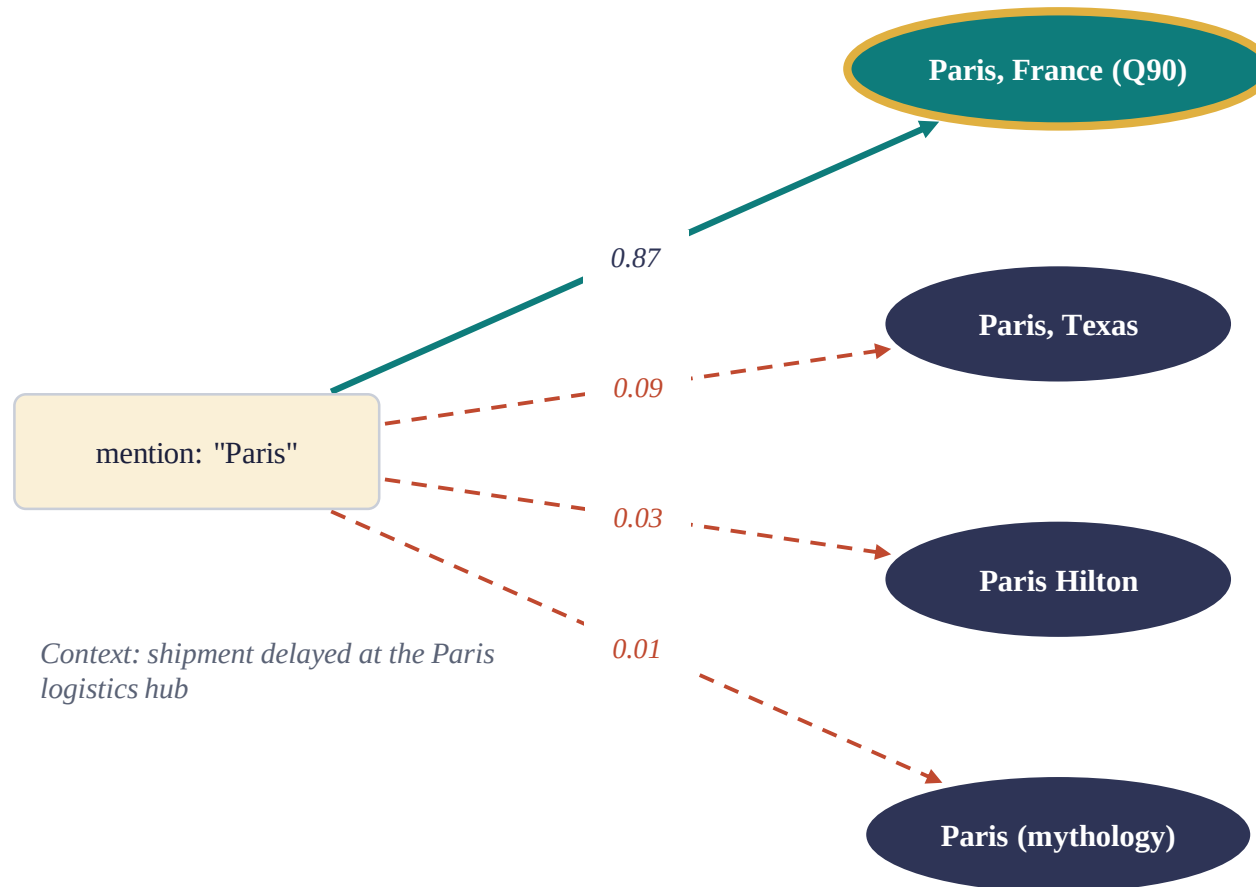


ANNOTATIONS

- **Problem:** pairwise decisions are not transitive. A short, vague record (Hanabi Co.) bridges two different companies.
- **Connected components:** simple but vulnerable to such bridges, producing giant clusters.
- **Better:** correlation clustering, centre or star clustering, or merge only when all pairs in a cluster agree.
- **Monitor:** the distribution of cluster sizes; unexpectedly large clusters almost always signal false merges.
- **Review:** send clusters with conflicting evidence to human curators.



Entity linking: mapping mentions to graph nodes



ANNOTATIONS

- **Mention detection:** find spans that may refer to entities (from NER).
- **Candidate generation:** look up alias tables built from labels, redirects and anchor texts.
- **Disambiguation:** combine popularity priors, context similarity, type compatibility and coherence with other linked entities.
- **NIL detection:** recognise when the entity is not yet in the graph and create a new node.
- **Tools:** REL, BLINK, spaCy entity linkers, or LLM reranking of candidates.

06

STAGE 6 OF 12

Knowledge Fusion and Integration

Combine facts about the same entity from many sources into one consistent, provenance rich graph, resolving conflicts and aligning schemas.

INPUTS

- Resolved entities
- Candidate facts with source metadata
- Source trust estimates
- Source schemas

KEY ACTIVITIES

- Schema and ontology matching
- Conflict detection
- Truth discovery and resolution
- Provenance capture
- Loading and merging

OUTPUTS

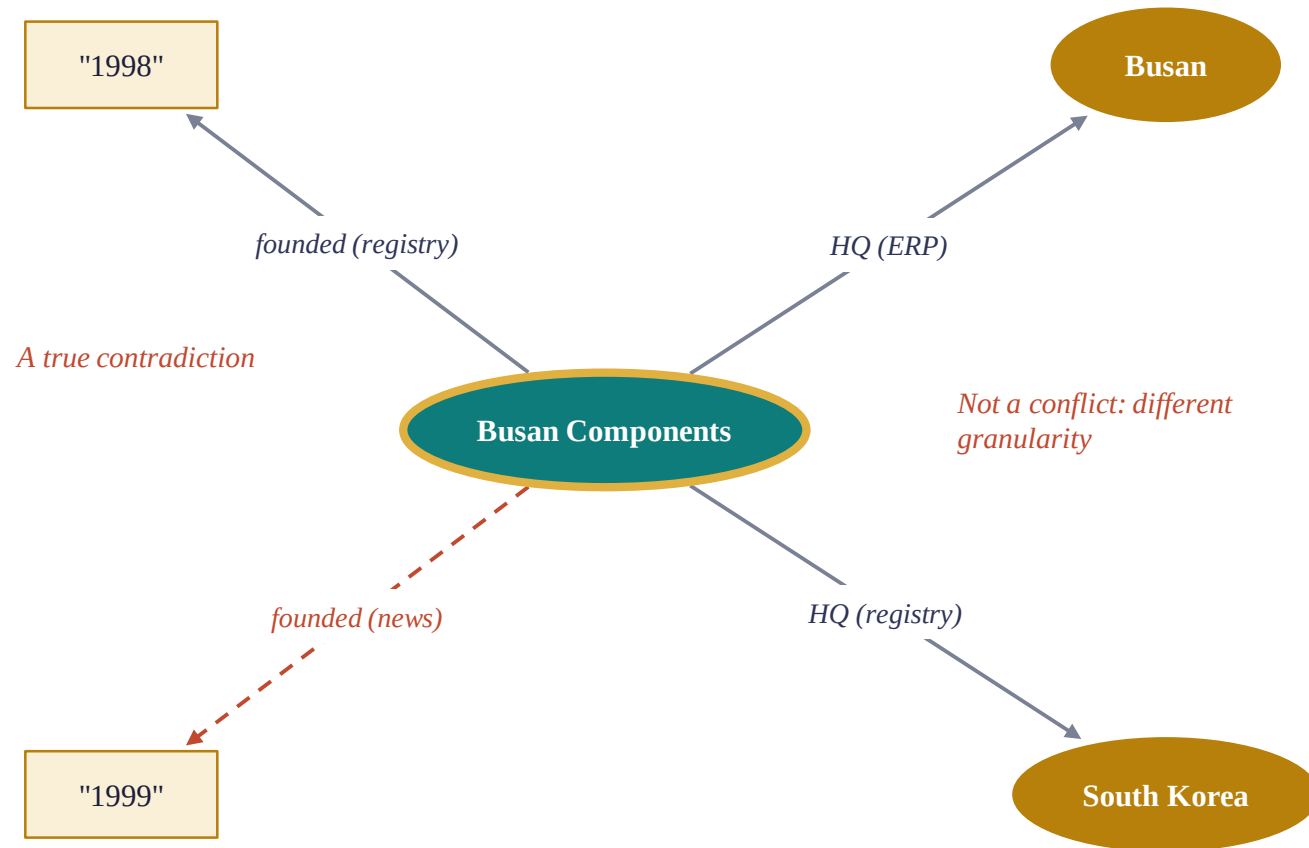
- Fused graph
- Per fact provenance and confidence
- Conflict and decision log

Guiding question: *When sources disagree, what do we believe, and why?*





Fusion in a picture: sources disagree



ANNOTATIONS

- **Fusion:** deciding which values to keep for each entity and attribute.
- **Not every difference is a conflict:** Busan lies in South Korea, so both HQ values can be true.
- **Real conflicts:** need a policy, not an ad hoc fix in a script.
- **Keep the losers:** store rejected values with provenance so decisions can be revisited.



Six kinds of apparent conflict

1 Contradiction

Values are mutually exclusive: founded in 1998 versus 1999. One source is wrong.

2 Granularity

Busan versus South Korea; a daily versus a monthly figure. Both can hold at different levels.

3 Temporal validity

Different CEOs from different years. Both were true; add validity intervals.

4 Units and formats

1,200 tonnes versus 1.2 kilotonnes. Normalise before comparing.

5 Uncertainty

An extracted fact with confidence 0.55 against a registry fact. Weigh by confidence.

6 Absence

One source is silent. Under the open world this is not a conflict at all.

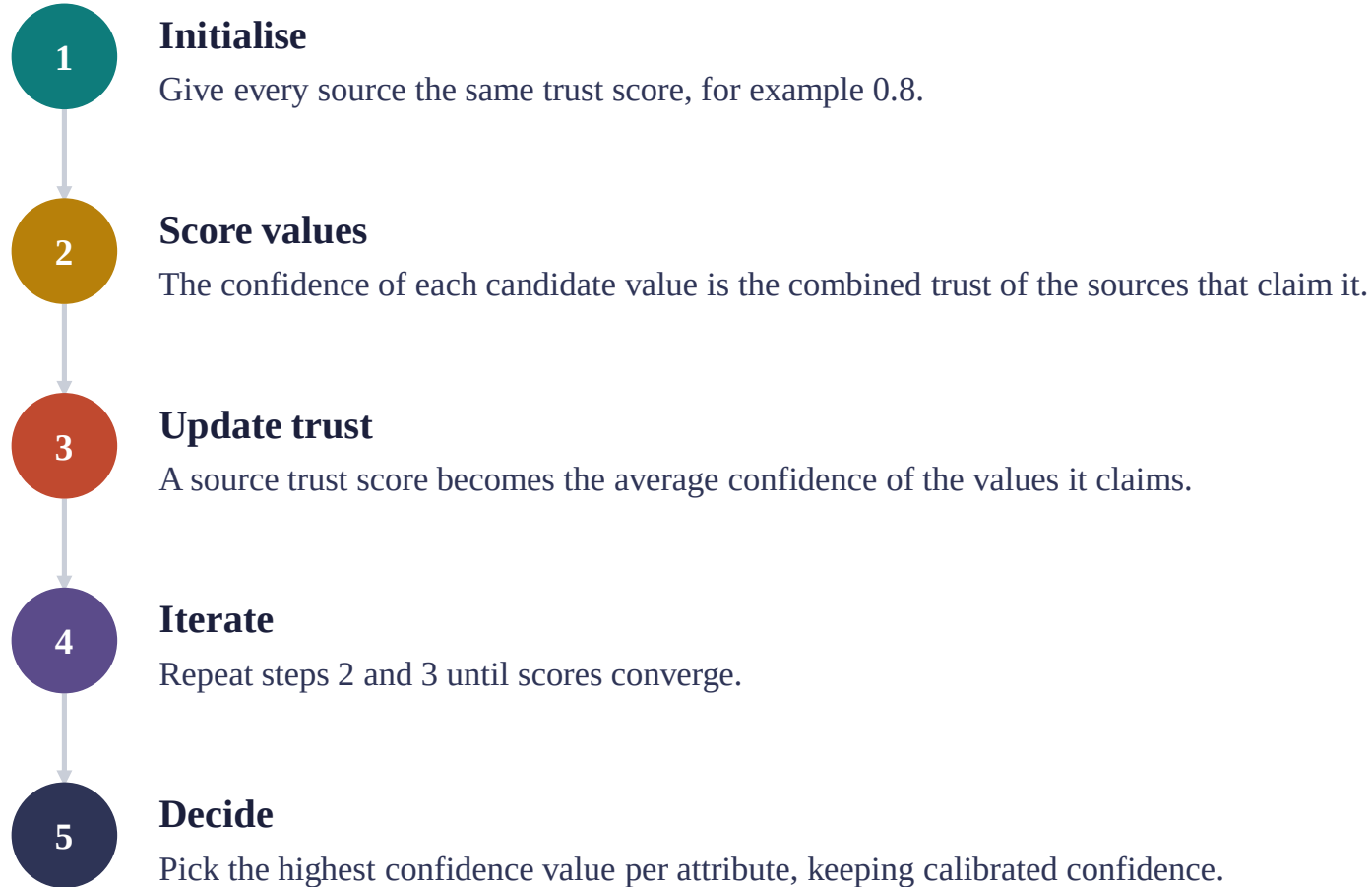


Conflict resolution strategies

Strategy	Rule	When appropriate	Risk
Source priority	Prefer the most authoritative source per attribute	Clear system of record exists	Authority is often attribute specific
Majority vote	Take the value most sources agree on	Many independent sources	Copying between sources inflates votes
Most recent	Take the latest observed value	Fast changing attributes	Recent is not always correct
Most specific	Prefer the finer grained compatible value	Hierarchical values such as places	Needs a hierarchy
Keep all	Store all values with provenance and let queries choose	Disagreement is itself informative	Pushes work to consumers
Truth discovery	Jointly estimate source trust and value truth	Many sources of unknown quality	Needs enough overlap to converge



Truth discovery: learning which sources to trust

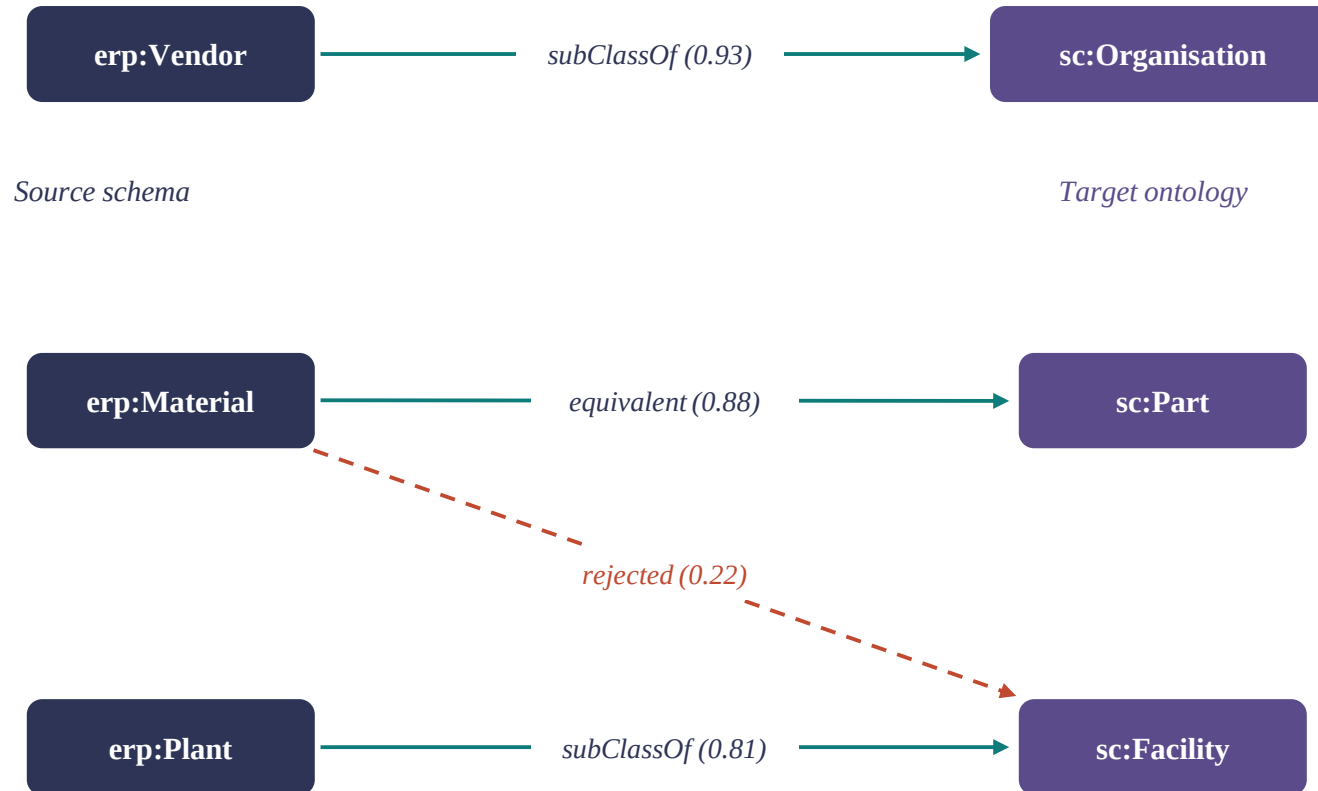


KEY POINTS

- **Intuition:** reliable sources claim true values, and true values are claimed by reliable sources; the algorithm resolves this circularity iteratively.
- **Classic work:** TruthFinder (Yin, Han and Yu, 2007).
- **At web scale:** Google Knowledge Vault (Dong et al., 2014) fused extractor outputs with learned priors.
- **Watch for:** sources copying each other; dependence detection prevents false consensus.



Schema and ontology alignment



ANNOTATIONS

- **Lexical:** compare labels, synonyms and definitions.
- **Structural:** compare positions in the hierarchy and attached properties.
- **Instance based:** classes that share many instances are likely related.
- **Learned:** embeddings and LLMs propose candidates; experts confirm them.
- **Standards:** record mappings in SSSOM with confidence and justification; the OAEI campaign benchmarks matchers such as LogMap and AgreementMakerLight.



An integration pipeline architecture

Serving

SPARQL and Cypher endpoints, APIs, search indexes, feature stores

Graph store

Versioned production graph with named graphs per source

Fuse and validate

Conflict resolution, SHACL validation, quality metrics

Resolve

Entity resolution service and canonical identifier registry

Transform

Mappings, NLP and LLM extraction, normalisation

Ingest

Connectors, change data capture, crawlers, raw snapshot storage

Sources

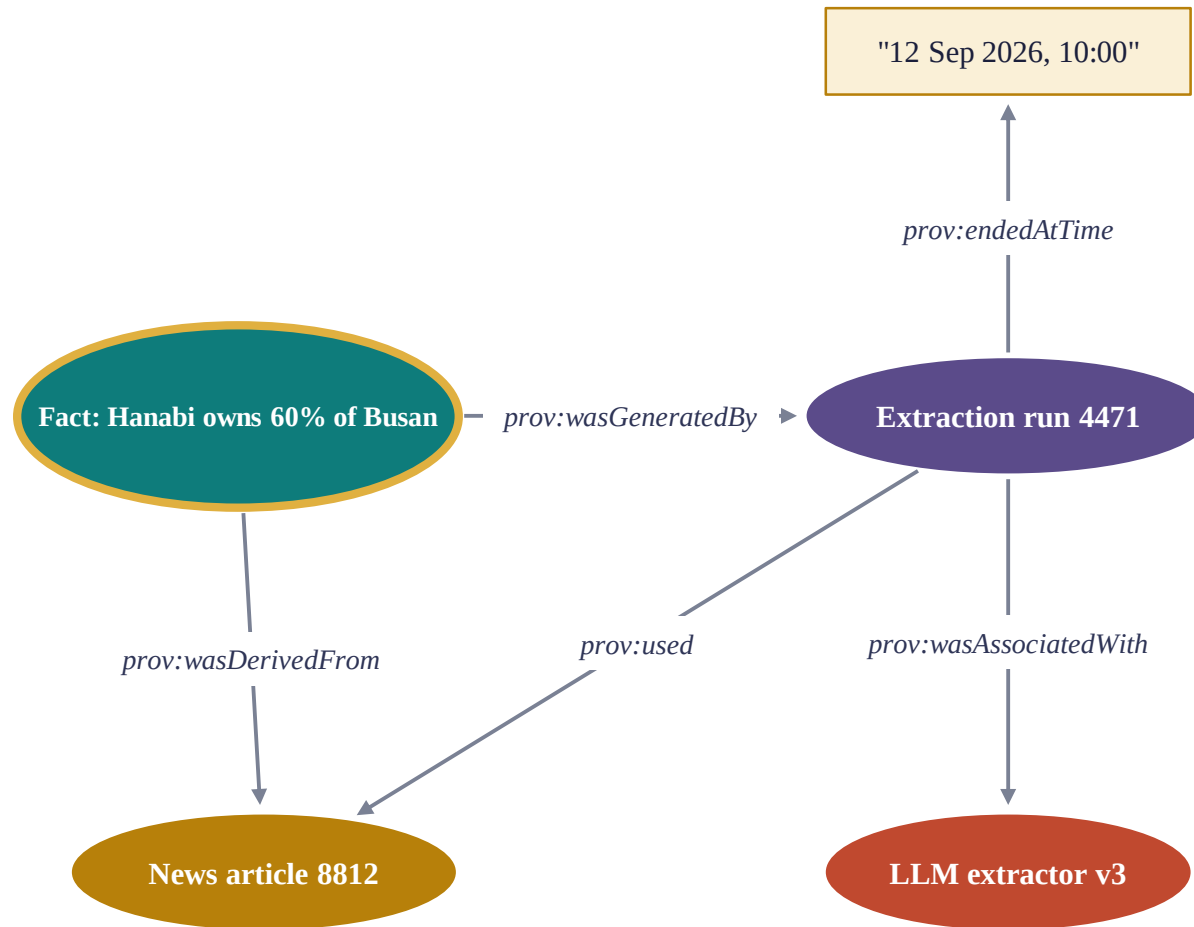
Databases, files, APIs, documents, open data

KEY POINTS

- **Staging zones:** raw, transformed and validated layers, as in a lakehouse.
- **Idempotence:** every step can be rerun safely, which makes recovery simple.
- **Orchestration:** Airflow, Dagster or Prefect schedule and monitor the flow.
- **Quality gates:** validation failures stop promotion to production.



Recording provenance with PROV O



ANNOTATIONS

- **PROV O:** a W3C Recommendation (2013) with three core classes: Entity, Activity and Agent.
- **Granularity:** per fact (via reification or triple terms) or per batch (via named graphs).
- **Uses:** trust, debugging, licence compliance, targeted deletion and explanation of answers.
- **Rule:** no fact enters the graph without source, method and time.



Batch rebuilds versus incremental integration

Batch rebuild

- Rebuild the whole graph from sources on a schedule.
- Simple, reproducible and easy to reason about.
- Fusion decisions are recomputed consistently each time.
- Latency of hours or days; costly for very large graphs.

vs

Incremental integration

- Apply only the changes since the last run.
- Fresh data within minutes; lower compute per update.
- Must handle deletions, retractions and re resolution of entities.
- Harder to guarantee the same result as a rebuild.

Common practice: incremental updates day to day, plus a periodic full rebuild to verify and correct drift.

07

STAGE 7 OF 12

Storage and Infrastructure

Choose and configure the database, indexing, partitioning and transaction model that will hold the graph and serve its workloads.

INPUTS

- Volume and growth estimates
- Query patterns from the CQs
- Reasoning requirements
- Latency, availability and security needs

KEY ACTIVITIES

- RDF or property graph decision
- Engine selection and benchmarking
- Index and partition design
- Bulk and incremental loading
- Backup and high availability

OUTPUTS

- Platform decision record
- Deployed and tuned store
- Performance baseline

Guiding question: *Where will the graph live, and can it answer our questions fast enough?*





Storage options for knowledge graphs

1 RDF triple and quad stores

Store triples or quads with permutation indexes; SPARQL, reasoning and named graphs built in.

2 Property graph databases

Nodes and relationships with properties; Cypher, GQL or Gremlin; strong traversal performance.

3 Multi model databases

Document, key value and graph in one engine; convenient, sometimes less specialised.

4 Relational with graph layer

Virtual knowledge graphs over SQL, or SQL/PGQ graph queries from SQL:2023.

5 Distributed graph analytics

Spark GraphFrames and parallel engines for whole graph algorithms at huge scale.

6 Companion indexes

Full text and vector indexes beside the graph for search, hybrid retrieval and GraphRAG.



A survey of widely used graph engines

Engine	Model	Query languages	Notable traits
GraphDB	RDF	SPARQL	Rule based materialisation (for example OWL 2 RL), full text connectors
Stardog	RDF	SPARQL, GraphQL	Virtual graphs over external sources, reasoning at query time
Virtuoso	RDF and relational	SPARQL, SQL	Hosts large public endpoints such as DBpedia
Apache Jena (TDB2, Fuseki)	RDF	SPARQL	Open source Java framework with rule engine
Amazon Neptune	RDF and LPG	SPARQL, Gremlin, openCypher	Fully managed on AWS
Neo4j	LPG	Cypher, GQL	Native graph storage and a graph data science library
TigerGraph	LPG	GSQL	Distributed, parallel deep link analytics

Always benchmark: run your own competency question workload on your own data; published benchmarks rarely match real query mixes.



How triple stores index: the six permutations

Systems such as Hexastore and RDF 3X keep sorted indexes for orderings of subject (S), predicate (P) and object (O), so any triple pattern becomes a range scan.

1 SPO

Subject bound: everything about ex:Hanabi.

2 SOP

Subject and object bound: how is Hanabi related to Busan?

3 PSO

Predicate first: all suppliesPart edges, grouped by supplier.

4 POS

Predicate and object bound: who supplies Part 7731?

5 OSP

Object bound: everything that points at ex:Tokyo.

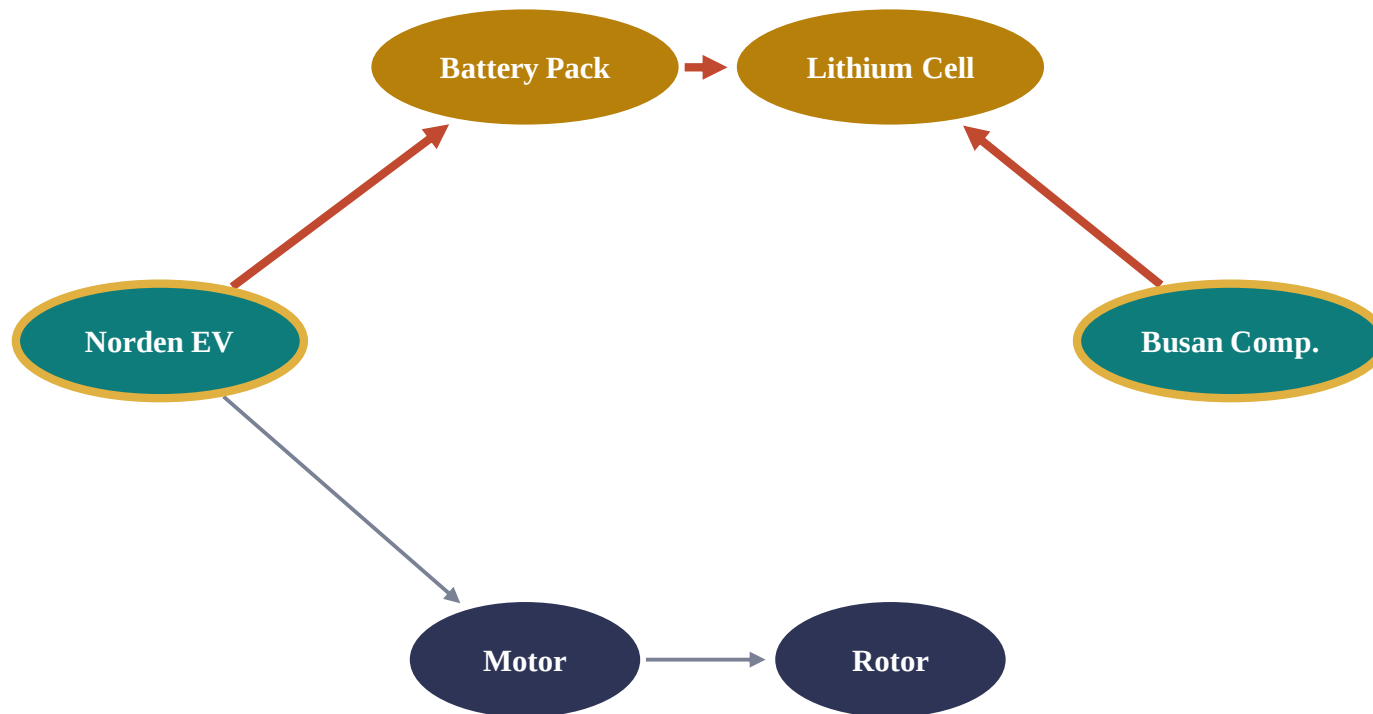
6 OPS

Object and predicate: what is located in Tokyo?



Index free adjacency in property graph stores

Traversal follows stored pointers

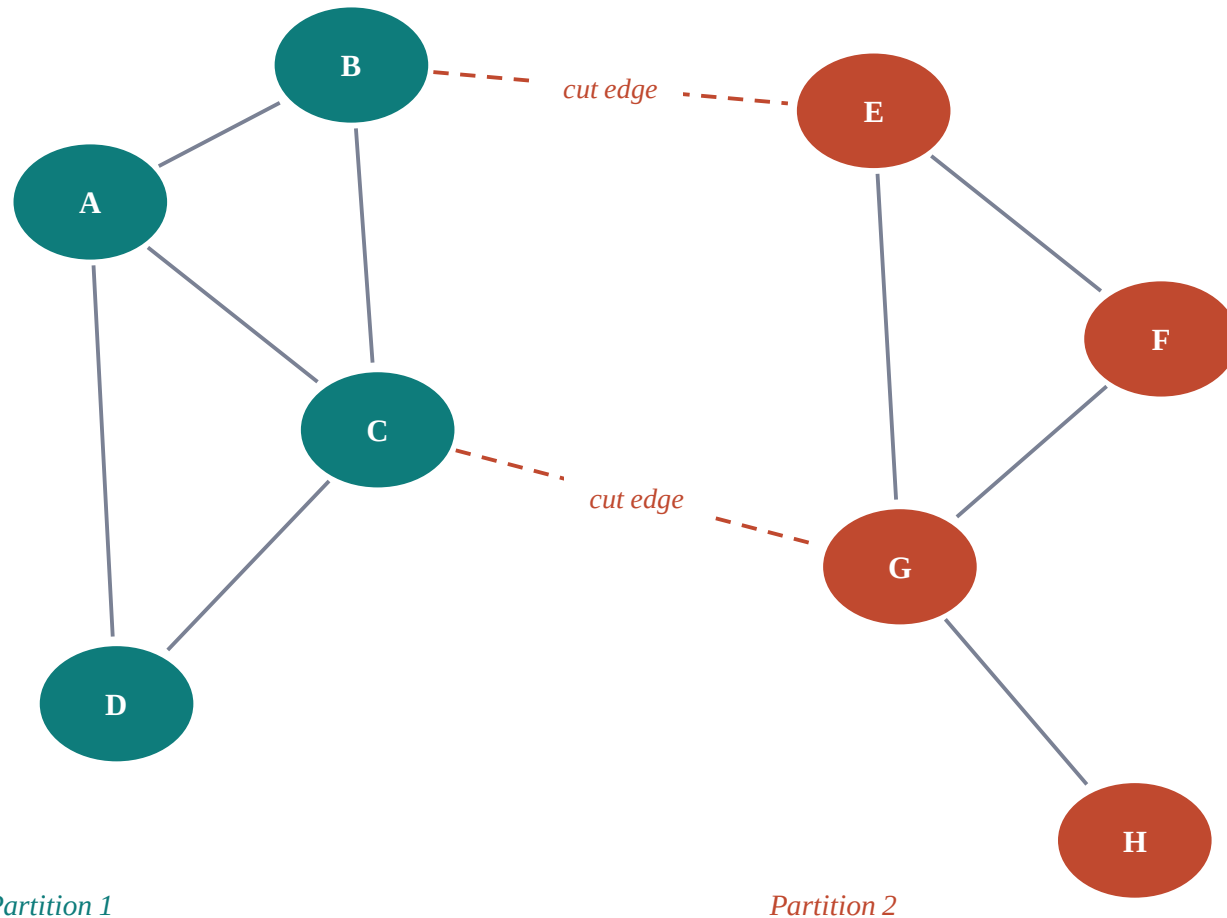


ANNOTATIONS

- **Idea:** each node record stores direct references to its relationships, so moving to a neighbour is a pointer hop, not an index lookup.
- **Consequence:** traversal cost grows with the part of the graph visited, not with the total graph size.
- **Relational contrast:** each join probes an index, costing roughly $\log n$ per step.
- **Nuance:** modern RDF stores also traverse efficiently; the physical design matters more than the label.



Scaling out: partitioning a graph



ANNOTATIONS

- **Goal:** balanced partitions with as few cut edges as possible, because each cut edge costs a network hop.
- **Edge cut versus vertex cut:** split edges between machines, or replicate high degree vertices across them.
- **Methods:** hash partitioning is simple but ignores structure; METIS style and community aware methods keep neighbours together.
- **Pragmatics:** many enterprise graphs fit on one large machine with replicas for reads; scale up before scaling out.



Decision matrix: triple store or property graph?

	RDF triple store	Property graph DB
Standards and interoperability	● ● ● ● ●	● ● ● ● ●
Formal reasoning	● ● ● ● ●	● ● ● ● ●
Deep traversal performance	● ● ● ● ●	● ● ● ● ●
Attributes on relationships	● ● ● ● ●	● ● ● ● ●
Developer familiarity	● ● ● ● ●	● ● ● ● ●
Cross organisation integration	● ● ● ● ●	● ● ● ● ●
Graph algorithms ecosystem	● ● ● ● ●	● ● ● ● ●

Illustrative scores for typical products; weight each criterion by your own requirements before deciding.

INTERPRETATION

- **Weight first:** multiply each score by the importance of the criterion for your use case.
- **Running example:** integration of registries and sanctions data and ownership reasoning favour RDF.
- **Hybrids:** some teams keep an RDF system of record and project subgraphs into a property graph for analytics.



A reference architecture for a production graph

Consumers

Applications, BI tools, notebooks, LLM agents

Access

SPARQL and Cypher endpoints, GraphQL and REST APIs, vector and text search

Semantic services

Reasoner, SHACL validator, entity resolution service, ID registry

Storage

Graph database with replicas, search index, object store for raw snapshots

Pipelines

Orchestration, streaming, mapping and extraction jobs, CI for ontologies

Cross cutting

Identity and access management, monitoring, backup, lineage

KEY POINTS

- **Transactions:** choose ACID where updates come from applications; batch loads can use bulk loaders.
- **Named graphs:** separate sources, inferred facts and predictions for access control and rollback.
- **Availability:** replicas for read scaling, tested backups and point in time restore.
- **Observability:** log query plans and latencies from day one.



Production readiness checklist for a graph store



Bulk loading: use the engine's offline loader for initial builds; it is often ten times faster than transactional inserts.



Query limits: timeouts, result caps and memory quotas protect the store from runaway recursion.



Named graphs: load each source and each release into its own graph so it can be replaced or retracted atomically.



Benchmark: test with your own queries and data volumes; public benchmarks such as LDBC SNB guide but do not decide.



Backups: schedule snapshots and test restores; a backup never restored is only a hope.



Security: authentication, encryption in transit and at rest, and graph level permissions from day one.



High availability: replicas behind a load balancer keep reads alive during failures and upgrades.



Observability: export query latency, cache hit rates and load times to the monitoring stack.

Rule of thumb: size memory so the hot indexes fit in RAM; graph traversals degrade sharply once they hit disk.

08

STAGE 8 OF 12

Reasoning and Enrichment

Derive implicit knowledge from explicit facts using ontological semantics, rules and graph analytics, and enrich the graph with it.

INPUTS

- Populated graph
- Ontology axioms
- Business rules
- External reference graphs

KEY ACTIVITIES

- Choosing an OWL profile
- Materialisation or query rewriting
- Authoring rules
- Running graph algorithms
- Linking to external graphs

OUTPUTS

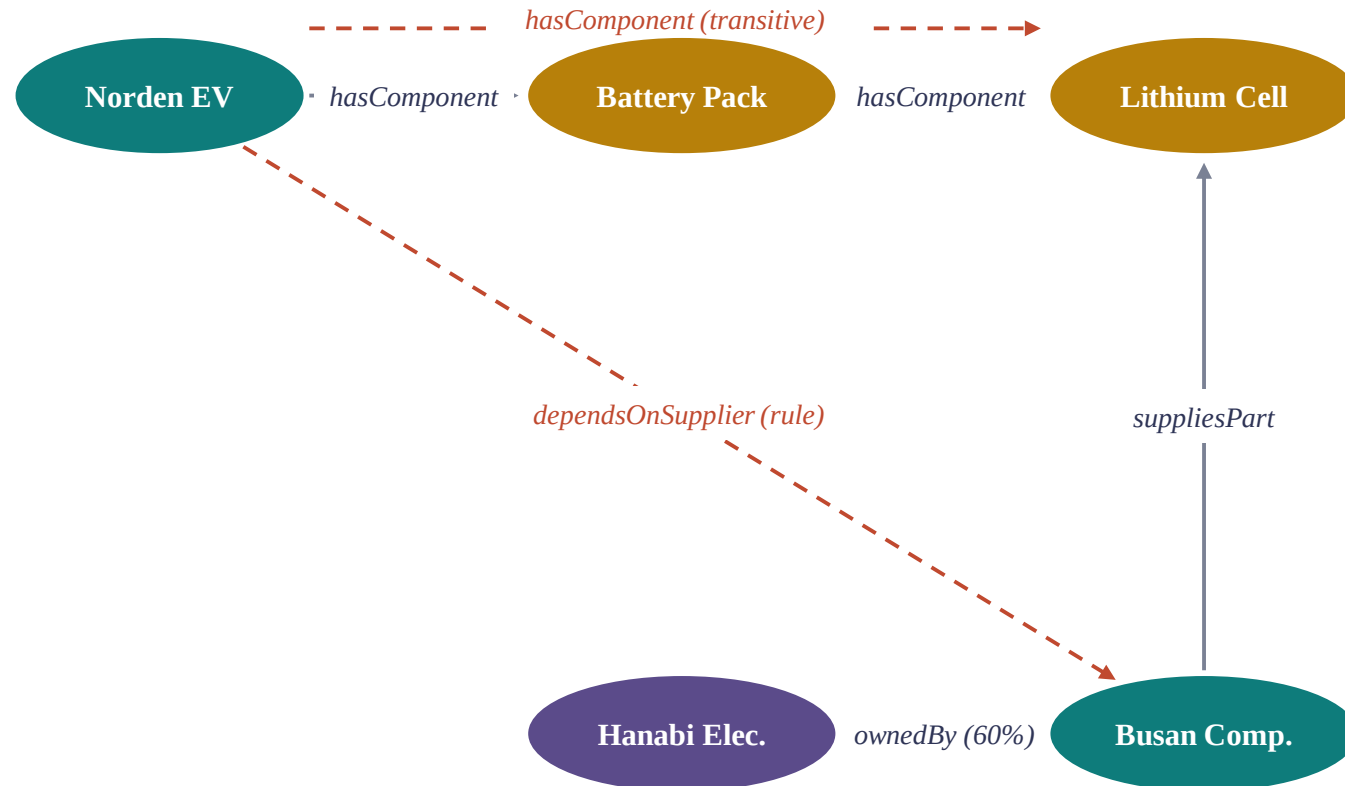
- Inferred triples, clearly flagged
- Rule library under test
- Enriched attributes and scores
- Consistency reports

Guiding question: *What does the graph know that nobody wrote down?*





Reasoning makes implicit knowledge explicit



Dashed: inferred, never typed in

ANNOTATIONS

- **Explicit facts:** four solid edges loaded from sources.
- **Axiom:** *hasComponent* is transitive, so Norden EV has Lithium Cell as a component.
- **Rule:** a product depends on every supplier of any of its components.
- **Result:** CQ1 is now a one hop lookup instead of a recursive query.
- **Governance:** store inferred facts separately so they can be recomputed or retracted.



RDFS entailment rules in practice

Rule	If the graph contains	Then infer
rdfs2	$p \text{ rdfs:domain } C \text{ and } x \text{ p } y$	$x \text{ rdf:type } C$
rdfs3	$p \text{ rdfs:range } C \text{ and } x \text{ p } y$	$y \text{ rdf:type } C$
rdfs5	$p \text{ rdfs:subPropertyOf } q \text{ and } q \text{ rdfs:subPropertyOf } r$	$p \text{ rdfs:subPropertyOf } r$
rdfs7	$p \text{ rdfs:subPropertyOf } q \text{ and } x \text{ p } y$	$x \text{ q } y$
rdfs9	$C \text{ rdfs:subClassOf } D \text{ and } x \text{ rdf:type } C$	$x \text{ rdf:type } D$
rdfs11	$C \text{ rdfs:subClassOf } D \text{ and } D \text{ rdfs:subClassOf } E$	$C \text{ rdfs:subClassOf } E$

Example: Company subClassOf Organisation, and Busan is a Company; rdfs9 gives Busan type Organisation, so queries for organisations find it.



OWL 2 profiles: trading expressivity for tractability

Profile	Designed for	Reasoning cost	Typical use
OWL 2 EL	Very large class hierarchies with existential restrictions	Polynomial time classification	Biomedical ontologies such as SNOMED CT
OWL 2 QL	Answering queries by rewriting them into SQL	Very low data complexity (in AC0)	Virtual graphs over relational data
OWL 2 RL	Implementation with forward chaining rules	Polynomial time	Materialisation in triple stores
OWL 2 DL	Full expressivity of SROIQ with decidability	N2EXPTIME complete in the worst case	Ontology design and classification
RDFS plus	RDFS with a few OWL features such as inverse and transitive properties	Low	Pragmatic enterprise graphs

Advice: choose the smallest profile that answers your competency questions; expressivity you do not use is cost you still pay.



Materialisation versus query rewriting

Forward chaining: materialise

- Compute all inferences at load time and store them.
- Queries are fast and simple: inferred facts are just data.
- Storage grows, sometimes several times over.
- Updates and deletions require incremental maintenance, for example the DRed algorithm.

vs

Backward chaining: rewrite

- Expand each query at run time to cover what could be inferred.
- No extra storage; always consistent with current data.
- Queries become larger and slower, especially with deep hierarchies.
- Natural fit for virtual graphs over live databases (OWL 2 QL).

Hybrid: materialise stable, frequently used inferences; rewrite for rarely used or fast changing ones.



Business rules as SPARQL CONSTRUCT

SPARQL

```
PREFIX sc: <http://example.org/supply#>

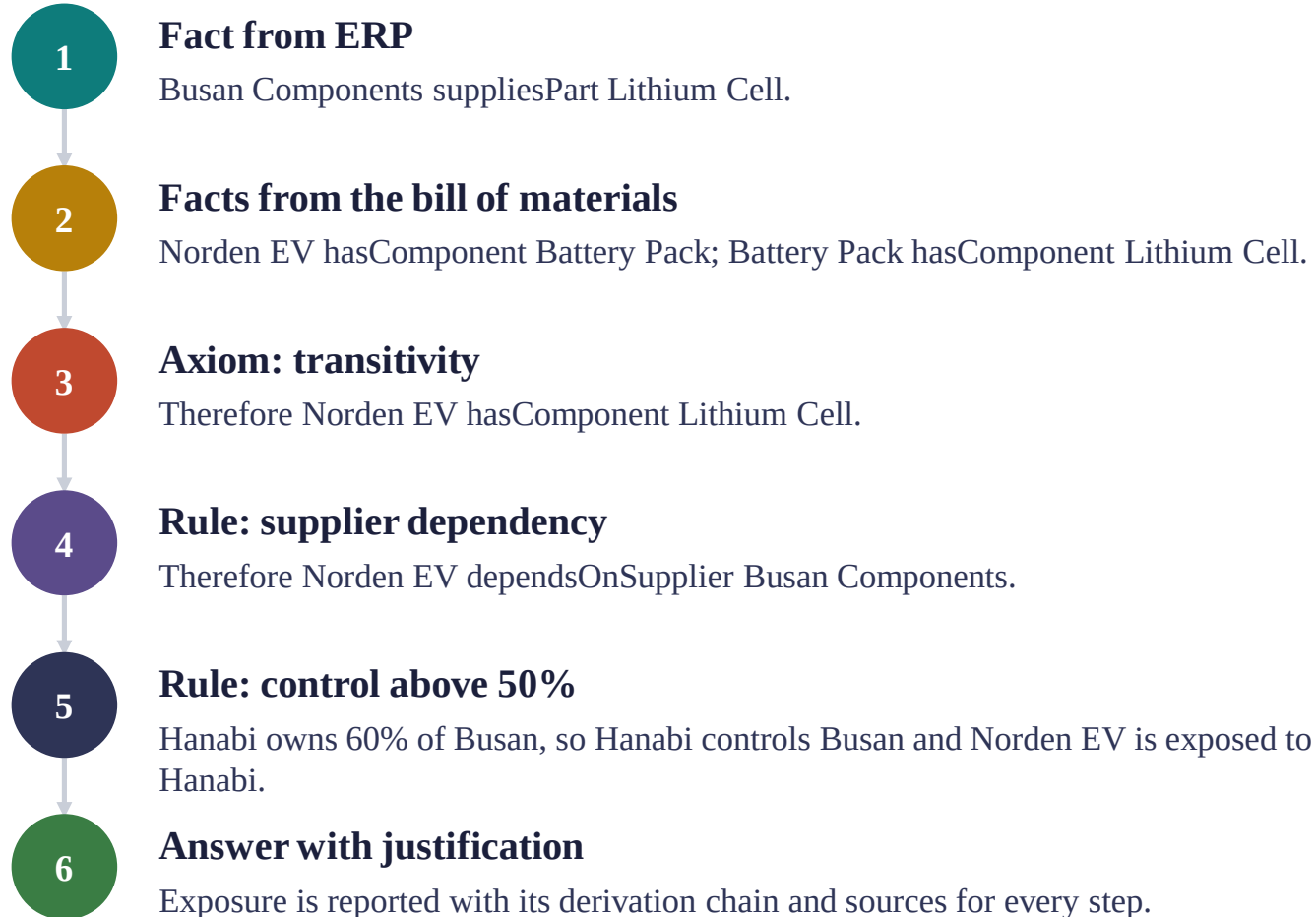
# Rule: a product depends on every supplier
# of any of its parts, at any depth
CONSTRUCT {
  ?product sc:dependsOnSupplier ?supplier .
}
WHERE {
  ?product a sc:Product ;
           sc:hasComponent+ ?part .
  ?supplier sc:suppliesPart ?part .
  FILTER NOT EXISTS {
    ?supplier sc:status sc:Terminated
  }
}
```

HOW TO READ IT

- **hasComponent+**: a property path meaning one or more hops.
- **FILTER NOT EXISTS**: excludes terminated suppliers.
- **Materialise**: run as INSERT into a named graph such as graph:inferred.
- **Alternatives**: SWRL, Datalog engines such as RDFox, SHACL rules, or Cypher procedures.
- **Test rules**: like code, with fixture graphs and expected outputs.



A worked inference chain



KEY POINTS

- **Explainability:** each inferred fact can carry a justification, the minimal set of facts and axioms that entail it.
- **Trust:** users accept derived answers far more readily when they can inspect the chain.
- **Debugging:** a wrong conclusion is traced to a wrong fact or an overly broad rule.



Enrichment beyond logical inference

1 External linking

Attach Wikidata IDs, GeoNames coordinates and LEIs to inherit context and identifiers.

2 Derived attributes

Counts, ages, flags and aggregates: number of single source parts per product.

3 Centrality

PageRank, betweenness and degree identify critical suppliers and chokepoints.

4 Community detection

Louvain and Leiden find clusters such as tightly linked supplier groups.

5 Similarity

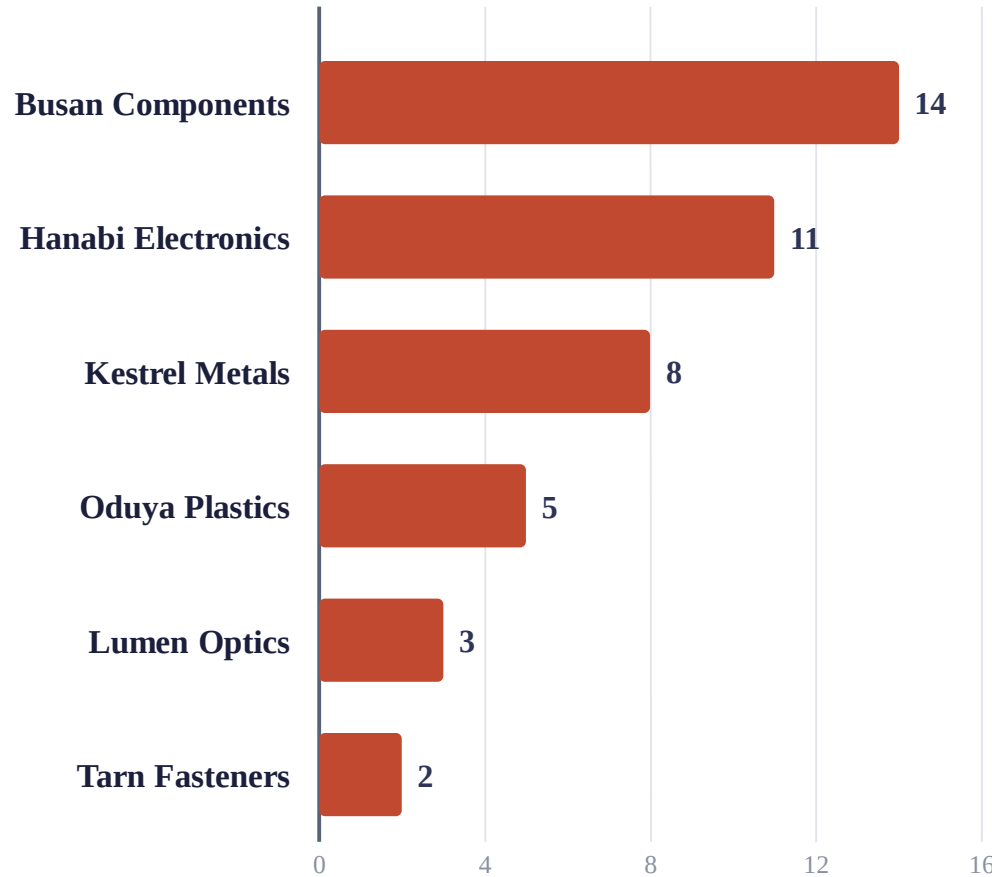
Node similarity and embeddings find substitute suppliers and parts.

6 Classification

Graph neural networks label nodes, for example predicted risk tiers.



Enrichment example: supplier criticality



Illustrative output of the dependsOnSupplier rule: number of finished products depending on each supplier.

READING THE CHART

- **Derived signal:** one rule turns raw supply edges into a ranking the business can act on.
- **Combine signals:** multiply dependency counts by risk indicators such as sanctions proximity or distress news.
- **Centrality adds nuance:** betweenness highlights suppliers that bridge otherwise separate parts of the network.
- **Store with provenance:** write scores back as attributes with the rule version and run date.

09

STAGE 9 OF 12

Quality Assessment and Validation

Measure how fit the graph is for its intended use, enforce structural constraints automatically, and estimate factual accuracy with statistically sound sampling.

INPUTS

- The integrated and enriched graph
- Ontology and business rules
- Competency questions
- A gold standard or human reviewers

KEY ACTIVITIES

- Define quality dimensions and metrics
- Write SHACL shapes
- Validate on every load
- Sample and audit accuracy
- Profile completeness

OUTPUTS

- Validation reports
- Accuracy estimates with intervals
- A quality dashboard and release gate

Guiding question: *Is the graph correct and complete enough for the decisions it supports?*





Quality is fitness for use: four categories

Zaveri et al. (2016) surveyed Linked Data quality and organised 18 dimensions into four categories. Quality is always judged against the use case.

1 Accessibility

Availability, licensing, interlinking, security and performance. Can consumers reach the data, and are they allowed to use it?

2 Intrinsic

Syntactic validity, semantic accuracy, consistency, conciseness and completeness. Is the data correct in itself, independent of any user?

3 Contextual

Relevancy, trustworthiness, understandability and timeliness. Is the data suitable for this task at this moment?

4 Representational

Conciseness of representation, interoperability, interpretability and versatility. Is the data expressed in a usable way?



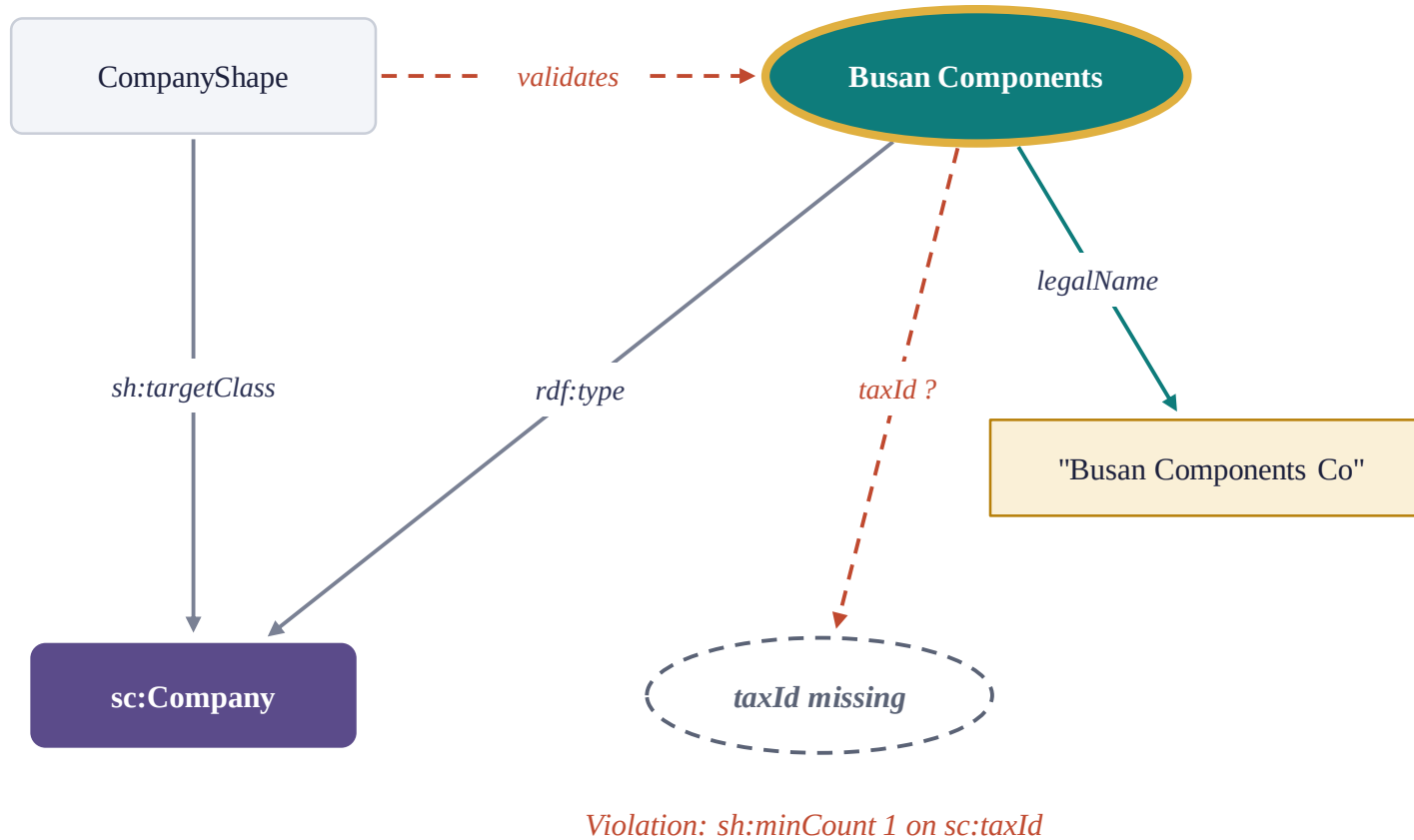
Turning dimensions into measurable metrics

Dimension	Metric	How it is computed	Example target
Accuracy	Share of sampled triples judged correct	Stratified random sample reviewed by experts	At least 95%
Completeness	Share of entities with a required property	COUNT of entities with the property over entities of the class	taxId at least 90%
Consistency	Constraint violations per thousand triples	SHACL report and OWL reasoner inconsistencies	Below 0.5
Conciseness	Duplicate entity rate	Pairs above the match threshold that were not merged	Below 1%
Timeliness	Age of the newest fact from each source	Load timestamp minus source timestamp, in hours	Below 24 hours
Interlinking	Share of entities linked to a reference ID	Entities with owl:sameAs or an LEI over all entities	At least 80%
Trustworthiness	Share of facts carrying provenance	Triples inside named graphs with PROV O metadata	100%

Principle: every metric needs a definition, a query that computes it, an owner and a threshold. Otherwise quality is only an opinion.



SHACL: validating the graph against shapes



ANNOTATIONS

- **SHACL:** the W3C Shapes Constraint Language, a Recommendation since 2017.
- **Shapes:** describe what a valid node looks like: required properties, types, cardinalities and patterns.
- **Targets:** *sh:targetClass* applies a shape to every instance of a class.
- **Closed world check:** SHACL deliberately tests what is present, complementing OWL, which assumes the open world.
- **Output:** a machine readable validation report, itself an RDF graph.



Writing a shape in Turtle

Turtle

```
sc:CompanyShape a sh:NodeShape ;
  sh:targetClass sc:Company ;
  sh:property [
    sh:path sc:legalName ;
    sh:minCount 1 ; sh:maxCount 1 ;
    sh:datatype xsd:string ] ;
  sh:property [
    sh:path sc:taxId ;
    sh:minCount 1 ;
    sh:pattern "^\\w{8,12}$" ] ;
  sh:property [
    sh:path sc:ownedBy ;
    sh:class sc:Organisation ] ;
  sh:property [
    sh:path sc:website ;
    sh:minCount 1 ;
    sh:severity sh:Warning ] .
```

HOW TO READ IT

- **Cardinality:** minCount and maxCount enforce exactly one legal name.
- **Datatype:** legalName must be a plain string, not a number or a date.
- **Pattern:** taxId must be 8 to 12 word characters.
- **Class:** the object of ownedBy must be an Organisation.
- **Severity:** a missing website is a Warning, not a Violation, so it reports without blocking a release.
- **Engines:** TopBraid SHACL, pySHACL, RDF4J and most triple stores.



Reading a validation report

Focus node	Path	Constraint	Severity	Message	Likely root cause
Busan Components	sc:taxId	MinCount 1	Violation	Less than 1 value	Registry feed not yet matched
Kestrel Metals	sc:legalName	MaxCount 1	Violation	More than 1 value	Fusion kept both source names
Oduya Plastics	sc:ownedBy	Class Organisation	Violation	Value is a Person	Owner typed wrongly at extraction
Tarn Fasteners	sc:taxId	Pattern	Violation	Does not match pattern	Spaces kept by the parser
Lumen Optics	sc:website	MinCount 1	Warning	Less than 1 value	Not captured in ERP

Triage by root cause: each violation points back to a pipeline stage. Fix the stage, not the triple, or the error will return on the next load.



Estimating accuracy by sampling

385

triples sampled: enough for a margin of 5 points at 95% confidence

354

judged correct by two independent reviewers

92.0%

estimated accuracy of the population of triples

± 2.7

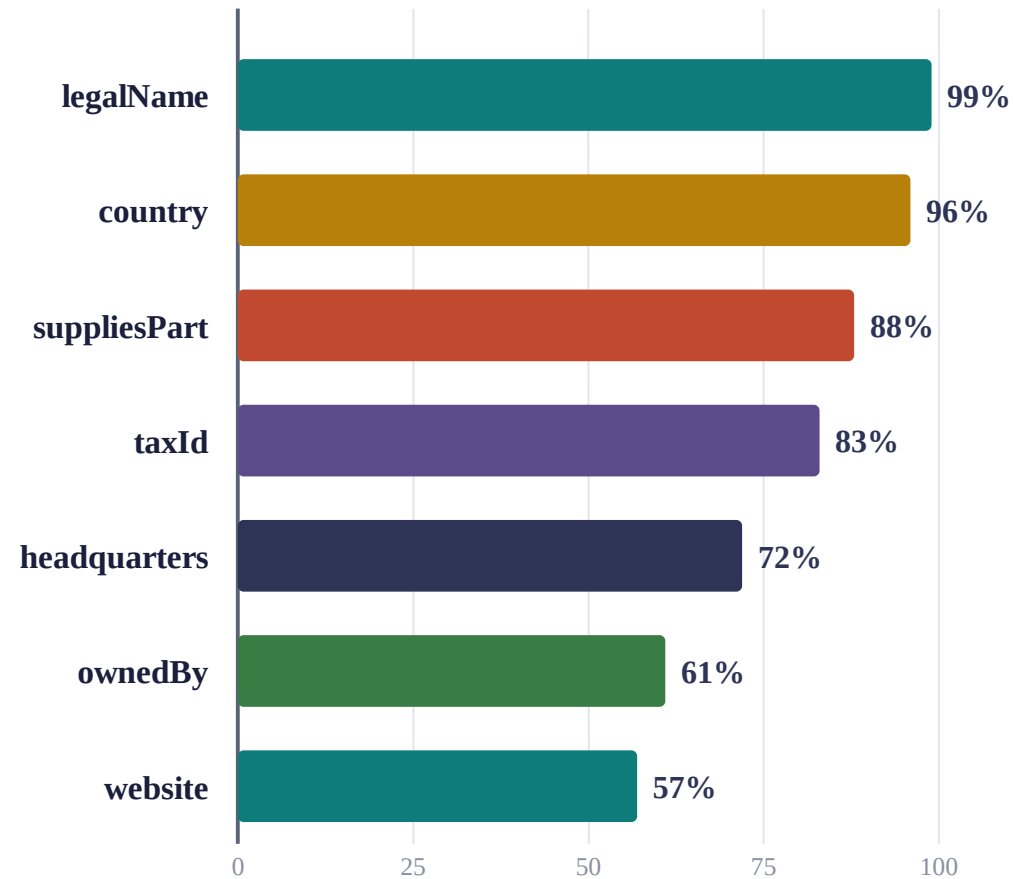
points: the 95% Wald confidence interval

- **Sample size:** n equals $1.96^2 \times p \times (1 - p)$, divided by the margin squared. With p at 0.5 and a margin of 0.05, n is 384.2, so 385.
- **Interval:** the standard error is the square root of 0.92×0.08 over 385, about 0.0138. Multiply by 1.96 to get 0.027.
- **Stratify:** sample separately per source and per predicate, because accuracy varies widely, and weight the strata by size.
- **Agreement:** measure reviewer agreement with Cohen's kappa and adjudicate disagreements before counting.

For large graphs, cluster sampling of entities reduces review cost: a reviewer checks all facts of one entity at once (Gao et al. 2019).



Profiling completeness per property



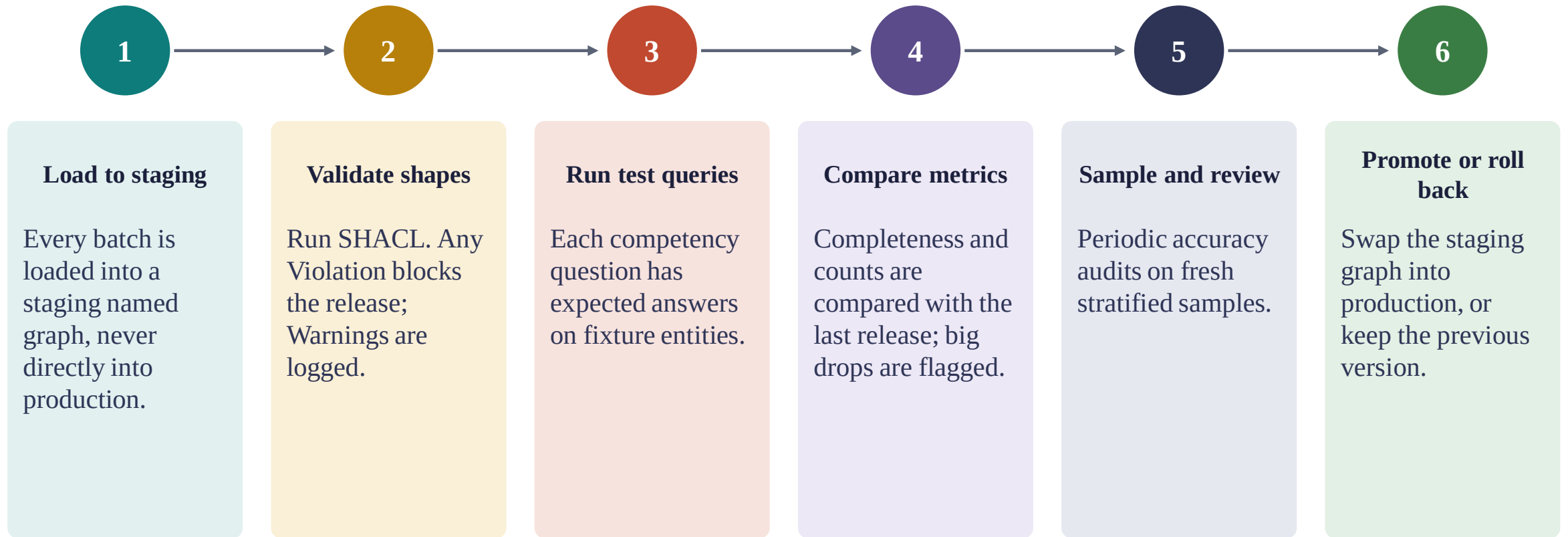
Share of the 18,400 Company nodes that have at least one value for each property (illustrative).

READING THE CHART

- **Priority:** ownedBy is only 61% complete but drives CQ2 on sanctions exposure, so it is the first gap to close.
- **Open world:** a missing value is unknown, not false. Never read absence as "no owner".
- **Relative completeness:** compare against a reference such as a company registry to estimate the true gap.
- **Action:** gaps feed back to Stage 2 (new sources) and Stage 10 (completion).
- **Query:** each bar is one SPARQL COUNT over a class and a property.



Quality as a continuous integration gate



Treat data like code: shapes, tests and thresholds live in version control and run automatically on every change.

10

STAGE 10 OF 12

Knowledge Graph Completion

Predict missing facts from the patterns already in the graph using embeddings, graph neural networks, rule mining and language models, and admit them only with confidence and review.

INPUTS

- The validated graph
- Completeness profile from Stage 9
- Held out facts for evaluation
- Compute for training

KEY ACTIVITIES

- Frame the task as link prediction
- Train embedding or GNN models
- Mine logical rules
- Evaluate with ranking metrics
- Review and admit predictions

OUTPUTS

- Scored candidate triples
- Model and rule artefacts
- Evaluation reports

Guiding question: Which true facts are missing, and how sure are we?





Why completion matters: real graphs are sparse

71%

of people in Freebase had no recorded place of birth

75%

had no recorded nationality

61%

of companies in our example lack an ownedBy link

OWA

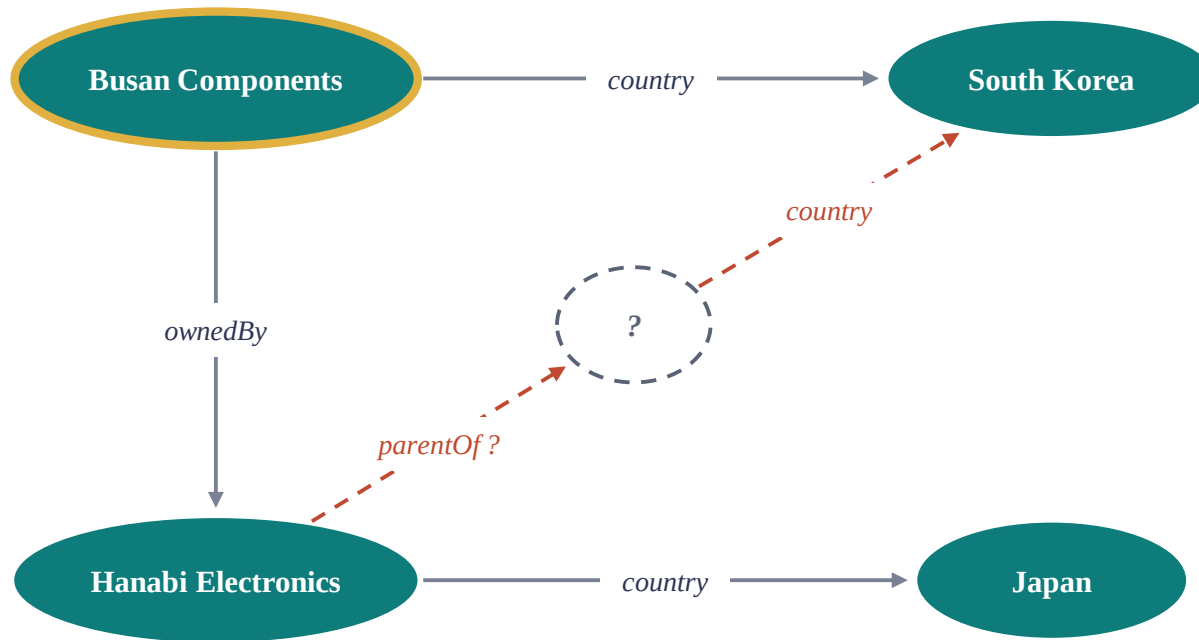
the open world assumption:
missing is unknown, not false

- **Source:** Dong et al. (2014), Knowledge Vault, measured these gaps in one of the largest public graphs of its day.
- **Two routes:** extract more from text (Stage 4), or infer from the graph's own structure. This stage covers the second.
- **Signal:** regularities such as "companies usually share the country of their headquarters city" can be learned.
- **Caution:** predictions are hypotheses. Store them apart from asserted facts, with scores and provenance.

Completion also includes type prediction, attribute value prediction and entity alignment, but link prediction is the core task.



Link prediction: filling in the question mark



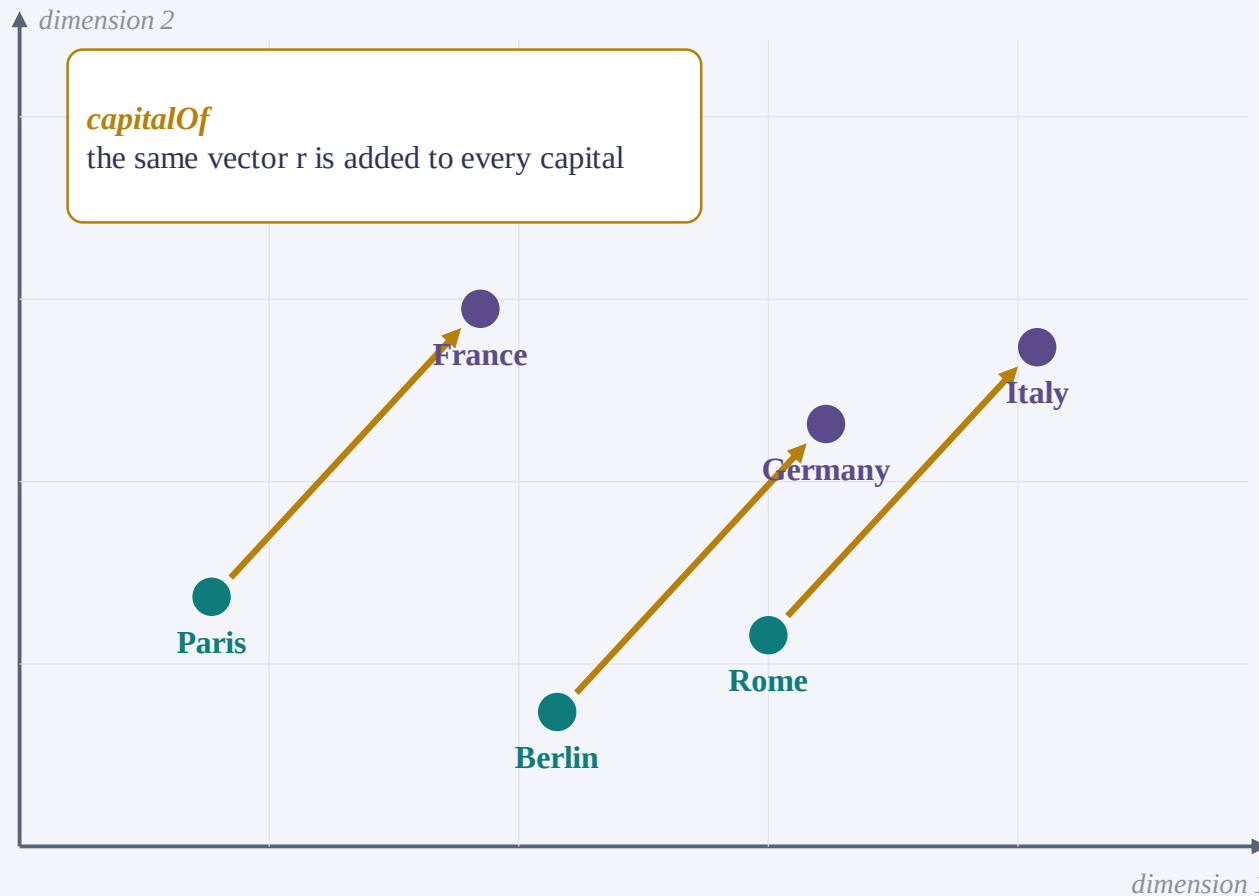
Query (Hanabi, parentOf, ?): rank every entity as a candidate tail.

ANNOTATIONS

- **Task:** given a head and a relation, rank all entities as the tail; or given relation and tail, rank heads.
- **Score:** a model assigns a plausibility score $f(h, r, t)$ to any triple, observed or not.
- **Learning signal:** observed triples should score higher than corrupted ones.
- **Here:** the inverse of `ownedBy` strongly suggests (Hanabi, `parentOf`, Busan).
- **Output:** a ranked candidate list, thresholded and sent for review.



TransE: relations as translations in vector space



HOW IT WORKS

- **Idea:** each entity and relation is a vector. For a true triple, $h + r$ lands close to t .
- **Score:** the distance between $h + r$ and t ; smaller means more plausible.
- **Prediction:** to answer $(?, \text{capitalOf}, \text{Italy})$, find the entity whose vector plus r lies nearest to Italy: Rome.
- **Origin:** Bordes et al. (2013), NeurIPS.
- **Limitation:** one to many and symmetric relations collapse, since several tails cannot all equal $h + r$.



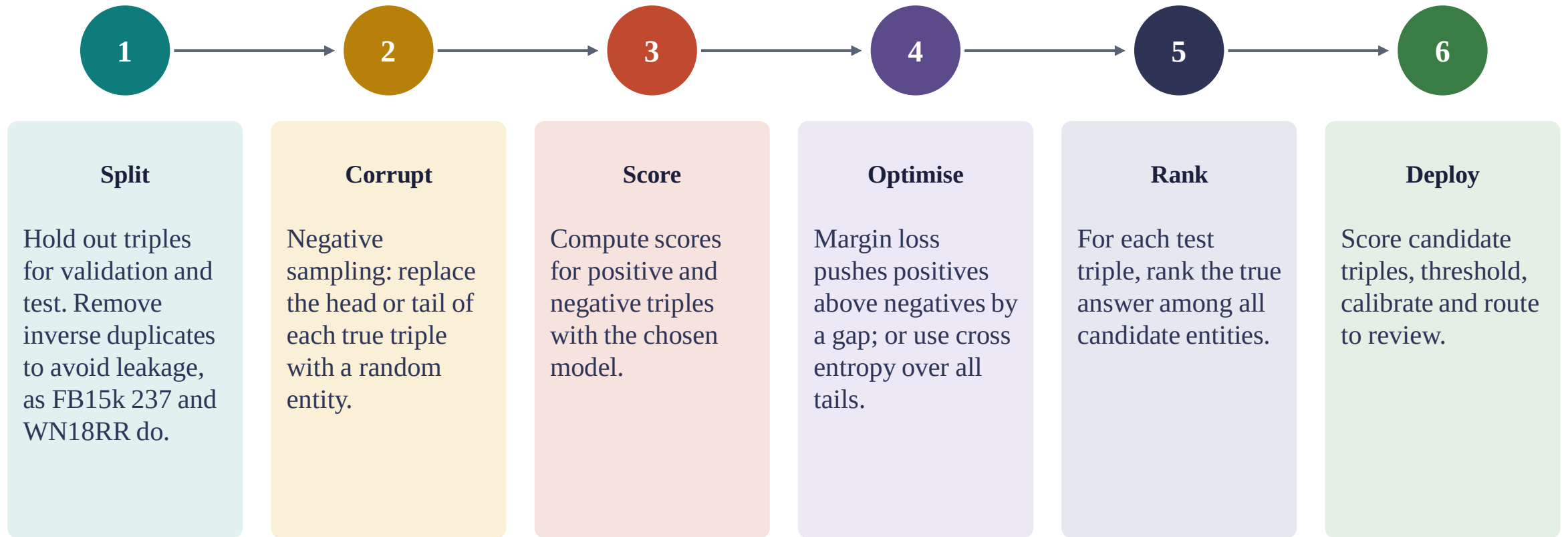
A family tree of embedding models

Model	Year	Core idea	Relation patterns handled well
TransE	2013	Relation as a translation: $h + r$ close to t	Inversion, composition; fails on symmetry and one to many
TransH	2014	Translate on a relation specific hyperplane	Better one to many and many to one
DistMult	2015	Bilinear score with a diagonal relation matrix	Symmetric relations only
ComplEx	2016	DistMult in complex space with conjugate tails	Symmetric and antisymmetric
ConvE	2018	2D convolution over reshaped head and relation	Expressive, parameter efficient
RGCN	2018	Relational graph convolution as an encoder	Uses neighbourhood context
RotatE	2019	Relation as a rotation in complex space	Symmetry, antisymmetry, inversion, composition
TuckER	2019	Tucker decomposition of the binary tensor	Fully expressive; strong baseline

In practice: with careful tuning, older models remain competitive (Ruffinelli et al. 2020). Tune before switching architecture.



Training and evaluating a completion model



Libraries: PyKEEN for reproducible experiments, DGL KE for large scale training, PyTorch Geometric for GNN models.



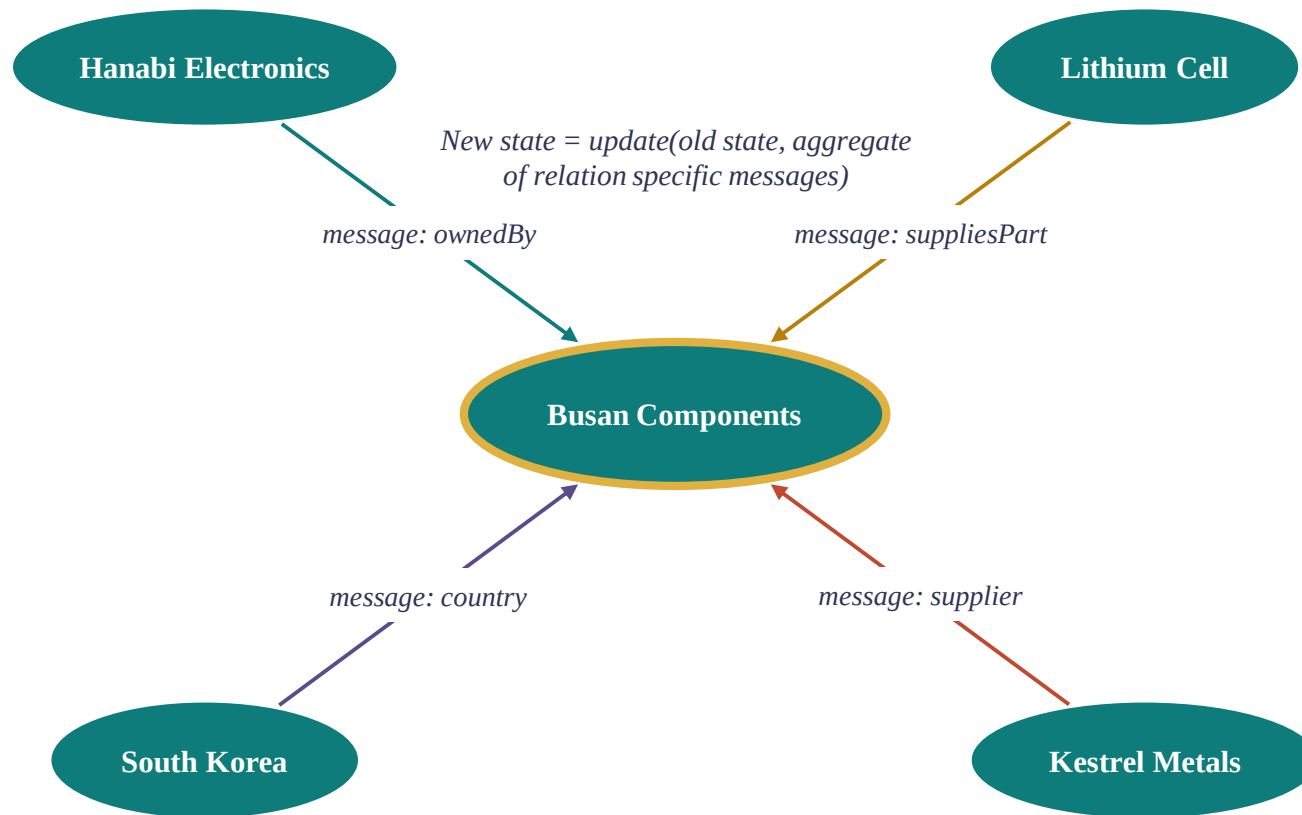
Worked example: MRR and Hits@k

Test query	True answer	Filtered rank	Reciprocal rank	Hit at 3?
(Busan Components, country, ?)	South Korea	1	1.000	Yes
(Hanabi Electronics, parentOf, ?)	Busan Components	3	0.333	Yes
(Battery Pack, hasComponent, ?)	Lithium Cell	2	0.500	Yes
(?, suppliesPart, Lithium Cell)	Kestrel Metals	10	0.100	No
Aggregate		MR 4.0	MRR 0.483	Hits@3 0.75

Filtered setting: before ranking, remove other candidates that are already known true triples, so a model is not penalised for ranking a correct answer first.



Graph neural networks: learning from neighbourhoods



ANNOTATIONS

- **Message passing:** each node gathers transformed messages from its neighbours, one weight matrix per relation.
- **Layers:** after k layers a node sees its k hop neighbourhood.
- **RGCN:** Schlichtkrull et al. (2018) introduced relation specific weights with basis sharing.
- **CompGCN:** Vashishth et al. (2020) embeds relations and nodes jointly.
- **Decoder:** GNN encodings are scored by a decoder such as DistMult to predict links.
- **Inductive:** some GNNs generalise to unseen entities, unlike plain embeddings.



Other completion methods and safeguards

1 Rule mining

AMIE and AnyBURL mine rules such as "if X ownedBy Y and Y country Z then X ultimateCountry Z" with support and confidence. Rules are explainable and auditable.

2 Text aware models

KG BERT scores triples by encoding their labels with a language model, helping sparse entities that have little structure.

3 LLM based completion

Prompt a large language model with the entity context and candidate tails. Useful for recall, but outputs must be verified against evidence.

4 Human in the loop

Predictions go to a separate named graph with score, model version and date. Reviewers accept, reject or defer; decisions become new training data.

11

STAGE 11 OF 12

Querying, Access and Applications

Expose the graph to people and systems through query languages, APIs, search, analytics and natural language interfaces, and deliver the applications that justify the investment.

INPUTS

- The production graph
- Competency questions
- User personas and access needs
- Latency and volume requirements

KEY ACTIVITIES

- Choose query languages and endpoints
- Build APIs and search
- Graph analytics
- Question answering and GraphRAG
- Application delivery

OUTPUTS

- Query endpoints and APIs
- Dashboards and applications
- AI assistants grounded in the graph

Guiding question: *How do people and machines get answers out of the graph?*





The landscape of graph query languages

Language	Data model	Standard or origin	Style	Typical engines
SPARQL 1.1	RDF	W3C Recommendation, 2013	Declarative pattern matching	GraphDB, Stardog, Virtuoso, Neptune
Cypher	Property graph	Neo4j; openCypher since 2015	Declarative ASCII art patterns	Neo4j, Memgraph, Neptune
GQL	Property graph	ISO/IEC 39075:2024	Declarative, Cypher like	Emerging support across vendors
SQL/PGQ	Property graph view	Part 16 of SQL:2023	Graph patterns inside SQL	Oracle, DuckDB extension
Gremlin	Property graph	Apache TinkerPop	Imperative traversal steps	JanusGraph, Neptune, Cosmos DB
GraphQL	Any, via schema	Facebook, 2015	API query of typed fields	Hasura, Apollo, Stardog

Milestone: GQL is the first new ISO database query language since SQL, which signals that property graphs are now mainstream.



Answering CQ1 with SPARQL

SPARQL

```
PREFIX sc: <http://example.org/supply#>

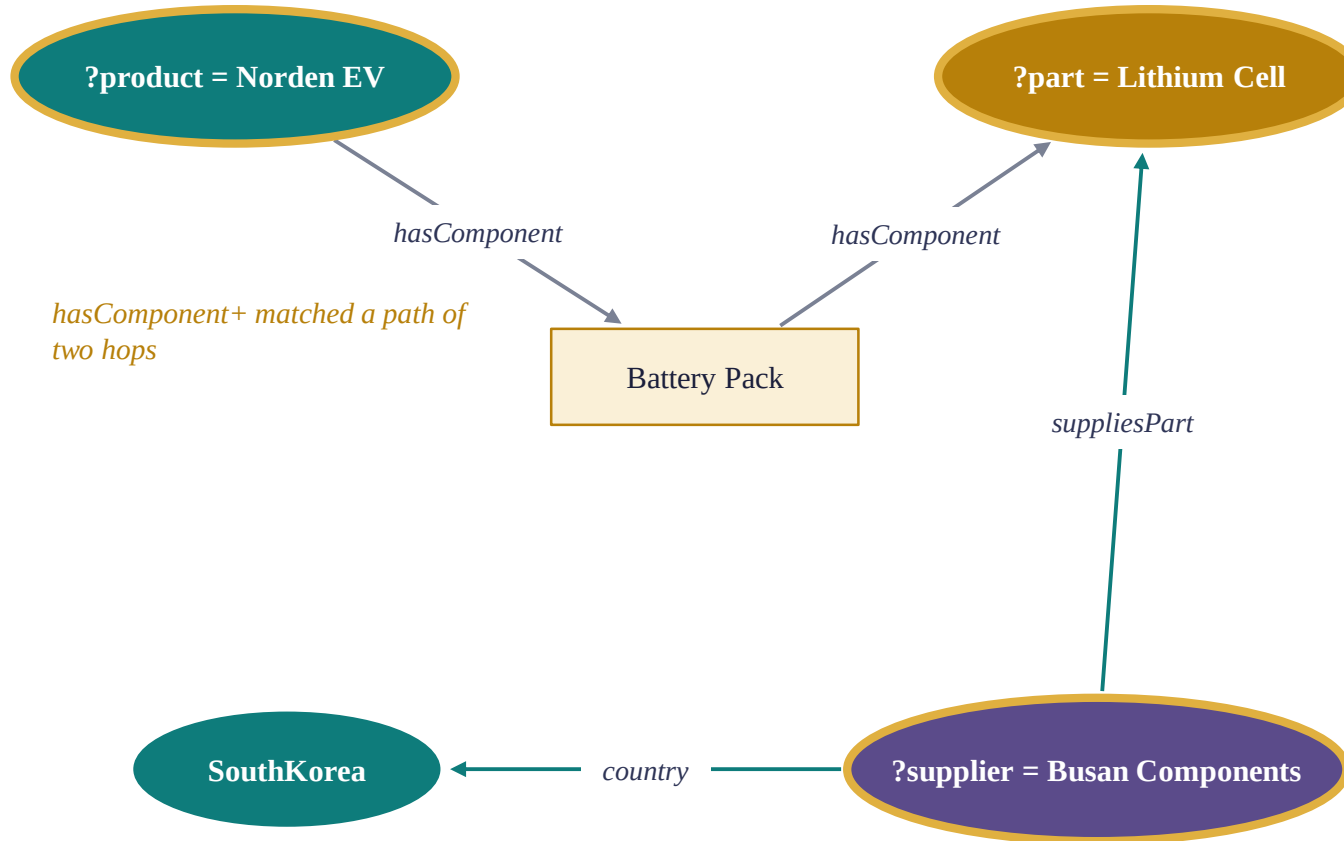
# CQ1: which products depend most
# on suppliers in a given country?
SELECT ?product
      (COUNT(DISTINCT ?supplier) AS ?n)
WHERE {
  ?product a sc:Product ;
           sc:hasComponent+ ?part .
  ?supplier sc:suppliesPart ?part ;
           sc:country sc:SouthKorea .
}
GROUP BY ?product
ORDER BY DESC(?n)
LIMIT 20
```

HOW TO READ IT

- **Basic graph pattern:** the triple patterns in WHERE must all match together.
- **Variables:** names starting with ? are bound to nodes during matching.
- **Property path:** hasComponent+ follows one or more hops through the bill of materials.
- **Aggregation:** GROUP BY and COUNT DISTINCT count suppliers per product.
- **Ranking:** ORDER BY DESC and LIMIT return the top 20 most exposed products.



How a query matches: binding variables to nodes



ANNOTATIONS

- **Solution:** one row of bindings: product, part and supplier, all consistent with every pattern.
- **Join:** shared variables such as ?part act as join keys between patterns.
- **Engine:** the optimiser orders patterns by selectivity, starting with the most restrictive.
- **Result set:** every consistent combination of bindings is one solution; aggregation then groups them.
- **Explain:** most engines show the plan, which is the first place to look when a query is slow.



Six SPARQL features every practitioner needs

1 OPTIONAL

Keep a result even when a pattern is missing, for example a supplier with no website. Essential in an open world.

2 FILTER

Restrict bindings with conditions: numeric comparisons, regex, language tags, NOT EXISTS.

3 Aggregates

COUNT, SUM, AVG, MIN, MAX and GROUP_CONCAT with GROUP BY and HAVING.

4 Property paths

Sequences, alternatives, inverses (^), and repetition with * and + for recursive traversal.

5 SERVICE

Federation: send part of a query to a remote endpoint such as Wikidata and join the answers.

6 Query forms

SELECT returns tables; CONSTRUCT returns graphs; ASK returns true or false; DESCRIBE summarises a resource.



An access architecture: from engine to end user

Applications

Risk dashboards, supplier 360 pages, alerting, chat assistants.

Semantic APIs

GraphQL or REST endpoints generated from the ontology; stable contracts for developers.

Search and NL

Full text and vector indexes on labels; question answering and GraphRAG.

Analytics

Centrality, communities, paths and embeddings exported to data science tools.

Query endpoints

SPARQL, Cypher or GQL with authentication, quotas and timeouts.

Graph store

The production graph, its replicas and caches.

KEY POINTS

- **Few users write SPARQL:** most reach the graph through APIs and applications.
- **Protect the endpoint:** time limits and quotas stop runaway recursive queries.
- **Cache:** precompute frequent answers such as supplier exposure per product.
- **Contracts:** API schemas are versioned with the ontology (Stage 12).



Knowledge graph question answering (KGQA)



Question

"Which Korean suppliers feed the Norden EV battery?"

Link entities

Map "Korean" to SouthKorea and "Norden EV battery" to Battery Pack (Stage 5 techniques).

Map relations

Map "feed" to suppliesPart and hasComponent+ using the ontology.

Generate query

Build SPARQL by templates, semantic parsing or an LLM given the schema.

Execute

Run on the endpoint; repair the query and retry if it fails or is empty.

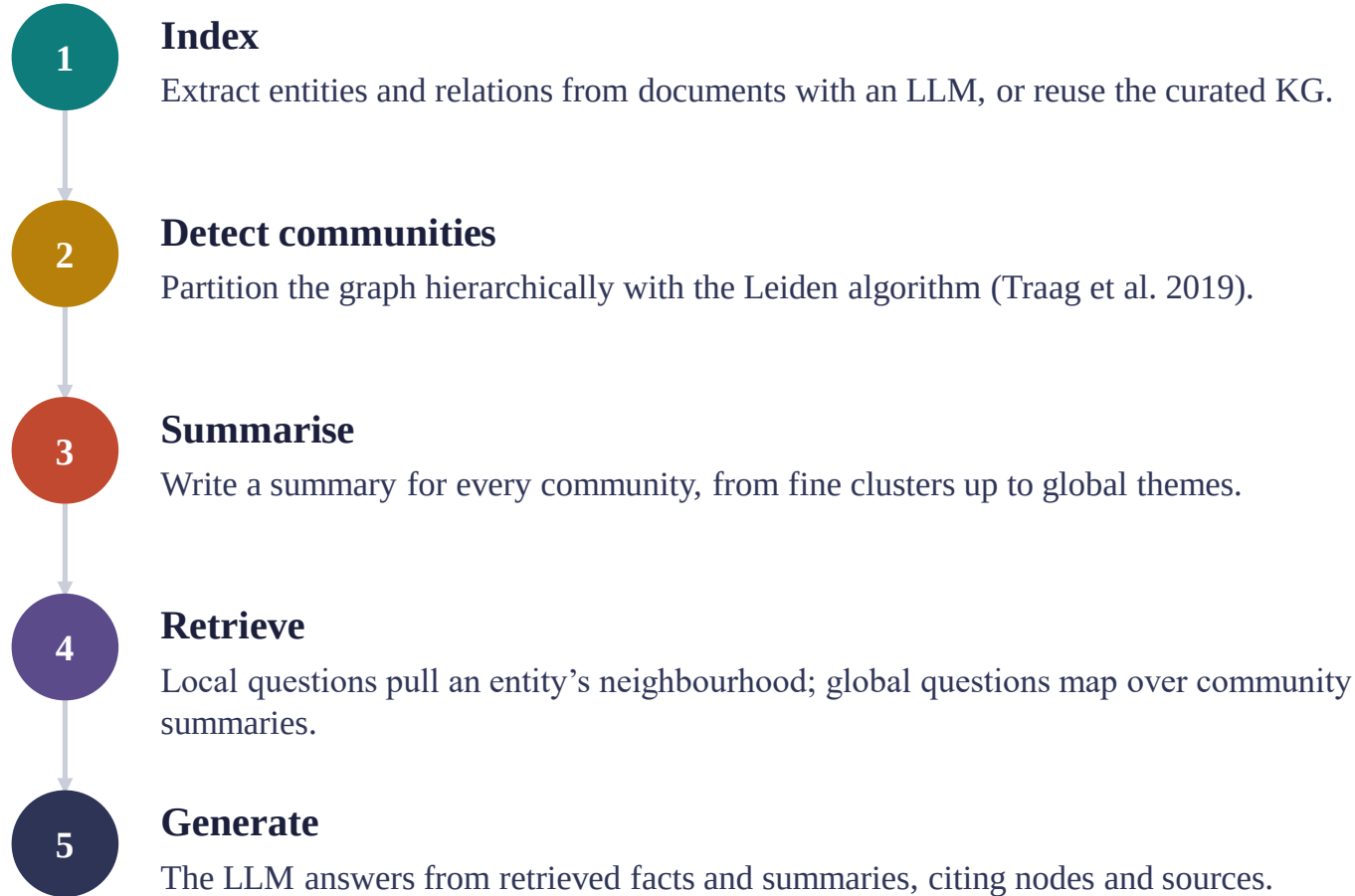
Answer

Verbalise the result with the supporting triples as evidence.

Benchmarks: LC QuAD 2.0 (Wikidata and DBpedia), the QALD challenge series, and WebQuestionsSP (Freebase).



GraphRAG: grounding language models in a graph

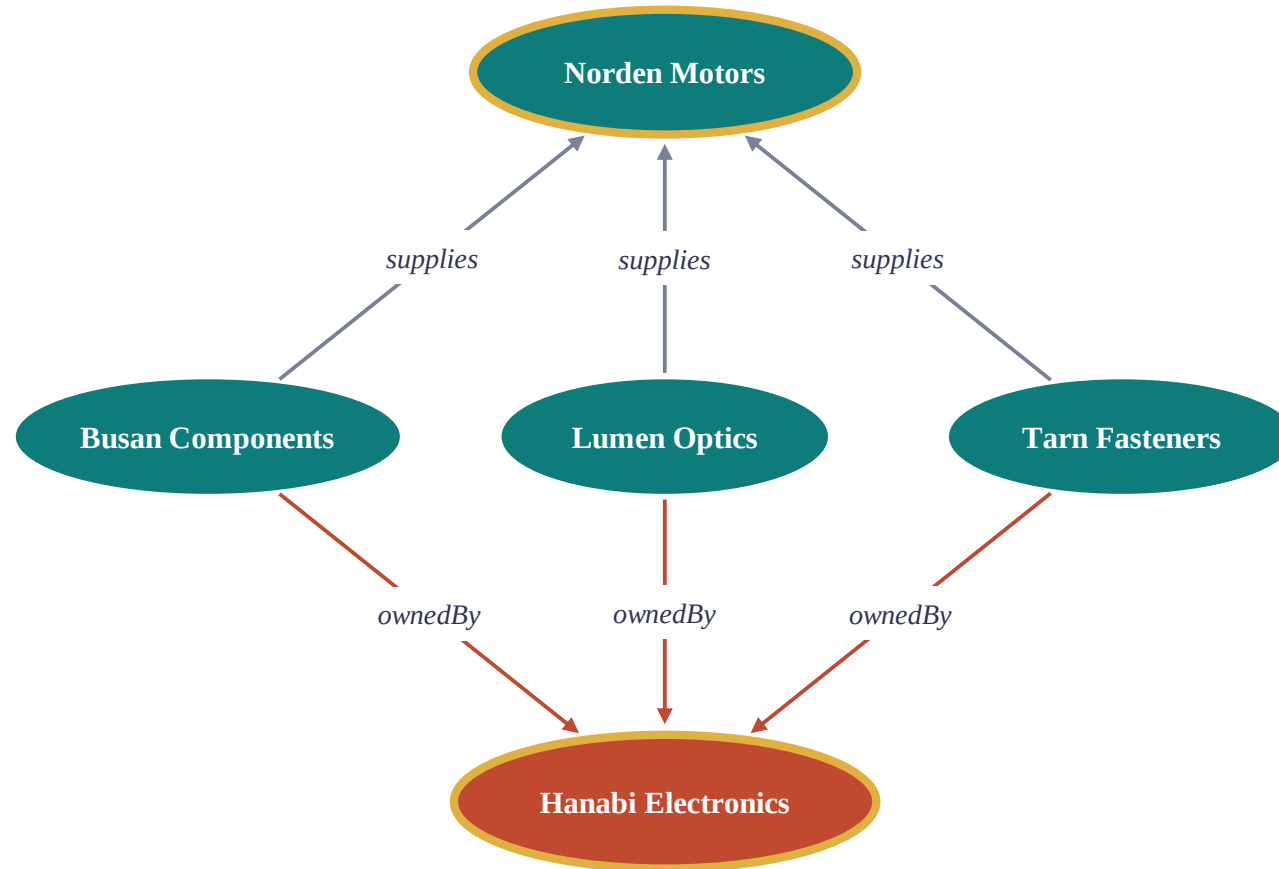


KEY POINTS

- **Origin:** Edge et al. (2024), Microsoft Research.
- **Why:** plain vector retrieval struggles with questions about a whole corpus and with multihop relations.
- **Curated KGs:** a validated graph gives cleaner retrieval than an automatically extracted one.
- **Guardrail:** require every claim in the answer to cite a triple or a source document.
- **Evaluate:** faithfulness, completeness and diversity of answers, not only fluency.



Graph analytics in action: exposing a hidden dependency



ANNOTATIONS

- **Finding:** three apparently independent suppliers share one ultimate parent, so diversification is an illusion.
- **Algorithm:** follow `ownedBy` to its end and group suppliers by ultimate owner.
- **Same pattern:** fraud rings in banking share phones, addresses and devices behind separate accounts.
- **Other analytics:** PageRank for influence, betweenness for chokepoints, Louvain or Leiden for communities.
- **Tooling:** Neo4j Graph Data Science, NetworkX, igraph, GraphFrames on Spark.

12

STAGE 12 OF 12

Maintenance, Evolution and Governance

Keep the graph current, trustworthy and secure over years: refresh data, evolve the ontology without breaking consumers, assign ownership, control access and monitor health.

INPUTS

- Production graph and pipelines
- Change requests and feedback
- Regulatory obligations
- Monitoring signals

KEY ACTIVITIES

- Refresh and change data capture
- Ontology versioning
- Ownership and stewardship
- Security and privacy
- Monitoring and feedback

OUTPUTS

- Versioned releases
- Governance policies and roles
- Health dashboards and audit logs

Guiding question: *Will the graph still be right, and trusted, a year from now?*





Four forces that make a graph decay

A knowledge graph is a living system. Without maintenance its accuracy falls every week, even if no one touches it.

1 Data drift

Sources change formats, identifiers and meanings. A renamed ERP column silently empties a property.

2 Schema evolution

New questions need new classes, such as Facility, or a split of a property into finer ones.

3 Usage evolution

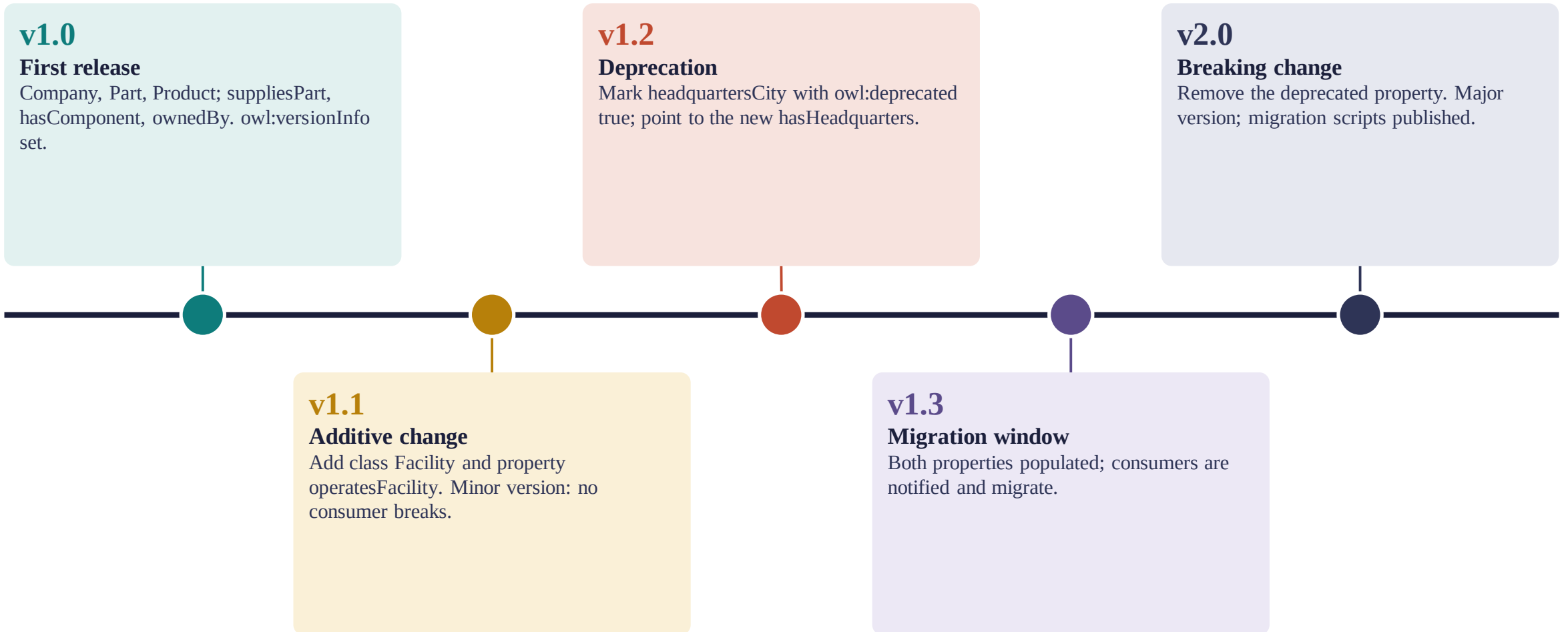
New applications arrive with latency, access and quality demands the original use case never had.

4 World change

Companies merge, are acquired, renamed, sanctioned or dissolved. Facts carry a validity period.



Ontology versioning without breaking consumers





Refresh strategies: keeping the graph current

Strategy	How it works	Freshness	Cost and complexity	Best for
Full rebuild	Recompute the whole graph from sources on a schedule	Hours to days	Simple, but heavy compute	Small graphs, early stages
Incremental batch	Load only records changed since the last run	Hours	Needs change detection and deletes	Most enterprise graphs
Change data capture	Stream database changes with Debezium into Kafka	Seconds to minutes	Streaming infrastructure	Operational systems of record
Event driven	Business events such as "contract signed" update the graph	Seconds	Needs an event model	Workflows and alerts
Virtualisation	Query sources live through mappings, as in Ontop	Always current	Slower queries	Volatile or restricted data

Deletes are the hard part: when a source record disappears, its triples, derived facts and inferences must be retracted too.



Who does what: a RACI matrix for the graph

Activity	KG product owner	Ontologist	Data engineer	Domain steward	Consumers
Prioritise competency questions	A	C	I	R	C
Change the ontology	A	R	C	C	I
Onboard a new source	A	C	R	C	I
Fix a quality violation	I	C	R	A	I
Approve predicted facts	I	I	C	A, R	I
Grant access to data	A	I	R	C	I
Report an error	I	I	C	A	R

Key: R responsible, A accountable, C consulted, I informed. Exactly one A per row keeps decisions from stalling.



Security, privacy and compliance

1 Access control

Role based (RBAC) or attribute based (ABAC) policies, enforced per named graph, class or property. Contract prices might be visible to procurement only.

2 Right to erasure

Under GDPR a person's data must be deleted everywhere: in asserted triples, derived facts, embeddings and caches. Provenance makes this traceable.

3 Inference leakage

Reasoning can reveal hidden facts: combining permitted edges may expose a restricted ownership link. Test policies against inferred results.

4 Audit

Log who queried and changed what, and when. Signed releases and immutable history support regulators and incident response.



Monitoring the health of a production graph

6 h

median freshness: age of the newest facts per source

0.4

SHACL violations per thousand triples

12,500

queries per day across APIs and endpoints

850 ms

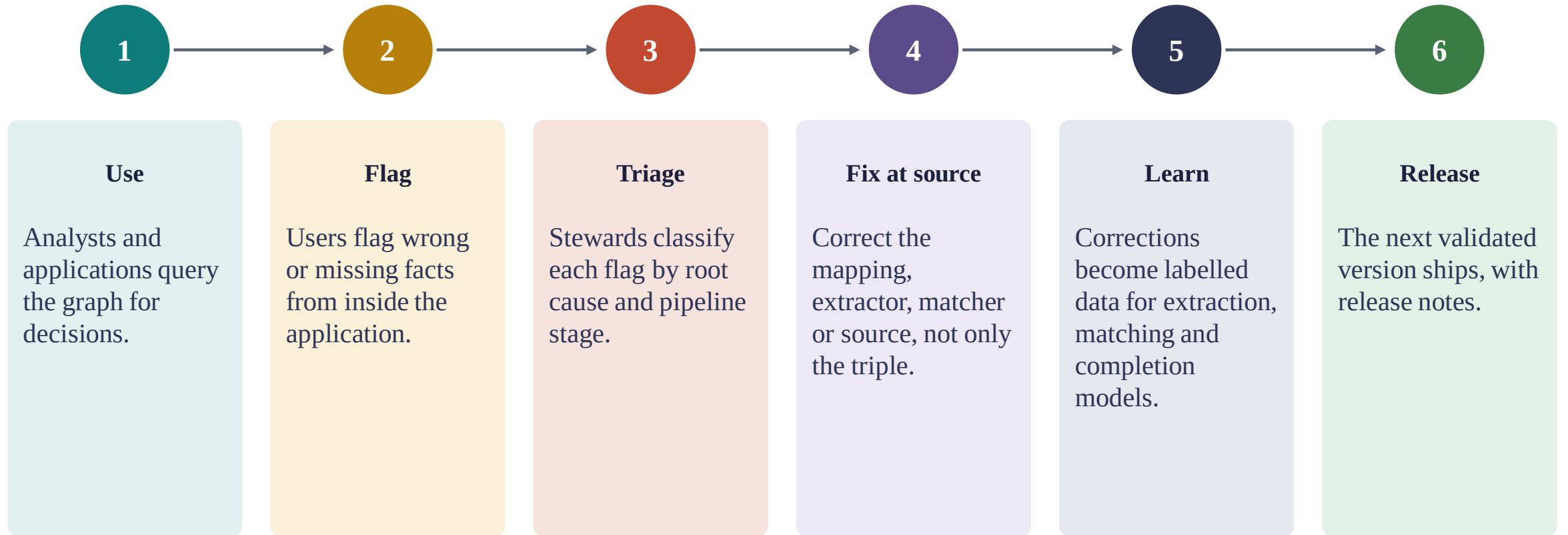
p95 query latency for dependency questions

- **Data health:** freshness, violation rate, completeness and duplicate rate, tracked per source and per release.
- **Service health:** latency percentiles, error rate, timeouts and endpoint availability.
- **Value:** active users, answered competency questions and decisions supported. This is what funds the graph.
- **Alerts:** thresholds on each signal page the owner named in the RACI matrix.

Figures are an illustrative dashboard for the supplier risk graph used throughout this tutorial.



Closing the loop: the graph improves through use



Lifecycle: governance returns to Stage 1. New questions become new requirements, and the twelve stages run again.



The FAIR principles for sustainable graphs

Wilkinson et al. (2016) proposed FAIR as guiding principles for data stewardship. Knowledge graphs are a natural way to meet them.

1 Findable

Globally unique, persistent identifiers; rich metadata; registered in a searchable catalogue.

2 Accessible

Retrievable by identifier through a standard protocol such as HTTP, with authentication where needed.

3 Interoperable

Formal, shared vocabularies and qualified links to other data, exactly what RDF and ontologies provide.

4 Reusable

Clear licence, detailed provenance and community standards, so others can trust and reuse the graph.

PART IV

Putting It All Together

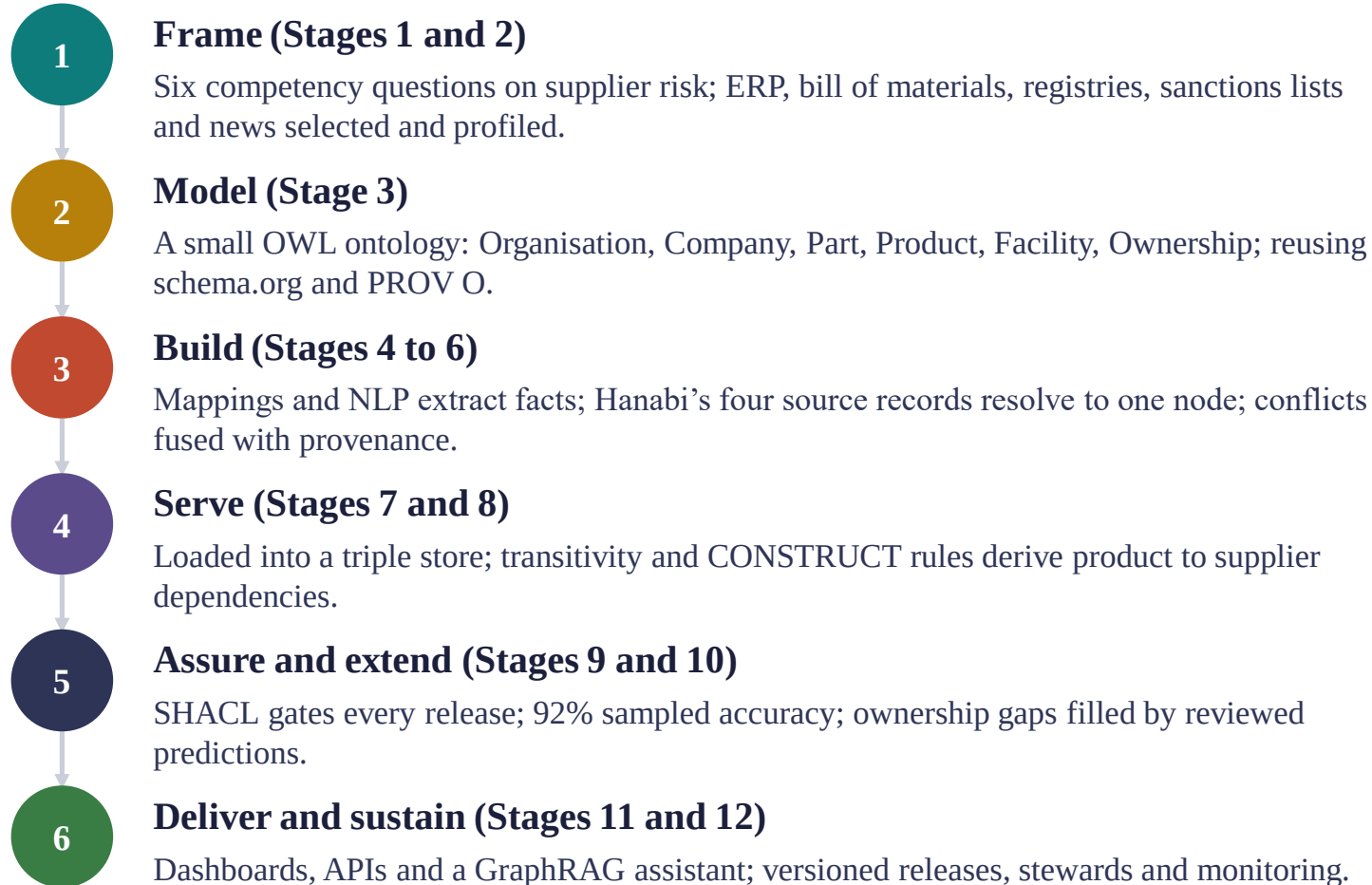
An end to end view, tools, pitfalls, maturity, and where to go next.

- The twelve stages as one delivery plan
- Tooling map per stage
- Common pitfalls and how to avoid them
- A maturity model, glossary and references





The whole journey in six phases



LESSONS

- **Start thin:** deliver one competency question end to end before widening.
- **Iterate:** each loop through the stages adds sources, classes and users.
- **Measure value:** count decisions supported, not triples loaded.
- **Outcome here:** Norden Motors discovers that three suppliers share one parent, and diversifies.



A tooling map for every stage

Stage	Open source options	Commercial or managed options
Requirements, ontology	Protégé, WebProtégé, ROBOT, OOPS!, Widoco	TopBraid EDG, PoolParty, metaphactory
Extraction and mapping	RML with Morph KGC, Ontop, spaCy, Stanza, REBEL	Diffbot, AWS Comprehend, Azure AI Language
Resolution and fusion	Splink, Dedupe, Zingg, JedAI, OpenRefine	Senzing, Tamr, Quantexa
Storage and reasoning	Apache Jena Fuseki, RDF4J, Virtuoso, Neo4j Community, Memgraph	GraphDB, Stardog, RDFox, Neo4j Enterprise, Amazon Neptune, TigerGraph
Quality	pySHACL, Jena SHACL, RDFUnit	TopBraid SHACL, Stardog ICV
Completion and ML	PyKEEN, DGL KE, PyTorch Geometric, AnyBURL	Neo4j Graph Data Science, Vertex AI
Access and apps	YASGUI, LangChain, LlamaIndex, GraphRAG	Stardog Voicebox, Neo4j Bloom, Linkurious

Choose by constraint, not fashion: standards compliance, scale, reasoning needs, team skills and total cost of ownership decide.



Eight pitfalls that sink knowledge graph projects

1 Boiling the ocean

Modelling the whole enterprise before answering one question. Start from competency questions.

2 Ontology in a vacuum

Designed without domain experts or real data. It will not fit the sources.

3 Ignoring identity

Skipping entity resolution leaves duplicates that break every count and path.

4 No provenance

Without source and date on facts, conflicts and errors cannot be traced or fixed.

5 Tool first thinking

Buying a database before knowing the queries, scale and reasoning needs.

6 Quality as an afterthought

No shapes, tests or sampling, so trust erodes after the first wrong answer.

7 Mixing guesses with facts

Predicted or LLM extracted triples stored beside asserted ones without scores.

8 No owner

A project, not a product. When the team moves on, the graph decays.



A maturity model for knowledge graph capability

Level 5: Intelligent

Completion, GraphRAG and agents act on the graph; continuous learning from feedback.

Level 4: Enterprise

Multiple domains share one ontology; governance, SLAs and stewardship are formal.

Level 3: Productised

Automated pipelines, SHACL gates, versioned releases and APIs used by several teams.

Level 2: Project

One use case in production with a curated ontology and manual refreshes.

Level 1: Exploratory

Prototype graph from a few sources; ad hoc queries by a small team.

USING THE MODEL

- **Assess:** place each capability (modelling, pipelines, quality, access, governance) on a level; the lowest limits the whole.
- **Progress:** move one level at a time; skipping levels creates fragile systems.
- **Most organisations:** sit at Level 2 or 3; Level 5 needs trustworthy data at Level 3 first.



Glossary of key terms

Term	Meaning
Triple	A statement of subject, predicate and object; the atomic unit of an RDF graph.
IRI	Internationalised Resource Identifier: a global name for a node or relation.
Ontology	A formal, explicit specification of the classes, relations and constraints of a domain.
TBox and ABox	Terminological knowledge (schema axioms) versus assertional knowledge (facts about individuals).
Property graph	A graph whose nodes and edges carry labels and key value properties.
Open world assumption	A missing statement is unknown rather than false.
Entity resolution	Deciding which records refer to the same real world entity and merging them.
Provenance	Metadata about where a fact came from, when, and how it was produced.
Materialisation	Computing and storing inferred triples ahead of query time.
SHACL	The W3C language for validating graphs against shapes.
Link prediction	Ranking candidate entities to complete a triple with a missing head or tail.
GraphRAG	Retrieval augmented generation that retrieves from a graph and its community summaries.



Key references and further reading

- Hogan, A. et al. (2021). Knowledge Graphs. ACM Computing Surveys 54(4). The most complete survey of the field.
- Paulheim, H. (2017). Knowledge graph refinement: a survey of approaches and evaluation methods. Semantic Web 8(3).
- Noy, N. and McGuinness, D. (2001). Ontology Development 101. Stanford Knowledge Systems Laboratory.
- Grüninger, M. and Fox, M. (1995). Methodology for the design and evaluation of ontologies. IJCAI Workshop.
- Fellegi, I. and Sunter, A. (1969). A theory for record linkage. Journal of the American Statistical Association 64.
- Bordes, A. et al. (2013). Translating embeddings for modeling multi relational data. NeurIPS.
- Dong, X. et al. (2014). Knowledge Vault: a web scale approach to probabilistic knowledge fusion. KDD.
- Zaveri, A. et al. (2016). Quality assessment for Linked Data: a survey. Semantic Web 7(1).
- Wilkinson, M. et al. (2016). The FAIR Guiding Principles for scientific data management and stewardship. Scientific Data 3.
- Edge, D. et al. (2024). A Graph RAG approach to query focused summarization. Microsoft Research.
- W3C Recommendations: RDF 1.1 (2014), OWL 2 (2012), SPARQL 1.1 (2013), SHACL (2017), PROV O (2013).

CLOSING

Key takeaways

A knowledge graph is a product built in stages, not a database you install.

- Start from questions: competency questions drive every design decision.
- Meaning lives in the ontology; identity lives in entity resolution.
- Provenance, validation and sampling are what make a graph trusted.
- Reasoning, completion and GraphRAG turn stored facts into new answers.
- Governance keeps the graph alive: owners, versions, feedback.



Thank you